Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase VI · Diseño y construcción de optimizadores con IA

Módulo 28

LLM como optimizadores y del prototipo a la producción

Búsqueda evolutiva de programas, optimización de sistemas con LLM, AutoML, MLOps, valor y riesgo, y guía de diseño

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Este módulo cierra la fase VI y el curso. Tiene dos mitades. La primera estudia los modelos de lenguaje como *generadores de candidatos* dentro de un bucle de optimización con un evaluador automático: programas que resuelven un problema (FunSearch y AlphaEvolve), instrucciones y cadenas de llamadas a un modelo (OPRO y DSPy), cambios de código que lo hacen más rápido y configuraciones completas de aprendizaje automático (AutoML y búsqueda de arquitecturas). La idea común es que el LLM propone y un evaluador fiable decide, de modo que la garantía la da el evaluador y no el modelo. La segunda mitad recorre el camino del prototipo a la producción: la arquitectura de un optimizador desplegado, la deuda técnica propia de los sistemas con aprendizaje automático, la vigilancia de la deriva y la evaluación de decisiones sin experimentar; después, cómo cuantificar el valor y el riesgo de un optimizador con intervalos de confianza, valor actual neto, análisis de modos de fallo y el marco normativo europeo. El último tema reúne todo el curso en una guía de doce pasos y una plantilla de documento de diseño, que se aplican a tres casos: una fábrica, un flujo de producción de contenido 3D y un centro de datos. Los casos de estudio (una heurística de empaquetado en línea generada por búsqueda evolutiva y el mantenimiento predictivo de una línea de envasado) se implementan en el cuaderno de prácticas.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. formular la búsqueda de programas como un problema de optimización con evaluador, describir los componentes de FunSearch y AlphaEvolve (esqueleto, evaluador, base de programas en islas, muestreo con temperatura) y explicar por qué funcionan y dónde fallan;
2. demostrar cotas sencillas para heurísticas de empaquetado en línea y comparar con rigor una heurística generada con *first fit* y *best fit* usando la cota L_1;
3. plantear la optimización de instrucciones y de cadenas de llamadas a un LLM (OPRO, DSPy), cuantificar el sesgo optimista del ganador sobre un conjunto de validación y elegir el número de muestras de la autoconsistencia;
4. optimizar código con un agente usando el perfilado, la ley de Amdahl, las pruebas como restricción dura y un presupuesto, y medir tiempos de ejecución sin engañarse;
5. formular AutoML como selección de algoritmo e hiperparámetros, describir las tres dimensiones de la búsqueda de arquitecturas y calcular el coste de los métodos de multifidelidad;
6. diseñar la arquitectura de un optimizador en producción (datos, características, modelos, resolvedor, API, interfaz humana), identificar la deuda técnica de un sistema con aprendizaje automático, vigilar la deriva y decidir cuándo reentrenar;

7. modelizar la fiabilidad de una máquina con la distribución de Weibull, obtener la política óptima de sustitución por edad y planificar un calendario de paradas con un modelo de riesgo y un programa entero;
8. cuantificar el valor de un optimizador con intervalos de confianza, *bootstrap* y valor actual neto, analizar sus modos de fallo y situarlo en el Reglamento europeo de inteligencia artificial;
9. aplicar una guía de doce pasos y una plantilla de documento de diseño para construir un optimizador con IA en cualquier ámbito.

Requisitos previos

Del módulo 0.9 se usan los intervalos de confianza (tema 4, §§4.3–4.4), los contrastes t (tema 5, §5.3), la potencia y el tamaño muestral (§5.4) y las comparaciones múltiples (§5.6). Del módulo 5 se usa el método de Monte Carlo (tema 5, §5.3); del módulo 6, la desigualdad de Jensen (tema 2, §2.1) y la entropía relativa (§2.2); del módulo 8, la separación en entrenamiento, validación y prueba (tema 1, §1.4) y la validación cruzada (tema 6, §6.5). Del módulo 21 se usan las recompensas verificables (tema 6, §6.2) y la sobreoptimización de la recompensa (§6.4); del módulo 22, la autoconsistencia (tema 1, §1.4, definición 1.14 y proposición 1.16), la contaminación de las evaluaciones (tema 3, §3.4) y la fiabilidad de una cadena de pasos de un agente (tema 6, §6.4). De la fase VI se reutilizan: del módulo 24, la arquitectura de un optimizador con IA (tema 1, §1.2, figura 1.2 y definición 1.8), el mapa de métodos (§1.3), la programación entera (tema 2), los algoritmos genéticos y las estrategias evolutivas (tema 5, §5.3), y la decisión con predicciones, el valor de la información perfecta y la evaluación fuera de muestra (tema 6, §§6.1 y 6.3, definición 6.6); del módulo 25, los modelos sustitutos (tema 3), la optimización bayesiana (tema 4) y el ajuste de hiperparámetros con Hyperband (tema 6); del módulo 26, el horizonte deslizante (tema 3, §3.2), la comparación de controladores con números aleatorios comunes (tema 5, §5.1), el caso de los centros de datos (§5.5) y la puesta en producción en planta: modos de operación (tema 6, §6.2), validación en sombra (§6.4), filtro de seguridad (§6.5) y normas (§6.7); del módulo 27, la selección de niveles de detalle con presupuesto (tema 5, §5.4) y el esquema de generar y verificar (tema 6, §6.3).

Organización

Tema	Contenido y caso de estudio	Fuentes principales
1	Búsqueda evolutiva de programas guiada por LLM: FunSearch y AlphaEvolve. Caso: heurística de empaquetado en línea	[44], [34], [14], [50], [23], [17]
2	Optimización de instrucciones, de sistemas con LLM y de código. Casos: clasificación de partes de avería y aceleración de un servicio	[51], [22], [36], [49], [47], [32]
3	AutoML y búsqueda de arquitecturas: optimización sobre programas, multifidelidad y coste. Caso: AutoML del clasificador de averías	[19], [48], [12], [21], [27], [52]
4	Arquitectura en producción, MLOps y evaluación fuera de línea. Caso: mantenimiento predictivo de una línea de envasado	[45], [46], [18], [2], [29], [11]
5	Cuantificar el valor y el riesgo: intervalos, *bootstrap*, VAN, modos de fallo y regulación. Caso: valoración del optimizador de mantenimiento	[33], [37], [31], [24], [42], [6]
6	Guía paso a paso y plantilla de diseño. Mini-casos: fábrica, software 3D y centro de datos	[30], [46], [5], [14], [13]

Cada tema contiene teoría formal, al menos un caso de estudio con datos numéricos, una tabla de tecnologías, una observación sobre errores frecuentes en la práctica, una lista de lecturas y ejercicios graduados en tres niveles: ⋆ comprensión, ⋆⋆ formulación y cálculo a mano, y ⋆⋆⋆ demostración o diseño razonado. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros universitarios; el enunciado se ha redactado de nuevo y, en la mayoría de los casos, se han cambiado los datos. Las soluciones completas están en el apéndice A. El cuaderno `M28-practica.ipynb` no llama a ninguna API externa: simula la búsqueda evolutiva de heurísticas con un generador de mutaciones de programas que sustituye al LLM, simula la optimización de instrucciones sobre un espacio sintético, aplica halving sucesivo a la selección del clasificador y construye el caso de mantenimiento predictivo con scikit-learn y un programa entero.

Sobre las fuentes

El texto es de redacción propia. La búsqueda de programas sigue el artículo de FunSearch de Romera-Paredes *et al.* [44] y su repositorio oficial [16], del que proceden los datos de referencia del empaquetado, y el informe técnico de AlphaEvolve de Novikov *et al.* [34] con la nota de Google DeepMind [14]; la lectura crítica de Herrmann y Pallez [17] matiza el alcance de las heurísticas generadas. Las cotas de empaquetado siguen la §3.3 de Williamson y Shmoys [50], y la representación de programas como expresiones, el capítulo «Expression Optimization» de Kochenderfer y Wheeler [23]. La optimización de instrucciones sigue a Yang *et al.* [51], Khattab *et al.* [22] y Opsahl-Ong *et al.* [36]; la maldición del optimizador, a Smith y Winkler [47]; la medida de tiempos, la clase 10 del curso 6.172 del MIT [32]. AutoML sigue el libro editado por Hutter, Kotthoff y Vanschoren [19] y los artículos originales [48, 21, 27, 3, 53, 38, 28, 41, 26], con el capítulo 19 de Zhang *et al.* [52]. La producción sigue el curso CS329S de Stanford [46], el libro de Huyen [18], Sculley *et al.* [45] y Breck *et al.* [5]; la fiabilidad, a Barlow y Proschan [2] y al curso 1.010 del MIT [29]; la evaluación contrafactual, a Dudík, Langford y Li [11]. La valoración sigue a Montgomery [33], al curso 18.05 del MIT [37], al curso 15.401 del MIT [31], a Kohavi, Tang y Xu [24] y a Rockafellar y Uryasev [43]. El Reglamento europeo de IA se cita por su texto público [42] y por la información publicada sobre su modificación de 2026 [6],

solo en términos generales; no constituye asesoramiento jurídico. La guía final adapta las tareas del curso 16.842 del MIT [30]. Las herramientas se describen según su documentación oficial [35, 22, 1, 10, 7, 8, 40, 9, 25].

Notación

Símbolo	Significado
$\mathcal{P}$, $\pi \in \mathcal{P}$	espacio de programas (o de instrucciones, configuraciones) y un elemento
$h(\pi)$, $\hat{h}(\pi)$	puntuación verdadera del programa y puntuación medida por el evaluador
q	probabilidad de que una propuesta mejore al mejor programa actual
C, s_j, r, $g = r - s$	capacidad del contenedor, tamaño del objeto j, capacidad restante y hueco tras colocar
$L_1 = \lceil \sum_j s_j / C \rceil$	cota inferior del número óptimo de contenedores
K, n	número de candidatos comparados y tamaño del conjunto de validación
η, $r_{\min}$, $r_{\max}$	factor de reducción y presupuestos mínimo y máximo de la multifidelidad
$R(t)$, $F(t)$, $f(t)$, $\lambda(t)$	fiabilidad, función de distribución, densidad y tasa de fallo de una vida útil
β, θ	parámetros de forma y de escala de la distribución de Weibull
c_p, c_f	coste de una intervención preventiva y de una avería
PSI, KL	índice de estabilidad poblacional y divergencia de Kullback–Leibler
VAN, ρ	valor actual neto y tipo de descuento anual
CVaR_α	valor en riesgo condicional al nivel α
€, h, d, mm/s, ºC	euros, horas, días, milímetros por segundo (vibración) y grados Celsius

Índice

Presentación del módulo **ii**

1 Búsqueda evolutiva de programas guiada por LLM **1**
1.1 Optimizar en el espacio de programas . . . 1
1.2 FunSearch . . . 2
1.3 AlphaEvolve . . . 4
1.4 Por qué funciona y cuándo no . . . 4
1.5 Empaquetado en línea . . . 5
1.6 Caso de estudio: una heurística de empaquetado generada . . . 6
Ejercicios del tema 1 . . . 8

2 Optimización de instrucciones, de sistemas con LLM y de código **10**
2.1 Instrucciones como variables de decisión . . . 10
2.2 El sesgo del ganador . . . 11
2.3 Caso de estudio: clasificación de partes de avería . . . 12
2.4 La autoconsistencia como problema de optimización . . . 12
2.5 Optimización de código con agentes . . . 13
Ejercicios del tema 2 . . . 15

3 AutoML y búsqueda de arquitecturas **17**
3.1 AutoML como optimización sobre programas . . . 17
3.2 Búsqueda de arquitecturas neuronales . . . 18
3.3 Multifidelidad y coste . . . 18
3.4 Caso de estudio: AutoML del clasificador de averías . . . 19
Ejercicios del tema 3 . . . 21

4 Del prototipo a la producción: arquitectura, MLOps y evaluación **23**
4.1 Arquitectura de un optimizador en producción . . . 23
4.2 Deuda técnica en sistemas con aprendizaje automático . . . 24
4.3 Deriva y reentrenamiento . . . 25
4.4 Evaluar decisiones sin experimentar . . . 26
4.5 Fiabilidad y mantenimiento . . . 26
4.6 Caso de estudio: mantenimiento predictivo de una línea de envasado . . . 27
Ejercicios del tema 4 . . . 30

5 Cuantificar el valor y el riesgo **32**
5.1 Coste y beneficio . . . 32
5.2 Intervalos de confianza para la mejora . . . 32
5.3 Caso de estudio: ¿cuánto vale el optimizador de mantenimiento? . . . 33
5.4 Robustez y riesgo de cola . . . 34
5.5 Seguridad, supervisión y modos de fallo . . . 35
5.6 El Reglamento europeo de inteligencia artificial . . . 35
Ejercicios del tema 5 . . . 37

6 Guía para construir un optimizador con IA en cualquier ámbito **39**
6.1 Los doce pasos . . . 39
6.2 Plantilla de documento de diseño . . . 40
6.3 Mini-caso A: secuenciación de formatos en una fábrica de bebidas . . . 40
6.4 Mini-caso B: presupuesto de geometría en un videojuego . . . 41
6.5 Mini-caso C: asignación de trabajos en un centro de datos . . . 42
Ejercicios del tema 6 . . . 43

A Soluciones de los ejercicios **45**
A.1 Tema 1 . . . 45
A.2 Tema 2 . . . 49
A.3 Tema 3 . . . 52
A.4 Tema 4 . . . 55
A.5 Tema 5 . . . 58
A.6 Tema 6 . . . 61

Bibliografía **65**

Tema 1

Búsqueda evolutiva de programas guiada por LLM

En muchos problemas lo que se busca no es un número ni un vector, sino un *procedimiento*: una regla de prioridad para un planificador, una heurística de empaquetado, un núcleo de cálculo más rápido. Este tema estudia cómo un modelo de lenguaje, encerrado en un bucle evolutivo con un evaluador automático, puede buscar en el espacio de programas. Sigue el artículo de FunSearch de Romera-Paredes *et al.* [44] y su implementación de referencia [16], el informe de AlphaEvolve de Novikov *et al.* [34] y la nota técnica de Google DeepMind [14]; para la representación de programas como expresiones se apoya en el capítulo «Expression Optimization» de Kochenderfer y Wheeler [23], y para el empaquetado, en la §3.3 de Williamson y Shmoys [50]. La lectura crítica de Herrmann y Pallez [17] sirve de contrapeso.

1.1 Optimizar en el espacio de programas

Definición 1.1 (Búsqueda de programas). Sea $\mathcal{P}$ el conjunto de programas que respetan una firma dada (por ejemplo, funciones que reciben un objeto y la lista de contenedores abiertos y devuelven una puntuación por contenedor). Sea $\mathcal{D}$ una distribución de instancias del problema y $u(\pi, x) \in \mathbb{R} \cup \{-\infty\}$ la utilidad de ejecutar π sobre la instancia x, con $u = -\infty$ si el programa falla, excede el tiempo o produce una salida inválida. La *búsqueda de programas* es

$$\max_{\pi \in \mathcal{P}} \; h(\pi), \qquad h(\pi) = \mathbb{E}_{x \sim \mathcal{D}}\big[u(\pi, x)\big].$$

Como $\mathcal{D}$ no se conoce, se usa un *evaluador* $\hat{h}(\pi) = \frac{1}{m}\sum_{i=1}^{m} u(\pi, x_i)$ sobre instancias de entrenamiento $x_1, \ldots, x_m$, y la calidad final se mide con instancias de prueba independientes.

El problema es de caja negra en el sentido de la definición 5.1 del módulo 24: no hay gradiente, el espacio es discreto y enorme, y cada evaluación cuesta una ejecución. La programación genética clásica busca en ese espacio con mutaciones y cruces aleatorios de árboles sintácticos [23]; su dificultad es que casi todas las mutaciones aleatorias producen programas inútiles. La idea de FunSearch es sustituir la mutación aleatoria por un modelo de lenguaje entrenado con código, que propone modificaciones *plausibles*.

Observación 1.2 (Por qué buscar programas y no soluciones). (i) Un programa *generaliza*: una heurística de empaquetado sirve para instancias nuevas, mientras que un empaquetado concreto solo sirve para la suya. (ii) Un programa corto puede *describir* un objeto enorme: en el problema del *cap set*, FunSearch buscó programas que construyen el conjunto, no el conjunto [44]. (iii) Un programa se puede *leer*: el experto puede entender, simplificar y auditar la regla encontrada. (iv) Un programa se puede *verificar*: el evaluador lo ejecuta y comprueba la validez de la salida.

Definición 1.3 (Esqueleto y función que evoluciona). Un *esqueleto* es un programa fijo que resuelve el problema llamando a una función con firma conocida, la única que se modifica durante la búsqueda. En el empaquetado en línea, el esqueleto recorre los objetos en orden de llegada, calcula los contenedores en los que cabe cada uno y coloca el objeto en el de mayor `prioridad(objeto, contenedores)`; solo `prioridad` evoluciona.

El esqueleto encierra el conocimiento del problema y garantiza la factibilidad (un objeto nunca se coloca donde no cabe), de modo que cualquier función que devuelva números produce una solución válida. Es el mismo principio de diseño que el filtro de seguridad del módulo 26 (tema 6, §6.5): la parte aprendida propone dentro de un marco que impide lo inaceptable.

1.2 FunSearch

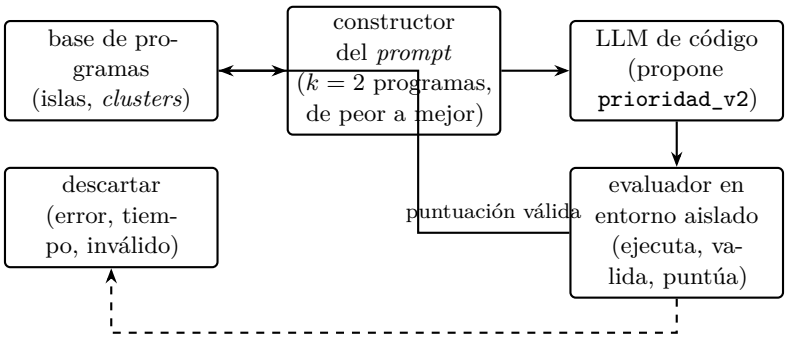


Figura 1.1: Bucle de FunSearch. El LLM nunca ve el evaluador: solo ve programas anteriores ordenados por su puntuación.

El bucle de la figura 1.1 se ejecuta de forma asíncrona y distribuida. Según el artículo y la implementación publicada [44, 16]:

1. *Constructor del prompt.* Se elige una isla al azar y, dentro de ella, $k = 2$ programas; se ordenan de peor a mejor puntuación, se renombran como `prioridad_v0` y `prioridad_v1`, y el prompt termina con la cabecera de `prioridad_v2`, «versión mejorada de la v1». A esta técnica los autores la llaman *best-shot prompting*.

2. *Muestreo.* Un LLM preentrenado con código (en el trabajo original, un modelo de la familia PaLM 2; en la actualización de 2024, Gemini 1.5 Flash [15]) completa el cuerpo de la nueva función. La configuración por defecto de la implementación usa 15 muestreadores que piden 4 continuaciones por prompt.

3. *Evaluación.* El programa se ejecuta en un entorno aislado sobre las instancias de entrenamiento; si falla, tarda demasiado o produce algo inválido, se descarta. La configuración por defecto usa 140 evaluadores en paralelo.

4. *Base de programas.* Los programas válidos se guardan en 10 islas que evolucionan por separado. Dentro de cada isla se agrupan en *clusters* por su *firma* (el vector de puntuaciones en cada instancia de entrenamiento). Periódicamente se reinicia la mitad de las islas con peor mejor puntuación, sembrando cada una con el mejor programa de una isla superviviente elegida al azar.

Definición 1.4 (Muestreo de clusters con temperatura). Sean $s_1, \dots, s_c$ las puntuaciones de los clusters de una isla y $T > 0$ una temperatura. Se elige el cluster i con probabilidad

$$p_i(T) = \frac{\exp(s_i/T)}{\sum_{l=1}^{c} \exp(s_l/T)}.$$

En la implementación de referencia, T decrece linealmente con el número n de programas registrados en la isla, $T = T_0\big(1 - (n \bmod P)/P\big)$, con $T_0 = 0{,}1$ y periodo $P = 30\,000$; dentro del cluster elegido se prefieren los programas más cortos, con una softmax sobre la longitud normalizada cambiada de signo [16].

Proposición 1.5 (Límites de la temperatura). *Sea $M = \max_i s_i$ y $A = \{i : s_i = M\}$. Cuando $T \to 0^+$, $p_i(T) \to 1/|A|$ si $i \in A$ y $p_i(T) \to 0$ en otro caso. Cuando $T \to \infty$, $p_i(T) \to 1/c$ para todo i.*

Demostración. Dividiendo numerador y denominador por $\exp(M/T)$, $p_i(T) = \exp\big((s_i - M)/T\big) \big/ \sum_l \exp\big((s_l - M)/T\big)$. Si $T \to 0^+$, cada término con $s_l < M$ tiende a 0 y cada término con $s_l = M$ vale 1, luego el denominador tiende a $|A|$ y el numerador a $\mathbf{1}[i \in A]$. Si $T \to \infty$, todos los exponentes tienden a 0, cada término tiende a 1 y $p_i \to 1/c$. ∎

La temperatura regula, pues, el equilibrio entre *explotar* los mejores programas y *explorar* otros que pueden ser punto de partida de mejoras mayores; es la misma distribución de Boltzmann del recocido simulado (módulo 24, tema 5, §5.2). Las islas y los reinicios cumplen una función parecida a otra escala: mantienen poblaciones que no se contaminan entre sí y, de vez en cuando, abandonan las que se han estancado.

Proposición 1.6 (Elitismo y espera de una mejora). *Sea b_t la mejor puntuación de la base de programas tras t evaluaciones, en un algoritmo que nunca elimina el mejor programa. Entonces:*

(i) $b_{t+1} \geq b_t$ *para todo t.*

(ii) Si, mientras no haya mejora, cada propuesta supera a b_t con probabilidad al menos $q > 0$, de forma independiente, el número N de evaluaciones hasta la primera mejora cumple $\mathbb{E}[N] \leq 1/q$ y $\mathbb{P}(N > B) \leq (1-q)^B \leq e^{-qB}$.

(iii) Si I islas independientes tienen cada una probabilidad p de alcanzar una puntuación objetivo con su presupuesto, la probabilidad de que la alcance alguna es $1 - (1-p)^I$.

Demostración. (i) El mejor programa permanece en la base y $b_{t+1} = \max(b_t, \hat{h}(\pi_{t+1}))$. (ii) El suceso $\{N > B\}$ exige B fracasos seguidos, cada uno con probabilidad a lo sumo $1 - q$, luego $\mathbb{P}(N > B) \leq (1-q)^B$; como $1 - q \leq e^{-q}$, se tiene $(1-q)^B \leq e^{-qB}$. Además $\mathbb{E}[N] = \sum_{B \geq 0} \mathbb{P}(N > B) \leq \sum_{B \geq 0} (1-q)^B = 1/q$. (Los reinicios de islas no eliminan el mejor programa global, que sigue en alguna isla.) (iii) Ninguna lo alcanza con probabilidad $(1-p)^I$ por independencia. ∎

La proposición explica el papel del LLM con una sola cifra: el presupuesto necesario es del orden de $1/q$. Una mutación aleatoria de un árbol sintáctico tiene una q minúscula; un modelo que ha visto millones de programas concentra la probabilidad en ediciones que tienen sentido (cambiar un umbral, añadir un caso, simplificar una expresión), y eso multiplica q.

1.3 AlphaEvolve

AlphaEvolve [34] generaliza FunSearch de función a sistema. El usuario marca con comentarios (`EVOLVE-BLOCK-START` y `EVOLVE-BLOCK-END`) los bloques de un fichero que pueden cambiar; el resto actúa como esqueleto. Los LLM devuelven *diferencias* en formato de búsqueda y sustitución en lugar de reescribir el fichero, lo que permite evolucionar programas de cientos de líneas en cualquier lenguaje. Un conjunto de modelos combina uno rápido (Gemini 2.0 Flash, muchas propuestas) y uno más capaz (Gemini 2.0 Pro, menos propuestas y de más calidad). La evaluación es una *cascada*: el programa pasa por pruebas de dificultad creciente y solo avanza si supera las anteriores, de modo que los candidatos malos se descartan con poco coste. La base de programas combina MAP-Elites (una población que guarda el mejor programa en cada celda de un espacio de características, para mantener diversidad) con el modelo de islas. El cuadro 1.1 resume las diferencias que los propios autores señalan.

Cuadro 1.1: FunSearch frente a AlphaEvolve, según la tabla 1 de [34].

	FunSearch	AlphaEvolve
qué evoluciona	una función	un fichero entero, con varios bloques
tamaño	10–20 líneas	hasta cientos de líneas
lenguaje	Python	cualquiera
evaluación	≤ 20 min en una CPU	horas, en paralelo, con aceleradores
muestras del LLM	millones	bastan miles

Ejemplo 1.7 (Resultados publicados de AlphaEvolve). Según el informe y la nota técnica de Google DeepMind [34, 14]: (i) una heurística de planificación para Borg, el gestor de los centros de datos de Google, recupera de media un 0,7 % de los recursos de cómputo de toda la flota; (ii) una mejor división de una multiplicación de matrices acelera un 23 % ese núcleo y reduce un 1 % el tiempo de entrenamiento de Gemini; (iii) un núcleo de FlashAttention se acelera hasta un 32,5 %; (iv) en más de 50 problemas matemáticos abiertos, igualó el mejor resultado conocido en torno al 75 % de los casos y lo mejoró en torno al 20 %, entre ellos una configuración de 593 esferas en dimensión 11 para el problema del número de besos; (v) encontró un procedimiento para multiplicar matrices complejas 4×4 con 48 multiplicaciones escalares. En todos los casos la garantía procede del evaluador: una prueba de corrección, una medida de tiempo o la verificación de una construcción matemática.

1.4 Por qué funciona y cuándo no

Tres ingredientes explican el éxito. *Asimetría entre generar y verificar*: comprobar que un empaquetado es válido y contar contenedores es inmediato, aunque encontrar una buena heurística sea difícil; es la misma asimetría que explota el aprendizaje por refuerzo con recompensas verificables (módulo 21, tema 6, §6.2) y el esquema de generar y verificar del módulo 27 (tema 6, §6.3). *Una distribución de propuestas informada*: el LLM eleva la probabilidad q de la proposición 1.6. *Memoria y diversidad*: la base de programas acumula lo aprendido y las islas evitan la convergencia prematura.

Los límites son igual de claros. (i) *El evaluador es la especificación*: lo que no mida no se optimiza, y lo que mida mal se explota, como en la sobreoptimización de la recompensa (módulo 21, tema 6, §6.4). (ii) *Generalización*: la heurística se ajusta a la distribución de entrenamiento; Herrmann y Pallez [17] concluyen que las heurísticas de empaquetado de

FunSearch no son robustas a cambios en la distribución de los datos y que en gran parte redescubren patrones conocidos. (iii) *Coste*: millones de muestras en FunSearch y horas de evaluación en AlphaEvolve. (iv) *Seguridad*: el código generado es código no confiable y debe ejecutarse aislado, sin red ni acceso a datos sensibles.

1.5 Empaquetado en línea

Definición 1.8 (Empaquetado unidimensional en línea). Se reciben, uno a uno, objetos de tamaños $s_1, s_2, \ldots, s_N \in (0, C]$ que deben colocarse en contenedores de capacidad C sin superarla. Cada objeto se coloca al llegar, sin conocer los siguientes, y no se puede mover después. El objetivo es minimizar el número de contenedores usados. Sea OPT el mínimo que se lograría conociendo todos los objetos de antemano.

Definición 1.9 (Reglas de prioridad clásicas). Sea r la capacidad restante de un contenedor abierto en el que cabe el objeto s y $g = r - s$ el hueco que dejaría. *First fit* (FF) coloca el objeto en el primer contenedor abierto en que cabe; *best fit* (BF), en el que deja menor hueco g; en ambos casos se abre uno nuevo solo si no cabe en ninguno. En términos del esqueleto, FF usa la prioridad $-$(índice del contenedor) y BF la prioridad $-g$. Un algoritmo es de tipo *any fit* si nunca abre un contenedor nuevo cuando el objeto cabe en uno abierto.

Lema 1.10 (Cota L_1). $\mathrm{OPT} \geq L_1 = \lceil \sum_j s_j / C \rceil$.

Demostración. Con m contenedores se puede guardar un volumen total de a lo sumo mC, luego $\mathrm{OPT} \cdot C \geq \sum_j s_j$; como OPT es entero, $\mathrm{OPT} \geq \lceil \sum_j s_j / C \rceil$. ■

Teorema 1.11 (Garantía de los algoritmos *any fit*). *Todo algoritmo* any fit*, en particular FF y BF, usa a lo sumo* $\lceil 2 \sum_j s_j / C \rceil \leq 2\,\mathrm{OPT}$ *contenedores.*

Demostración. Sea m el número de contenedores usados y $S = \sum_j s_j$. Se afirma que, al final, a lo sumo un contenedor tiene carga $\leq C/2$. Supóngase que dos contenedores i y l, con i abierto antes que l, tienen ambos carga final $\leq C/2$. Cuando llegó el primer objeto de l, de tamaño $s \leq C/2$ (porque la carga final de l no supera $C/2$), el contenedor i ya estaba abierto y su carga era $\leq C/2$ (las cargas solo crecen). Entonces el objeto cabía en i, y un algoritmo *any fit* no habría abierto l: contradicción. Por tanto, al menos $m - 1$ contenedores tienen carga $> C/2$ y $S > (m-1)C/2$, es decir, $m < 2S/C + 1$. Como m es entero, $m \leq \lceil 2S/C \rceil$. Finalmente $\lceil 2S/C \rceil \leq 2\lceil S/C \rceil = 2L_1 \leq 2\,\mathrm{OPT}$ por el lema 1.10. ■

La constante 2 no es la mejor posible para FF, pero el resultado ilustra el tipo de garantía que una heurística generada *no* trae consigo: lo que se sabe de ella es lo que mide el evaluador. Para el problema sin restricciones de llegada, Williamson y Shmoys demuestran que, salvo que P = NP, ningún algoritmo de tiempo polinómico garantiza un factor menor que 3/2 [50, §3.3].

Ejemplo 1.12 (FF frente a BF a mano). Con $C = 10$ llegan los objetos $5, 8, 6, 2, 4, 4, 7$ ($S = 36$, $L_1 = 4$). *FF*: $5 \to$ contenedor 1; $8 \to 2$; $6 \to 3$; $2 \to 1$ (carga 7); el primer 4 no cabe en 1 (resto 3) ni en 2 (resto 2), va al 3 (carga 10); el segundo 4 no cabe en ninguno y abre el 4; el 7 abre el 5. Total: 5 contenedores. *BF*: el 2 cabe en 1, 2 y 3 con huecos 3, 0 y 2, y va al 2 (carga 10); el primer 4 cabe en 1 (hueco 1) y en 3 (hueco 0), va al 3; el segundo 4 va al 1 (carga 9); el 7 abre el 4. Total: 4 contenedores, que es óptimo por el lema 1.10. BF ganó porque cerró contenedores exactos y no dejó restos de 2 o 3 unidades inservibles.

1.6 Caso de estudio: una heurística de empaquetado generada

Ejemplo 1.13 (Búsqueda evolutiva de una regla de prioridad). *Problema.* Un almacén automático agrupa paquetes en bandejas de capacidad $C = 100$ (unidades de volumen) a medida que llegan de la línea; las bandejas no se reabren. Como en los conjuntos de datos Weibull de FunSearch [16, 17], los tamaños siguen una Weibull de forma 3 y escala 45, truncados a enteros entre 1 y 100 (media $\approx 39{,}7$).

Formulación. Programas $\pi(s, r)$ que asignan una prioridad a cada bandeja abierta con $r \geq s$ y a una bandeja vacía ($r = C$); el esqueleto coloca el objeto en la de mayor prioridad. Utilidad: $u(\pi, x) = -(\text{bandejas usadas})$. Entrenamiento: 3 instancias de 1000 objetos. Prueba: 5 instancias nuevas de 5000 objetos. Métrica de prueba: exceso relativo sobre la cota L_1, $(\bar{m} - \bar{L}_1)/\bar{L}_1$.

Método. Sin acceso a un LLM, el cuaderno sustituye el modelo por un *generador de mutaciones de programas* que imita las ediciones de un programador: retocar una constante, añadir una regla del tipo «si el hueco g es menor que c, sumar w», saturar el hueco ($g \to \min(g, c)$), reescribir un subárbol o combinar las dos funciones del prompt. El resto replica FunSearch a pequeña escala: semilla BF, $k = 2$ programas por propuesta ordenados de peor a mejor, 4 islas, muestreo de clusters con temperatura, preferencia por programas cortos, reinicio de la mitad peor cada 75 evaluaciones y un presupuesto de 300 evaluaciones (unos 30 s en una CPU).

Resultado. El mejor programa encontrado es

$$\pi^*(s, r) = -\min(g, 9) - 5{,}82 \cdot \mathbf{1}[g \leq 20], \qquad g = r - s.$$

Su lectura es clara: los huecos casi nulos ($g \leq 3$) puntúan más que -9 y se prefieren; los huecos intermedios, $9 < g \leq 20$, puntúan $-14{,}82$, lo peor posible; con huecos mayores que 20 todas las bandejas empatan en -9 y gana la primera, como en FF. La regla *evita dejar restos de 10 a 20 unidades*, demasiado pequeños para casi cualquier paquete (más del 98 % de los tamaños supera 10). El cuadro 1.2 compara con las reglas clásicas y con la heurística publicada por FunSearch para estos datos [16], evaluadas sobre las mismas instancias de prueba.

Cuadro 1.2: Empaquetado en línea de 5 instancias de prueba de 5000 objetos ($\bar{L}_1 = 1985{,}6$). Datos del cuaderno.

Regla	Bandejas (media)	Exceso sobre L_1
first fit	2073,4	4,42 %
best fit	2064,8	3,99 %
regla evolucionada π^*	2011,2	1,29 %
heurística publicada por FunSearch	2000,0	0,73 %

Como referencia externa, en el conjunto Weibull de 5000 objetos que acompaña al artículo, BF usa de media 2067,0 bandejas y la heurística de FunSearch 2001,4, con $\bar{L}_1 = 1987{,}8$: excesos del 3,98 % y del 0,68 % [16]. La *prueba de robustez* del cuaderno repite la comparación con otra distribución, Weibull de escala 60 (objetos más grandes): BF deja un exceso del 5,19 %, la regla π^* un 5,21 % y la heurística publicada un 6,40 %, peor que FF (6,05 %). Con seis semillas distintas del generador, el exceso en prueba de la regla encontrada varía entre el 1,29 % y el 2,33 %, y cuatro de las seis reglas tienen la misma estructura: un término de *best fit* saturado más una penalización de los huecos intermedios. Las dos heurísticas generadas se

han especializado en la distribución con la que se evaluaron, tal como advierten Herrmann y Pallez [17].

Cuadro 1.3: Tecnologías del tema 1.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
FunSearch (repositorio) [16]	algoritmo evolutivo, manipulación de código y una versión de un solo hilo, sin LLM ni entorno aislado	abierta (Apache 2.0)	entender el método y adaptarlo a un LLM y una infraestructura propios
OpenEvolve [35]	implementación abierta inspirada en AlphaEvolve: islas, MAP-Elites, evaluación en cascada, conjuntos de LLM	abierta (Apache 2.0)	evolucionar ficheros completos con cualquier API compatible
AlphaEvolve [34]	agente de Google DeepMind	servicio, no publicado como código	referencia de diseño; resultados publicados
gplearn, DEAP	programación genética clásica sin LLM	abiertas (BSD-3; LGPL)	línea base barata para expresiones pequeñas
Contenedores aislados	ejecutar código no confiable sin red y con límites de tiempo y memoria	abierta o del proveedor	siempre que se ejecute código generado

Observación 1.14 (Errores frecuentes en la práctica). (a) *Evaluar con las mismas instancias con que se selecciona*: el mejor programa sobre 3 instancias está sesgado al alza (tema 2, proposición 2.4); hay que reservar instancias de prueba y otras distribuciones plausibles.

(b) *Un evaluador que se puede engañar*: si el programa puede leer los datos de prueba, modificar el contador o detectar que está siendo evaluado, acabará haciéndolo.

(c) *Ejecutar el código generado sin aislamiento*: el código de un LLM es código no confiable.

(d) *Comparar con presupuestos distintos*: una heurística evolucionada durante horas debe compararse con líneas base que hayan recibido un esfuerzo comparable (al menos FF, BF y una búsqueda aleatoria de constantes).

(e) *Ignorar el coste*: miles de llamadas al LLM y de evaluaciones tienen un precio; conviene estimar q con un piloto y presupuestar con la proposición 1.6.

(f) *No simplificar*: el programa ganador suele contener términos inertes; conviene podarlo y volver a evaluarlo antes de desplegarlo.

Lecturas recomendadas para este tema

- B. Romera-Paredes *et al.* [44] y su repositorio [16], cuaderno `bin_packing`.
- A. Novikov *et al.* [34], §§2–3, y la nota técnica [14].
- J. Herrmann y G. Pallez [17], para una lectura crítica.
- D. P. Williamson y D. B. Shmoys [50], §3.3 «The bin-packing problem».
- M. J. Kochenderfer y T. A. Wheeler [23], cap. 20 «Expression Optimization».

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Una empresa quiere descubrir una regla de prioridad para ordenar los trabajos en la cola de una máquina (cada trabajo tiene tiempo de proceso, fecha de entrega y peso). Identifique, según la definición 1.1 y la definición 1.3, el esqueleto, la firma de la función que evoluciona, la utilidad u, el conjunto de entrenamiento y el de prueba, y dos programas semilla razonables.

Ejercicio 1.2 ⋆⋆ Una isla contiene tres clusters con puntuaciones $-416{,}7$, $-409{,}3$ y $-405{,}3$ (menos el número medio de bandejas). (a) Calcule las probabilidades de la definición 1.4 con $T = 0{,}1$ aplicada a las puntuaciones brutas. (b) El cuaderno normaliza antes, $z_i = (s_i - \max_l s_l)/(\max_l s_l - \min_l s_l)$, y usa $\exp(z_i/T)$ con $T = 0{,}3$; calcule las probabilidades. (c) ¿Qué consecuencia práctica tiene la escala de las puntuaciones?

Ejercicio 1.3 ⋆⋆ Con $C = 10$ llegan los objetos $3, 7, 4, 7, 8, 3, 5$. Aplique FF y BF, calcule L_1 y diga si alguno de los dos es óptimo.

Ejercicio 1.4 ⋆⋆ *Next fit* (NF) mantiene abierto solo el último contenedor: si el objeto cabe, lo coloca allí; si no, cierra ese contenedor para siempre y abre otro. (a) Demuestre que NF usa a lo sumo $2\,\mathrm{OPT}$ contenedores observando que dos contenedores consecutivos suman más de C. (b) Con $C = 1$ y $N = 2k$ objetos que alternan $1/2$ y ε ($0 < \varepsilon < 1/(2k)$), calcule cuántos contenedores usa NF y cuántos bastan, y concluya que el factor 2 no puede mejorarse para NF.

Fuente: adaptado de D. P. Williamson y D. B. Shmoys [50], §3.3 «The bin-packing problem» (análisis de algoritmos voraces de empaquetado).

Ejercicio 1.5 ⋆⋆ En una prueba piloto, de 1000 propuestas del LLM, 2 mejoraron al mejor programa del momento. (a) Estime q. (b) ¿Cuántas evaluaciones hacen falta para tener al menos un 95 % de probabilidad de una mejora, según la proposición 1.6? (c) Si cada evaluación cuesta 0,004 € de LLM y 20 s de CPU a 0,05 €/h, ¿cuánto cuesta ese presupuesto? (d) Con 10 islas independientes de 300 evaluaciones cada una, ¿cuál es la probabilidad de que alguna obtenga al menos una mejora?

Ejercicio 1.6 ⋆⋆ Considere la gramática $E \to E + E \mid E \times E \mid x \mid 1$. Sea N_d el número de árboles de expresión de profundidad a lo sumo d (las hojas tienen profundidad 0). (a) Justifique la recurrencia $N_0 = 2$, $N_d = 2 + 2N_{d-1}^2$. (b) Calcule N_1, N_2 y N_3. (c) Relacione el resultado con la proposición 1.6: ¿qué valor de q tendría una propuesta uniforme entre los árboles de profundidad ≤ 3 si solo uno fuera mejor que la semilla?

Fuente: adaptado de M. J. Kochenderfer y T. A. Wheeler [23], cap. 20 «Expression Optimization» (gramáticas y crecimiento del espacio de expresiones).

Ejercicio 1.7 ⋆⋆⋆ Para la regla $\pi^*(s, r) = -\min(g, 9) - 5{,}82\,\mathbf{1}[g \le 20]$ del ejemplo 1.13: (a) escriba la prioridad como función a trozos de g y ordene los intervalos de g de preferido a menos preferido; (b) demuestre que, si ninguna bandeja deja $g \le 20$, la regla coincide con FF, y que, si alguna deja $g \le 3$, elige la de menor hueco; (c) determine si π^*, con el esqueleto del caso (que incluye siempre una bandeja vacía entre los candidatos), es una regla *any fit* y si le es aplicable el teorema 1.11.

Ejercicio 1.8 ⋆⋆⋆ Diseñe un sistema tipo FunSearch para descubrir la heurística que asigna máquinas virtuales a servidores en un centro de datos (dos recursos: CPU y memoria). Especifique el esqueleto y la firma, el evaluador (métricas, instancias, cómo evitar que se engañe), la separación entre entrenamiento, validación y prueba (incluidas distribuciones distintas), las salvaguardas para ejecutar el código generado, el presupuesto según la proposición 1.6 y el criterio para desplegar la regla en producción.

Tema 2

Optimización de instrucciones, de sistemas con LLM y de código

Un sistema construido con modelos de lenguaje tiene muchos parámetros que no son números: las instrucciones de cada llamada, los ejemplos que se incluyen, el orden de los pasos, el número de muestras que se votan. Este tema los trata como variables de decisión de un problema de optimización de caja negra con un objetivo medido sobre datos. Sigue a Yang *et al.* para OPRO [51], a Khattab *et al.* para DSPy [22] y a Opsahl-Ong *et al.* para MIPRO [36]; la autoconsistencia, a Wang *et al.* [49] y al módulo 22 (tema 1, §1.4); el sesgo del ganador, a Smith y Winkler [47]; la optimización de código y la medida de tiempos, a la clase 10 «Measurement and Timing» del curso 6.172 del MIT [32].

2.1 Instrucciones como variables de decisión

Definición 2.1 (Optimización de instrucciones). Sea $\mathcal{I}$ un conjunto de instrucciones (textos), M un modelo de lenguaje fijo, $D_{\text{val}} = \{(x_i, y_i)\}_{i=1}^{n}$ un conjunto de validación y $m(\hat{y}, y) \in [0, 1]$ una métrica (por ejemplo, $\mathbf{1}[\hat{y} = y]$). El problema es

$$\max_{p \in \mathcal{I}} J(p), \qquad J(p) = \mathbb{E}_{(x,y)}\big[m(M(p, x), y)\big], \qquad \hat{J}(p) = \frac{1}{n}\sum_{i=1}^{n} m\big(M(p, x_i), y_i\big).$$

Es un problema de caja negra (módulo 24, definición 5.1), discreto, ruidoso (el estimador $\hat{J}$ tiene varianza $J(1-J)/n$ si m es binaria) y caro (cada evaluación son n llamadas al modelo).

Definición 2.2 (OPRO). *Optimization by PROmpting* [51] usa un segundo LLM como optimizador. En cada paso se le da un *meta-prompt* con: una descripción de la tarea y unos pocos ejemplos, y la *trayectoria*: las mejores instrucciones encontradas hasta el momento con su puntuación, ordenadas de menor a mayor. El LLM optimizador genera nuevas instrucciones; se evalúan con el LLM que resuelve la tarea, se añaden a la trayectoria y se repite. En los experimentos del artículo, la trayectoria conserva las 20 mejores, se generan 8 instrucciones por paso con temperatura 1 y la puntuación es la exactitud sobre un subconjunto fijo del conjunto de entrenamiento.

OPRO es una búsqueda evolutiva como la del tema 1, con dos diferencias: el «programa» es un texto y la memoria es la propia trayectoria, que el LLM lee para inferir qué rasgos se asocian a puntuaciones altas. Los autores informan de que las mejores instrucciones encontradas superan a las diseñadas por personas hasta en 8 puntos en GSM8K y hasta en 50 en tareas de Big-Bench Hard [51]; también advierten de que, en el problema del viajante, el LLM se mantiene cerca de las heurísticas clásicas con 50 ciudades, pero en instancias mayores la inserción más lejana supera a todos los modelos probados.

Definición 2.3 (Programa de modelos de lenguaje y su compilación). En DSPy [22], un *programa de LM* es un grafo de *módulos*; cada módulo tiene una *firma* declarativa (entradas y salidas con nombre, por ejemplo `pregunta, contexto -> respuesta`) y unos parámetros: su instrucción y sus demostraciones. Un *optimizador* (antes llamado *teleprompter*) recibe el programa, un conjunto de entrenamiento y una métrica, y elige los parámetros de todos los módulos para maximizar la métrica del programa completo. Por ejemplo, el arranque de demostraciones ejecuta el programa sobre los ejemplos de entrenamiento y conserva como demostraciones las trazas que superan la métrica.

Khattab *et al.* informan de que los programas compilados superan a los mismos programas con pocos ejemplos estándar en más de un 25 % con GPT-3.5 y un 65 % con llama2-13b-chat [22]. MIPRO [36] optimiza conjuntamente instrucciones y demostraciones de cada módulo sin etiquetas intermedias: propone instrucciones teniendo en cuenta los datos y el programa, y usa evaluaciones en minilotes para ajustar un modelo sustituto del objetivo, como la optimización bayesiana del módulo 25 (tema 4); obtiene mejoras de hasta 13 puntos de exactitud con Llama-3-8B.

2.2 El sesgo del ganador

Todas estas técnicas eligen el máximo de muchas estimaciones ruidosas. El resultado es predecible.

Proposición 2.4 (Maldición del optimizador). *Sean K candidatos con valores verdaderos $J_1, \dots, J_K$ y estimaciones $\hat{J}_k = J_k + \varepsilon_k$ con $\mathbb{E}[\varepsilon_k] = 0$, y sea $\hat{k} = \arg\max_k \hat{J}_k$. Entonces:*

(i) $\mathbb{E}[\hat{J}_{\hat{k}}] \geq \max_k J_k \geq \mathbb{E}[J_{\hat{k}}]$*: la puntuación de validación del elegido sobrestima en media su valor verdadero.*

(ii) Si cada $\varepsilon_k \sim \mathcal{N}(0, \sigma^2)$ *(no necesariamente independientes),* $\mathbb{E}[\max_k \varepsilon_k] \leq \sigma\sqrt{2\ln K}$*. En particular, si todos los* J_k *son iguales, el sesgo* $\mathbb{E}[\hat{J}_{\hat{k}} - J_{\hat{k}}]$ *está entre* 0 *y* $\sigma\sqrt{2\ln K}$*.*

Demostración. (i) Como el máximo de variables aleatorias es mayor o igual que cada una, $\mathbb{E}[\max_k \hat{J}_k] \geq \mathbb{E}[\hat{J}_l] = J_l$ para todo l, luego $\mathbb{E}[\hat{J}_{\hat{k}}] \geq \max_l J_l$. Por otra parte, $J_{\hat{k}} \leq \max_l J_l$ siempre, y tomando esperanzas se obtiene la segunda desigualdad. (ii) Para $t > 0$, por la desigualdad de Jensen (módulo 6, tema 2, §2.1) aplicada a la función convexa exp,

$$e^{t\,\mathbb{E}[\max_k \varepsilon_k]} \leq \mathbb{E}\big[e^{t \max_k \varepsilon_k}\big] = \mathbb{E}\big[\max_k e^{t\varepsilon_k}\big] \leq \sum_{k=1}^{K} \mathbb{E}\big[e^{t\varepsilon_k}\big] = K\, e^{t^2\sigma^2/2},$$

donde se ha usado la función generadora de momentos de la normal. Tomando logaritmos, $\mathbb{E}[\max_k \varepsilon_k] \leq \ln K/t + t\sigma^2/2$, y el mínimo en $t = \sqrt{2\ln K}/\sigma$ vale $\sigma\sqrt{2\ln K}$. Si los J_k son iguales, $\hat{J}_{\hat{k}} - J_{\hat{k}} = \max_k \varepsilon_k$, que tiene esperanza no negativa por (i) y acotada por lo anterior. ∎

Ejemplo 2.5 (Veinte instrucciones igual de buenas). Veinte instrucciones tienen la misma exactitud verdadera, 0,70, y se evalúan con $n = 100$ ejemplos. Cada estimación tiene desviación típica $\sigma = \sqrt{0{,}7 \cdot 0{,}3/100} = 0{,}0458$. La cota da un sesgo de a lo sumo $0{,}0458\sqrt{2\ln 20} = 0{,}0458 \cdot 2{,}448 = 0{,}112$; la simulación del cuaderno, con 20 000 repeticiones, da un sesgo medio de 0,083. La instrucción «ganadora» aparenta un 78 % cuando vale un 70 %. Es la misma lección que las comparaciones múltiples del módulo 0.9 (tema 5, §5.6) y que la sobreoptimización de la recompensa del módulo 21 (tema 6, §6.4).

Corolario 2.6 (Protocolo de evaluación). *Para estimar sin sesgo el valor de un sistema optimizado se necesita un conjunto de* prueba *que no haya intervenido en ninguna elección (ni de instrucciones, ni de ejemplos, ni de hiperparámetros), igual que en la separación en entrenamiento, validación y prueba del módulo 8 (tema 1, §1.4). Su error típico,* $\sqrt{J(1-J)/n_{\text{test}}}$*, fija la resolución con la que se pueden comparar sistemas.*

2.3 Caso de estudio: clasificación de partes de avería

Ejemplo 2.7 (Optimizar las instrucciones de un clasificador). *Problema.* El departamento de mantenimiento de una planta de envasado recibe partes de avería en texto libre y quiere clasificarlos con un LLM en seis categorías (eléctrica, mecánica, neumática, sensores, software, otras) para encaminarlos al técnico adecuado. Se dispone de 100 partes etiquetados para validación y de 1000 para prueba.

Variables. La instrucción se construye con seis decisiones: papel asignado al modelo (ninguno, técnico, experto), formato de salida (libre, JSON, una palabra), número de ejemplos (0, 2, 4, 8), indicación de razonamiento (no, «paso a paso», «enumera los síntomas primero»), idioma de las etiquetas (español, inglés) y posición de la lista de etiquetas (antes o después del parte). Hay $3 \cdot 3 \cdot 4 \cdot 3 \cdot 2 \cdot 2 = 432$ instrucciones.

Simulación. Como el cuaderno no llama a ningún modelo, la exactitud verdadera de cada instrucción se define con una función sintética con interacciones (por ejemplo, pedir «una palabra» y a la vez razonar perjudica), cuyo máximo es 0,730 y cuya media sobre las 432 es 0,580. Evaluar una instrucción es observar una binomial de 100 ensayos.

Métodos. (a) Búsqueda aleatoria de 96 instrucciones. (b) Un optimizador de tipo OPRO con 12 pasos de 8 propuestas: guarda las 20 mejores, parte de una de las 3 mejores y cambia una o dos decisiones, eligiendo cada opción con una softmax de la puntuación media de las instrucciones de la trayectoria que la contienen (un sustituto sencillo de lo que el LLM infiere al leer la trayectoria).

Resultado. En 200 repeticiones, la instrucción elegida por OPRO tiene una exactitud verdadera media de 0,701 y la elegida por búsqueda aleatoria, 0,692; pero sus puntuaciones de validación son 0,791 y 0,774. El optimismo es de unos 9 puntos en ambos casos, como predice la proposición 2.4. Con los 1000 partes de prueba, el error típico es $\sqrt{0{,}7 \cdot 0{,}3/1000} = 0{,}0145$ y el intervalo del 95 % tiene una semiamplitud de unos 2,8 puntos: suficiente para confirmar que la instrucción supera el 67 % que obtiene el sistema actual de reglas, insuficiente para distinguir entre las dos estrategias de búsqueda.

2.4 La autoconsistencia como problema de optimización

La autoconsistencia (módulo 22, definición 1.14) devuelve la respuesta más votada entre m muestras. Más muestras dan más exactitud y más coste; el número de muestras es una variable de decisión.

Proposición 2.8 (Incremento de la exactitud del voto). *Sea* P_m *la probabilidad de que la mayoría de* m *(impar) respuestas independientes sea correcta cuando cada una lo es con probabilidad* $p > 1/2$ *(módulo 22, proposición 1.16). Entonces*

$$\Delta_m := P_{m+2} - P_m = \binom{m}{\frac{m-1}{2}} \big(p(1-p)\big)^{\frac{m+1}{2}} (2p-1) > 0, \qquad \frac{\Delta_{m+2}}{\Delta_m} = \frac{4(m+2)}{m+3}\, p(1-p) < 1.$$

Demostración. Al pasar de m a $m+2$ votos, el resultado de la mayoría solo puede cambiar si tras m votos la diferencia entre aciertos y fallos es exactamente uno. Si hay $(m+1)/2$ aciertos, la mayoría pasa a ser incorrecta si los dos votos nuevos fallan (probabilidad $(1-p)^2$); si hay $(m-1)/2$ aciertos, pasa a ser correcta si los dos aciertan (probabilidad p^2). Como $\binom{m}{(m+1)/2} = \binom{m}{(m-1)/2}$,

$$\Delta_m = \binom{m}{\frac{m-1}{2}}\Big[p^{\frac{m-1}{2}}(1-p)^{\frac{m+1}{2}}p^2 - p^{\frac{m+1}{2}}(1-p)^{\frac{m-1}{2}}(1-p)^2\Big] = \binom{m}{\frac{m-1}{2}}\big(p(1-p)\big)^{\frac{m+1}{2}}\big(p-(1-p)\big).$$

Para el cociente, $\binom{m+2}{(m+1)/2}\big/\binom{m}{(m-1)/2} = \frac{(m+2)(m+1)}{\frac{m+1}{2}\cdot\frac{m+3}{2}} = \frac{4(m+2)}{m+3}$, y se multiplica por $p(1-p)$. Como $4p(1-p) < 1$ para $p \neq 1/2$ y $(m+2)/(m+3) < 1$, el cociente es menor que 1. ■

Corolario 2.9 (Número óptimo de muestras). *Si una respuesta correcta vale V y cada muestra cuesta c, el beneficio neto $VP_m - cm$ (con m impar) se maximiza en el primer m tal que $V\Delta_m < 2c$.*

Demostración. Pasar de m a $m+2$ cambia el beneficio en $V\Delta_m - 2c$. Como Δ_m decrece (proposición 2.8), estos cambios forman una sucesión decreciente: son positivos hasta cierto m y negativos a partir de él, de modo que el máximo está en el primer m con $V\Delta_m - 2c < 0$. ■

Con $p = 0{,}65$, $V = 0{,}50$ € y $c = 0{,}01$ €: $\Delta_1 = 0{,}0683$, $\Delta_3 = 0{,}0466$ y $\Delta_5 = 0{,}0353$; $V\Delta_3 = 0{,}0233 \geq 0{,}02$ pero $V\Delta_5 = 0{,}0177 < 0{,}02$, de modo que $m^* = 5$, con exactitud $P_5 = 0{,}765$ y beneficio neto 0,332 € por parte. La hipótesis de independencia es optimista (módulo 22, ejemplo 1.17): conviene estimar P_m con datos y aplicar el mismo criterio.

2.5 Optimización de código con agentes

Un agente de programación (módulo 22, tema 6) puede usarse como optimizador de un programa existente. El planteamiento correcto es el de un problema con restricciones:

$$\min_{\pi}\ t(\pi) \quad \text{s.a.} \quad \pi \text{ supera todas las pruebas}, \quad \text{memoria}(\pi) \leq \bar{M}, \quad \text{coste del agente} \leq B,$$

donde t es el tiempo de ejecución en una carga representativa. Las pruebas son una *restricción dura*, no un término del objetivo: un programa más rápido que falla una prueba no es una solución. Es el mismo principio del evaluador del tema 1.

Proposición 2.10 (Ley de Amdahl). *Si una fracción f del tiempo de ejecución se acelera en un factor $a > 1$ y el resto no cambia, la aceleración global es*

$$A = \frac{1}{(1-f) + f/a} < \frac{1}{1-f}.$$

Demostración. Si el tiempo original es t, el nuevo es $(1-f)t + ft/a$, y A es el cociente. Como $f/a > 0$, el denominador es mayor que $1-f$. ■

La ley dice dónde empezar: por lo que el *perfilador* muestra que ocupa más tiempo. Un agente que no perfila tiende a optimizar lo que parece lento en el código, no lo que lo es.

Ejemplo 2.11 (Caso de estudio: acelerar un servicio de planificación). Un servicio calcula cada noche las rutas de reparto; tarda 40 min y debe bajar de 20 para dejar margen. El perfilado asigna un 55 % del tiempo al cálculo de distancias, un 25 % a la serialización de resultados y un 20 % al resto. El agente propone vectorizar el cálculo de distancias (factor 4) y cambiar el serializador (factor 5). Por la proposición 2.10: solo la primera mejora da $A = 1/(0{,}45 + 0{,}1375) = 1{,}70$ (23,5 min); solo la segunda, $1/(0{,}75 + 0{,}05) = 1{,}25$; las dos, $1/(0{,}20 + 0{,}1375 + 0{,}05) = 2{,}58$ (15,5 min), que cumple el objetivo. Aunque las distancias se hicieran instantáneas, sin tocar lo demás no se bajaría de $40 \cdot 0{,}45 = 18$ min.

Medida. Siguiendo la clase 10 del curso 6.172 del MIT [32]: el tiempo de un programa determinista solo puede aumentar por interferencias (otros procesos, interrupciones, escalado de frecuencia), así que el *mínimo* de varias repeticiones es el estimador más robusto; las versiones deben ejecutarse *intercaladas* y no una tras otra, para que el calentamiento de la máquina no favorezca a la primera; y para resumir cocientes de tiempos entre varias cargas se usa la *media geométrica*. Criterio de aceptación: en 20 pares intercalados, la versión nueva debe ganar al menos 15 veces; bajo la hipótesis de que ambas son igual de rápidas, $\mathbb{P}(\mathrm{Bin}(20, 1/2) \geq 15) = 0{,}021$.

Presupuesto. Cada iteración del agente cuesta tokens y una ejecución de las pruebas; se fija un máximo de iteraciones y se detiene la búsqueda cuando dos iteraciones seguidas no superan el criterio de aceptación.

Cuadro 2.1: Tecnologías del tema 2.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
DSPy [22]	programas de LM declarativos y optimizadores de instrucciones y demostraciones (incluido MIPRO)	abierta (MIT)	sistemas de varias llamadas con una métrica clara
OPRO [51]	método: un LLM como optimizador a partir de la trayectoria	artículo con código de los autores	instrucciones de una sola llamada; problemas pequeños
promptfoo	conjuntos de pruebas y comparación de instrucciones y modelos	abierta (MIT)	evaluación de regresión de instrucciones
pytest, pytest-benchmark	pruebas como restricción y medidas de tiempo repetidas	abiertas (MIT; BSD-2)	optimización de código en Python
perf, py-spy, Scalene	perfilado de CPU y memoria	abiertas (GPL-2.0; MIT; Apache 2.0)	antes de pedir al agente que optimice nada
asv	seguimiento del rendimiento a lo largo del historial del código	abierta (BSD-3)	evitar regresiones de rendimiento

Observación 2.12 (Errores frecuentes en la práctica). (a) *Informar de la puntuación de validación del ganador* como si fuera su rendimiento (proposición 2.4).

(b) *Validación demasiado pequeña*: con $n = 100$, el error típico ronda los 5 puntos; diferencias de 2 puntos son ruido.

(c) *Contaminación*: ejemplos de prueba que acaban como demostraciones o en la trayectoria

del optimizador (módulo 22, tema 3, §3.4).

(d) *Cambiar de modelo sin reoptimizar*: las instrucciones óptimas dependen del modelo; una actualización del proveedor puede degradarlas sin aviso.

(e) *Pruebas débiles como restricción*: si las pruebas no cubren un caso, el agente puede «acelerar» eliminándolo.

(f) *Medir una sola vez, o primero A y luego B*, y declarar una mejora que es ruido.

Lecturas recomendadas para este tema

- C. Yang *et al.* [51], §§2 y 4–5.
- O. Khattab *et al.* [22] y K. Opsahl-Ong *et al.* [36].
- J. E. Smith y R. L. Winkler [47], sobre la maldición del optimizador.
- MIT 6.172 [32], clase 10 «Measurement and Timing».

Ejercicios del tema 2

Ejercicio 2.1 ⋆ Escriba (en español y en forma esquemática) el meta-prompt de OPRO para el ejemplo 2.7: qué partes contiene, en qué orden aparece la trayectoria y por qué, y qué información *no* debe contener.

Ejercicio 2.2 ⋆ En DSPy, un optimizador necesita una métrica que reciba un ejemplo y una predicción y devuelva un número o un booleano. Para un sistema que responde preguntas sobre manuales de mantenimiento recuperando fragmentos, proponga una métrica que exija a la vez que la respuesta sea correcta y que el fragmento recuperado la contenga, y explique qué ocurre en la compilación si la métrica solo comprueba la respuesta.

Fuente: adaptado de CS224U, Stanford [39], tarea «Few-shot OpenQA with DSPy», función de validación del contexto y la respuesta.

Ejercicio 2.3 ⋆⋆ Se comparan $K = 50$ instrucciones con $n = 200$ ejemplos de validación; todas tienen exactitud verdadera 0,80. (a) Calcule σ y la cota de la proposición 2.4. (b) ¿Cuántos ejemplos harían falta para que la cota del sesgo fuera de 2 puntos? (c) ¿Y si solo se comparan $K = 5$?

Ejercicio 2.4 ⋆⋆ Con $p = 0{,}7$, $V = 0{,}20$ € y $c = 0{,}004$ € por muestra: (a) calcule Δ_1, Δ_3, Δ_5 y Δ_7 con la proposición 2.8; (b) obtenga m^* con el corolario 2.9; (c) calcule P_{m^*}.

Ejercicio 2.5 ⋆⋆ Un programa de simulación pasa un 70 % del tiempo en un bucle numérico, un 20 % en lectura de datos y un 10 % en el resto. (a) ¿Qué aceleración global se obtiene si el agente acelera el bucle un factor 8? (b) ¿Qué factor haría falta en el bucle para una aceleración global de 2? (c) ¿Es posible una aceleración global de 4 sin tocar la lectura?

Ejercicio 2.6 ⋆⋆ Se miden dos versiones A y B de un programa en cinco cargas; los tiempos mínimos (en s) son A: 2,0; 4,0; 1,0; 8,0; 3,0 y B: 1,6; 4,4; 0,5; 8,0; 2,4. (a) Calcule los cocientes t_B/t_A, su media aritmética y su media geométrica. (b) Calcule los cocientes t_A/t_B y sus dos

medias. (c) Explique por qué la media geométrica es la única de las dos que da conclusiones coherentes en (a) y (b). (d) En 20 pares intercalados, B gana 14 veces; ¿basta con el criterio del ejemplo 2.11?

Fuente: adaptado de MIT 6.172, otoño de 2018, clase 10 «Measurement and Timing» (resumen de cocientes con la media geométrica y comparación de dos programas) [32].

Ejercicio 2.7 ⋆⋆⋆ Demuestre que, si $\varepsilon_1, \dots, \varepsilon_K$ son independientes $\mathcal{N}(0, \sigma^2)$, $\mathbb{E}[\max_k \varepsilon_k]$ es creciente en K. Después, con $K = 2$, calcule $\mathbb{E}[\max(\varepsilon_1, \varepsilon_2)]$ exactamente (use que $\max(a, b) = \frac{a+b}{2} + \frac{|a-b|}{2}$) y compárelo con la cota $\sigma\sqrt{2 \ln 2}$.

Fuente: adaptado de J. E. Smith y R. L. Winkler [47] (sesgo del máximo de estimaciones con error).

Ejercicio 2.8 ⋆⋆⋆ Diseñe el protocolo completo para optimizar con DSPy un asistente que redacta órdenes de trabajo a partir de partes de avería: división de los datos, métrica (y cómo se valida la propia métrica si usa un LLM como juez), presupuesto de llamadas, criterio para aceptar la nueva versión frente a la actual con un intervalo de confianza, y qué hacer cuando el proveedor actualice el modelo.

Tema 3

AutoML y búsqueda de arquitecturas

Elegir un modelo, sus hiperparámetros y su preprocesado es también un problema de optimización sobre programas: cada configuración es una canalización ejecutable cuyo valor se mide con validación. Este tema formaliza ese problema siguiendo el libro editado por Hutter, Kotthoff y Vanschoren [19] (cap. 1 «Hyperparameter Optimization» y cap. 3 «Neural Architecture Search», de Elsken, Metzen y Hutter [12]) y los artículos originales: Auto-WEKA [48], búsqueda aleatoria [3], búsqueda de arquitecturas con refuerzo [53], con evolución [41], con pesos compartidos [38] y diferenciable [28], y la línea base de Li y Talwalkar [26]. La multifidelidad sigue a Jamieson y Talwalkar [21], a Li *et al.* [27] y a la §19.4 de Zhang *et al.* [52]. Complementa el tema 6 del módulo 25, que estudió el ajuste de hiperparámetros con optimización bayesiana y Hyperband; aquí el énfasis está en el espacio de búsqueda y en el coste.

3.1 AutoML como optimización sobre programas

Definición 3.1 (Selección conjunta de algoritmo e hiperparámetros). Se consideran R algoritmos de aprendizaje, $A^{(1)}, \dots, A^{(R)}$, cada uno con un espacio de hiperparámetros $\Lambda^{(j)}$, y sea D un conjunto de datos dividido en k pares $(D_{\text{tr}}^{(i)}, D_{\text{va}}^{(i)})$, cada uno formado por una parte de entrenamiento y otra de validación. El problema CASH (*combined algorithm selection and hyperparameter optimization*) [48] es

$$(j^*, \lambda^*) \in \operatorname*{arg\,min}_{j \in \{1,\dots,R\},\ \lambda \in \Lambda^{(j)}} \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}\big(A_\lambda^{(j)}, D_{\text{tr}}^{(i)}, D_{\text{va}}^{(i)}\big),$$

donde $\mathcal{L}$ es la pérdida de validación del modelo entrenado con $D_{\text{tr}}^{(i)}$.

El espacio de la definición 3.1 es *condicional*: la profundidad de un árbol solo existe si se elige un árbol. Se representa como un único espacio jerárquico cuya raíz es el hiperparámetro «algoritmo», de modo que métodos como la optimización bayesiana con bosques aleatorios (SMAC) o con estimadores de densidad en árbol (TPE) lo tratan sin cambios. Si además se eligen el preprocesado, la selección de variables y la combinación de modelos, la configuración es un programa completo; TPOT, por ejemplo, busca esas canalizaciones con programación genética, como en el tema 1 [19].

Observación 3.2 (Por qué la búsqueda aleatoria es una línea base seria). Si la pérdida depende sobre todo de unos pocos hiperparámetros, una rejilla desperdicia evaluaciones repitiendo los mismos valores de los importantes, mientras que n puntos aleatorios prueban n valores distintos de cada uno [3]. Además, la probabilidad de que al menos uno de n puntos aleatorios caiga en la fracción q mejor del espacio es $1 - (1-q)^n$, independiente de la dimensión: con $q = 0{,}05$ y $n = 60$ vale 0,954 (módulo 25, tema 6).

3.2 Búsqueda de arquitecturas neuronales

Elsken, Metzen y Hutter organizan la búsqueda de arquitecturas (NAS) en tres dimensiones [12]: el *espacio de búsqueda* (qué arquitecturas pueden representarse), la *estrategia de búsqueda* (cómo se explora) y la *estimación del rendimiento* (cómo se evalúa una arquitectura sin entrenarla del todo).

Definición 3.3 (Espacio de celdas). Una *celda* es un grafo dirigido acíclico con dos nodos de entrada (las salidas de las dos celdas anteriores) y B nodos intermedios. El nodo intermedio b elige dos entradas entre los $b+1$ nodos anteriores y aplica a cada una una operación de un conjunto $\mathcal{O}$ (convoluciones, agrupamientos, identidad…); su salida es la suma de ambas. La red se construye apilando copias de una celda normal y de una celda de reducción.

Si las dos elecciones de cada nodo son ordenadas y pueden repetirse, el número de celdas distintas es $\prod_{b=1}^{B}\big((b+1)|\mathcal{O}|\big)^2$: con $B=5$ y $|\mathcal{O}|=8$, unas $5{,}6\cdot 10^{14}$. La celda reduce el espacio respecto a una red arbitraria, pero sigue siendo inabarcable por enumeración.

Estrategias de búsqueda. Zoph y Le entrenaron con refuerzo una red recurrente que genera descripciones de arquitecturas y obtuvieron un 3,65 % de error en CIFAR-10 [53]. Real *et al.* usaron evolución por torneo con *envejecimiento* (se elimina el individuo más antiguo, no el peor), y en una comparación controlada con el mismo hardware la evolución llegó antes a buenos resultados que el refuerzo [41]. Liu, Simonyan y Yang hicieron el espacio continuo:

Definición 3.4 (Relajación continua de DARTS). En DARTS [28], cada arista (i,j) de la celda aplica una *mezcla* de todas las operaciones,

$$\bar{o}^{(i,j)}(x)=\sum_{o\in\mathcal{O}}\frac{\exp\big(\alpha_o^{(i,j)}\big)}{\sum_{o'\in\mathcal{O}}\exp\big(\alpha_{o'}^{(i,j)}\big)}\,o(x),$$

y los parámetros de arquitectura α se optimizan por descenso de gradiente en un problema binivel: $\min_\alpha \mathcal{L}_{\text{va}}\big(w^*(\alpha),\alpha\big)$ con $w^*(\alpha)=\arg\min_w \mathcal{L}_{\text{tr}}(w,\alpha)$. Al terminar, cada arista conserva la operación de mayor α.

Estimación del rendimiento. Entrenar cada candidata hasta el final es lo que hace prohibitiva la búsqueda. ENAS comparte los pesos entre todas las arquitecturas del espacio (cada candidata es un subgrafo de un modelo grande) y lo abarata en unas mil veces en horas de GPU respecto a la búsqueda original [38]. Li y Talwalkar mostraron que la búsqueda aleatoria con parada temprana iguala a ENAS en PTB y CIFAR-10, y con pesos compartidos lo supera, y señalaron problemas de reproducibilidad en la literatura [26]. La lección práctica es la del tema 1: compárese siempre con una búsqueda aleatoria del mismo presupuesto.

3.3 Multifidelidad y coste

Definición 3.5 (Fidelidad). Una *fidelidad* es un recurso $r\in[r_{\min},r_{\max}]$ que controla el coste y la precisión de la evaluación de una configuración λ: épocas de entrenamiento, fracción de los datos, resolución de las imágenes o número de árboles. Se escribe $f(\lambda,r)$ para la pérdida de validación con presupuesto r; la de interés es $f(\lambda,r_{\max})$.

Definición 3.6 (Halving sucesivo). Dados N configuraciones, un factor $\eta\geq 2$ y un presupuesto mínimo $r_{\min}$, el *halving sucesivo* [21] evalúa todas con $r_0=r_{\min}$, conserva la fracción

$1/\eta$ mejor, multiplica el presupuesto por η y repite: en el escalón i quedan $n_i = N/\eta^i$ configuraciones con presupuesto $r_i = r_{\min}\eta^i$, hasta $r_K = r_{\max}$.

Proposición 3.7 (Coste del halving sucesivo). *Si N es múltiplo de η^K y el coste de una evaluación es proporcional a su presupuesto, cada escalón cuesta $Nr_{\min}$ y el total es $(K+1)Nr_{\min}$, frente a $Nr_{\max} = Nr_{\min}\eta^K$ de evaluar todas con el presupuesto completo. El cociente es $(K+1)/\eta^K$.*

Demostración. El escalón i evalúa $n_i = N/\eta^i$ configuraciones con presupuesto $r_{\min}\eta^i$; su coste es $n_i r_i = Nr_{\min}$, independiente de i. Sumando los $K+1$ escalones $i = 0, \ldots, K$ se obtiene $(K+1)Nr_{\min}$; el cociente con $Nr_{\min}\eta^K$ es $(K+1)/\eta^K$. ■

Proposición 3.8 (Cuándo acierta el halving sucesivo). *Sea $\lambda^\dagger$ la mejor configuración a fidelidad completa. Si en cada escalón $i < K$ la ordenación por $f(\cdot, r_i)$ coloca a $\lambda^\dagger$ entre las n_{i+1} primeras de las que quedan, el halving sucesivo devuelve $\lambda^\dagger$. Si las curvas de aprendizaje se cruzan (una configuración peor al principio es mejor al final), puede descartarla.*

Demostración. Por inducción sobre los escalones, $\lambda^\dagger$ sobrevive a todos; en el último se evalúa a fidelidad completa, donde es la mejor por definición, y se devuelve. Para la segunda parte basta un ejemplo: dos configuraciones con $f(\lambda_1, r_0) < f(\lambda_2, r_0)$ y $f(\lambda_1, r_{\max}) > f(\lambda_2, r_{\max})$, $N = 2$, $\eta = 2$: se conserva λ_1, que no es la mejor. ■

La hipótesis de la proposición 3.8 se puede comprobar en un piloto con la correlación de rangos de Spearman entre las pérdidas a baja y a alta fidelidad. Hyperband [27] protege frente a una mala elección de $r_{\min}$ ejecutando varias rondas de halving sucesivo con distinta agresividad (módulo 25, tema 6). En la práctica, el coste no es exactamente proporcional al presupuesto: cargar los datos, compilar el modelo o arrancar una máquina son costes fijos que reducen el ahorro de la proposición 3.7.

3.4 Caso de estudio: AutoML del clasificador de averías

Ejemplo 3.9 (Selección del modelo de riesgo de avería). *Problema.* El tema 4 necesitará un clasificador que prediga, para cada máquina de una línea de envasado y cada día, si fallará en los 7 días siguientes, a partir de la vibración (mm/s), la temperatura del rodamiento (ºC), los días desde la última intervención, la carga de la línea, la pendiente de la vibración y el tipo de máquina. Se dispone de dos años de registros diarios de tres líneas con seis máquinas cada una.

Datos y división. Se separa *por tiempo*: los primeros 18 meses para entrenar (9594 filas) y los 6 siguientes para validar (3294 filas); una cuarta línea, no usada en ninguna decisión, sirve de prueba (4338 filas). La tasa de positivos es del 12,5 %. Métrica: pérdida logarítmica (penaliza las probabilidades mal calibradas, que el tema 4 usará para decidir).

Espacio. Tres familias: regresión logística ($C \in [10^{-3}, 10^2]$, escala logarítmica), *gradient boosting* (tasa de aprendizaje [0,01; 0,32], profundidad 2–6, 50–400 iteraciones) y bosque aleatorio (30–150 árboles, profundidad 3–11, mínimo de 1–29 ejemplos por hoja). Se muestrean 27 configuraciones.

Métodos. (a) Búsqueda aleatoria: las 27 con todos los datos de entrenamiento. (b) Halving sucesivo con $\eta = 3$ y fidelidad igual a la fracción (la más reciente) de los datos de entrenamiento: 27 configuraciones con 1/9, 9 con 1/3 y 3 con todo.

Resultado. Por la proposición 3.7, el coste del halving es $27\cdot\frac{1}{9}+9\cdot\frac{1}{3}+3\cdot 1 = 9$ entrenamientos completos equivalentes, frente a 27; el tiempo de reloj medido baja solo de 13,3 a 9,1 s por los costes fijos. La correlación de Spearman entre las pérdidas con 1/9 de los datos y con todos es 0,80. La búsqueda aleatoria elige un bosque (114 árboles, profundidad 6, 26 ejemplos por hoja) con pérdida de validación 0,2378; el halving elige otro bosque (93 árboles, profundidad 6, 9 por hoja), que es el cuarto de los 27 en validación, con 0,2384. En la línea de prueba ambos obtienen 0,2735 de pérdida logarítmica (frente a 0,3358 de predecir siempre la tasa base) y un área bajo la curva ROC de 0,815 y 0,817. La diferencia de validación entre los dos era ruido; el halving obtuvo lo mismo con un tercio del cómputo.

Cuadro 3.1: Tecnologías del tema 3.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
Optuna [1]	optimización de hiperparámetros con espacios definidos en el propio código (TPE y otros) y poda con halving sucesivo o Hyperband	abierta (MIT)	ajuste de cualquier modelo en Python con poco código
Ray Tune [40]	ajuste distribuido en un clúster con planificadores de multifidelidad	abierta (Apache 2.0)	muchas pruebas en paralelo sobre varias máquinas
SMAC3	configuración algorítmica con bosques aleatorios como sustituto	abierta (BSD-3)	espacios condicionales, configurar resolvedores
auto-sklearn, AutoGluon	AutoML completo para datos tabulares (selección, ajuste, conjuntos)	abiertas (BSD-3; Apache 2.0)	primera línea base fuerte en datos tabulares
NNI	búsqueda de arquitecturas y de hiperparámetros	abierta (MIT)	NAS con estrategias variadas sobre PyTorch

Observación 3.10 (Errores frecuentes en la práctica). (a) *Validación aleatoria con datos temporales*: mezclar días de entrenamiento y validación filtra el futuro y premia a los modelos que memorizan; la división debe respetar el tiempo, y la prueba, a ser posible, otra unidad (otra línea, otra planta).

(b) *Reutilizar la validación como prueba*: el modelo ganador sobre 27 tiene sesgo optimista (proposición 2.4).

(c) *Fidelidades engañosas*: con pocas épocas o pocos datos ganan los modelos que aprenden rápido, no los mejores (proposición 3.8); conviene medir la correlación de rangos.

(d) *Espacios de búsqueda sesgados*: rangos mal elegidos (tasa de aprendizaje en escala lineal, profundidades imposibles) hacen inútil cualquier estrategia.

(e) *No contar el coste total*: horas de GPU, energía, tiempo de ingeniería y mantenimiento del modelo elegido; un modelo un 0,1 % mejor pero diez veces más caro de servir puede no compensar.

Lecturas recomendadas para este tema

– F. Hutter, L. Kotthoff y J. Vanschoren (eds.) [19], caps. 1 y 3.

– C. Thornton *et al.* [48], para la formulación CASH.

– L. Li y A. Talwalkar [26], sobre líneas base y reproducibilidad.

– A. Zhang *et al.* [52], §§19.1 y 19.4.

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Para cada uno de estos sistemas, identifique el espacio de búsqueda, la estrategia y la estimación del rendimiento: (a) ENAS; (b) DARTS; (c) la búsqueda de Real *et al.*; (d) el halving sucesivo del ejemplo 3.9.

Fuente: adaptado de T. Elsken, J. H. Metzen y F. Hutter [12], cap. 3 de [19] (las tres dimensiones de NAS).

Ejercicio 3.2 ⋆⋆ Un modelo tiene 6 hiperparámetros, pero la pérdida depende casi solo de 2. Se dispone de 64 evaluaciones. (a) Con una rejilla de 2 valores por hiperparámetro, ¿cuántos valores distintos de cada hiperparámetro importante se prueban y cuántas combinaciones distintas de los dos importantes? (b) ¿Y con 64 puntos aleatorios? (c) ¿Cuántos puntos aleatorios hacen falta para que, con probabilidad 0,95, al menos uno caiga en el mejor 5 % del espacio?

Fuente: adaptado de A. Zhang *et al.* [52], §19.1, ejercicio 3 (búsqueda aleatoria frente a rejilla cuando pocos hiperparámetros importan).

Ejercicio 3.3 ⋆⋆ Formule como CASH (definición 3.1) la elección entre k vecinos más próximos ($k \in \{1, \dots, 50\}$, distancia euclídea o Manhattan), máquina de vectores soporte (núcleo lineal o gaussiano; C; γ solo si el núcleo es gaussiano) y bosque aleatorio (número de árboles y profundidad). Dibuje el árbol del espacio condicional y diga qué hiperparámetros están activos en cada rama.

Ejercicio 3.4 ⋆⋆ Con $r_{\min} = 2$ épocas, $\eta = 2$ y $r_{\max} = 10$, los escalones de presupuesto son $2, 4, 8, 10$ (el último se recorta a $r_{\max}$). Se empieza con $N = 16$ configuraciones. (a) ¿Cuántas configuraciones se evalúan en cada escalón? (b) ¿Cuántas épocas cuesta en total? (c) Compárelo con entrenar las 16 durante 10 épocas. (d) ¿Por qué el ahorro es menor que el de la proposición 3.7?

Fuente: adaptado de A. Zhang *et al.* [52], §19.4 «Multi-Fidelity Hyperparameter Optimization» (ejemplo con $r_{\min} = 2$, $\eta = 2$, $r_{\max} = 10$).

Ejercicio 3.5 ⋆⋆ Con la definición 3.3: (a) cuente las celdas posibles con $B = 4$ nodos intermedios y $|\mathcal{O}| = 5$ operaciones; (b) si una red usa una celda normal y otra de reducción elegidas por separado, ¿cuántas redes hay?; (c) si evaluar una arquitectura cuesta 1 hora de GPU, ¿cuánto costaría la enumeración?

Ejercicio 3.6 ⋆⋆ En una arista de DARTS hay cuatro operaciones escalares: identidad $o_1(x) = x$, cuadrado $o_2(x) = x^2$, cero $o_3(x) = 0$ y doble $o_4(x) = 2x$, con parámetros $\alpha = (1{,}0;\ 0{,}2;\ -0{,}5;\ 0{,}8)$. (a) Calcule los pesos de la mezcla y $\bar{o}(1{,}5)$. (b) ¿Qué operación se conserva al discretizar? (c) Calcule $\partial \bar{o}(x)/\partial \alpha_1$ en $x = 1{,}5$.

Ejercicio 3.7 ⋆⋆ Entrenar el clasificador completo cuesta 12 min en un nodo de 0,60 €/h. Suponiendo además un coste fijo de 1 min por entrenamiento (carga de datos y arranque), compare el tiempo y el coste en euros de: (a) una rejilla de 4 valores en 3 hiperparámetros; (b) 27 configuraciones aleatorias; (c) halving sucesivo con $N = 27$, $\eta = 3$ y fracciones 1/9, 1/3 y 1 de los datos. (d) ¿Qué cociente de coste entre (c) y (b) se obtiene, y por qué no es 1/3?

Ejercicio 3.8 ⋆⋆⋆ Las curvas de aprendizaje de dos configuraciones son $f_1(r) = 0{,}30 + 0{,}20e^{-r/2}$ y $f_2(r) = 0{,}25 + 0{,}40e^{-r/8}$ (r en épocas). (a) Calcule el r a partir del cual la segunda es mejor. (b) Demuestre que el halving sucesivo con $N = 2$, $\eta = 2$, $r_{\min} = 16$ y $r_{\max} = 32$ elige la peor. (c) Proponga una modificación del procedimiento, y justifique su coste, que la habría elegido bien.

Tema 4

Del prototipo a la producción: arquitectura, MLOps y evaluación

Un cuaderno que predice averías y propone un calendario es un prototipo; un sistema que lo hace cada día durante años, con datos que cambian, usuarios que dependen de él y auditores que preguntan por qué decidió algo, es otra cosa. Este tema describe esa otra cosa. La arquitectura y el ciclo de vida siguen el curso CS329S de Stanford [46] y el libro de Huyen [18]; la deuda técnica, a Sculley *et al.* [45]; la lista de pruebas, a Breck *et al.* [5]; la evaluación contrafactual, a Dudík, Langford y Li [11]; la fiabilidad, al capítulo 4 «Optimum Maintenance Policies» de Barlow y Proschan [2] y al curso 1.010 del MIT [29]. Completa la arquitectura del módulo 24 (tema 1, §1.2) y la puesta en planta del módulo 26 (tema 6), que trató el control por capas, los modos de operación, las alarmas y la ciberseguridad.

4.1 Arquitectura de un optimizador en producción

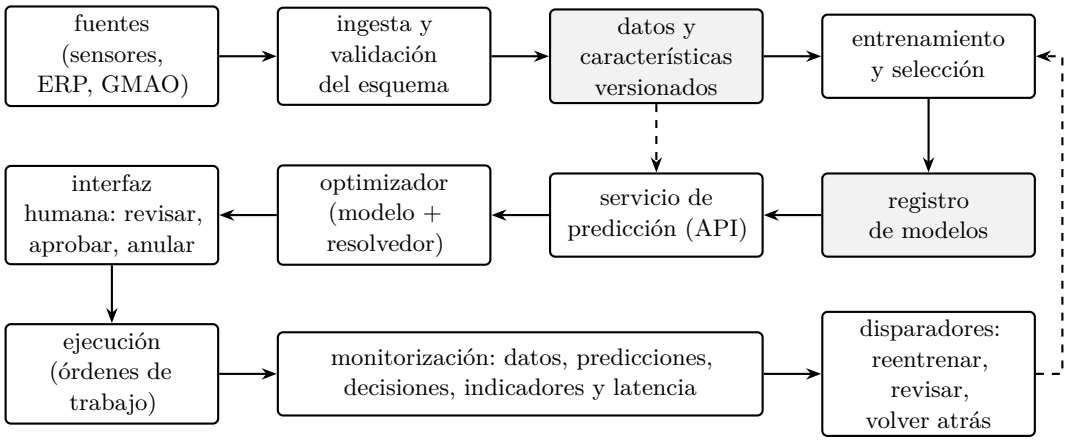


Figura 4.1: Arquitectura de un optimizador en producción. Las cajas sombreadas son almacenes versionados; las flechas discontinuas, los lazos de reentrenamiento y de consulta de características. GMAO: gestión del mantenimiento asistido por ordenador.

La figura 4.1 separa responsabilidades que en el prototipo estaban mezcladas.

1. *Datos*: la ingesta valida un *esquema* (tipos, rangos, unidades, frecuencias, ausencias) antes de aceptar nada; los datos y las características se *versionan*, de modo que se pueda reconstruir exactamente con qué datos se entrenó cada modelo.
2. *Características*: se calculan con el mismo código al entrenar y al servir, y respetando el tiempo: cada fila de entrenamiento solo puede usar información disponible en su instante.

3. *Modelos*: se entrenan con una canalización reproducible y se guardan en un *registro* con sus métricas, sus datos de origen y su estado (candidato, en sombra, en producción, retirado).
4. *Servicio*: una API con un contrato explícito (entradas, salidas, unidades, versión del modelo, latencia máxima) desacopla a los consumidores del modelo concreto.
5. *Optimizador*: combina las predicciones con el modelo de decisión y un resolvedor (módulo 24); sus restricciones duras no dependen del modelo aprendido.
6. *Interfaz humana*: presenta la propuesta con sus motivos, permite aprobarla, modificarla o anularla, y registra la decisión final; es el lugar donde se ejerce la supervisión que exige la regulación (tema 5).
7. *Monitorización*: vigila los datos, las predicciones, las decisiones, los indicadores de negocio y la salud del servicio, y dispara acciones definidas de antemano.

Definición 4.1 (Sesgo entre entrenamiento y servicio). Hay *sesgo entre entrenamiento y servicio* cuando la distribución de las entradas que el modelo recibe en producción difiere de la de entrenamiento por causas del propio sistema y no del mundo: características calculadas con código distinto, unidades distintas, datos que en entrenamiento estaban disponibles y en servicio llegan tarde, o filas de entrenamiento que contienen información del futuro.

4.2 Deuda técnica en sistemas con aprendizaje automático

Sculley *et al.* [45] observan que en un sistema real el código del modelo es una pequeña parte del total y enumeran riesgos propios de estos sistemas, que generan *deuda técnica*: costes futuros que se contraen al tomar atajos hoy.

Definición 4.2 (Factores de riesgo de Sculley *et al.*). (a) *Erosión de fronteras y entrelazamiento*: en un modelo, cambiar cualquier entrada cambia todo lo demás (principio CACE, *changing anything changes everything*); no hay módulos independientes.

(b) *Bucles de realimentación ocultos*: las decisiones del sistema cambian los datos con que se reentrena.

(c) *Consumidores no declarados*: otros sistemas usan las salidas del modelo sin que nadie lo sepa, y se rompen al cambiarlo.

(d) *Dependencias de datos*: entradas inestables (su significado cambia) o infrautilizadas (aportan poco y añaden riesgo).

(e) *Antipatrones de sistema*: código pegamento, junglas de canalizaciones, rutas experimentales muertas.

(f) *Deuda de configuración*: miles de parámetros de configuración sin revisión ni pruebas.

(g) *Cambios en el mundo exterior*: umbrales fijos que dejan de valer, correlaciones que desaparecen.

Observación 4.3 (El bucle oculto del mantenimiento predictivo). Si el sistema adelanta las intervenciones de las máquinas con riesgo alto, esas máquinas fallarán menos, y los datos futuros mostrarán pocos fallos asociados a vibraciones altas, porque se intervino antes. Un modelo reentrenado ingenuamente con esos datos concluirá que la vibración alta es menos peligrosa y relajará las intervenciones. Es el bucle de realimentación degenerado de la clase 10 de

CS329S [46]. Remedios: registrar cada decisión con su probabilidad (apartado 4.4), tratar las intervenciones como censura en el modelo de supervivencia y mantener una pequeña fracción de decisiones exploratorias cuando sea seguro.

Breck *et al.* [5] convierten estos riesgos en una lista de 28 pruebas agrupadas en cuatro bloques: datos y características, desarrollo del modelo, infraestructura y monitorización; cada prueba manual puntúa medio punto y cada prueba automatizada, uno. Es una buena lista de comprobación antes de desplegar.

4.3 Deriva y reentrenamiento

Definición 4.4 (Tipos de cambio de distribución). Sea X la entrada y Y la salida, con $P(X,Y) = P(Y \mid X)P(X) = P(X \mid Y)P(Y)$. Siguiendo la clase 10 de CS329S [46], hay *cambio de covariables* si cambia $P(X)$ y no $P(Y \mid X)$; *cambio de etiquetas* si cambia $P(Y)$ y no $P(X \mid Y)$; y *deriva de concepto* si cambia $P(Y \mid X)$. Además pueden cambiar las propias características (unidades, definición) o el esquema de las etiquetas.

Definición 4.5 (Índice de estabilidad poblacional). Sea una variable discretizada en B intervalos, con proporciones p_b en la referencia (entrenamiento) y q_b en una ventana reciente, todas positivas. El *índice de estabilidad poblacional* es

$$\mathrm{PSI}(p,q) = \sum_{b=1}^{B} (q_b - p_b) \ln \frac{q_b}{p_b}.$$

Proposición 4.6 (El PSI es la divergencia simetrizada). $\mathrm{PSI}(p,q) = \mathrm{KL}(p \,\|\, q) + \mathrm{KL}(q \,\|\, p) \geq 0$, *con igualdad si y solo si* $p = q$.

Demostración. $\mathrm{KL}(q\|p) + \mathrm{KL}(p\|q) = \sum_b q_b \ln(q_b/p_b) + \sum_b p_b \ln(p_b/q_b) = \sum_b (q_b - p_b)\ln(q_b/p_b)$. Cada divergencia es no negativa y nula solo si las distribuciones coinciden (módulo 6, tema 2, §2.2). ∎

Ejemplo 4.7 (Cambio de sensor). Tras sustituir el acelerómetro de la llenadora, la vibración diaria se reparte en cuatro intervalos con proporciones $q = (0{,}15;\ 0{,}30;\ 0{,}30;\ 0{,}25)$, frente a $p = (0{,}25;\ 0{,}35;\ 0{,}25;\ 0{,}15)$ en entrenamiento. Entonces $\mathrm{PSI} = (-0{,}10)\ln 0{,}6 + (-0{,}05)\ln\frac{0{,}30}{0{,}35} + 0{,}05\ln 1{,}2 + 0{,}10\ln\frac{0{,}25}{0{,}15} = 0{,}0511 + 0{,}0077 + 0{,}0091 + 0{,}0511 = 0{,}119$. Se usan a menudo umbrales orientativos de 0,1 (vigilar) y 0,25 (actuar), pero son convenciones; es mejor calibrarlos con el PSI observado entre ventanas de un periodo estable. Aquí el valor justifica revisar el sensor antes de reentrenar: un cambio de calibración es un sesgo del sistema (definición 4.1), no un cambio del mundo.

Las métricas de exactitud son la señal más directa de degradación, pero sus etiquetas llegan con retraso (en el caso de estudio, 7 días). Mientras tanto se vigilan las entradas (PSI, contraste de Kolmogorov–Smirnov por variable) y la distribución de las predicciones, más barata por ser de baja dimensión [46]. Muchas alarmas de deriva resultan ser errores internos: valores ausentes, cambios de unidades, canalizaciones rotas.

Proposición 4.8 (Periodo óptimo de reentrenamiento). *Supóngase que, tras reentrenar, la pérdida diaria por degradación del modelo crece linealmente, at (€/día) con $a > 0$, y que cada reentrenamiento (cómputo, validación, despliegue) cuesta c_r. Con un periodo fijo T, el coste medio por día es $c_r/T + aT/2$, que se minimiza en $T^* = \sqrt{2c_r/a}$ con coste $\sqrt{2ac_r}$.*

Demostración. En un ciclo, la pérdida acumulada es $\int_0^T a t\,dt = aT^2/2$; sumando c_r y dividiendo por T se obtiene el coste medio. Por la desigualdad entre medias aritmética y geométrica, $c_r/T + aT/2 \geq 2\sqrt{c_r a/2} = \sqrt{2ac_r}$, con igualdad si $c_r/T = aT/2$, es decir, $T = \sqrt{2c_r/a}$. ■

Es la fórmula del lote económico aplicada al modelo. En la práctica se combina con disparadores: se reentrena antes si la monitorización detecta una deriva o una caída de las métricas, y nunca se despliega un modelo reentrenado sin pasar las mismas validaciones que el primero.

4.4 Evaluar decisiones sin experimentar

Para saber si una política nueva es mejor que la actual hay tres caminos: un experimento en la planta (tema 5), un simulador (el gemelo digital del módulo 26, tema 2) o los registros históricos. Los registros solo muestran el resultado de la decisión que se tomó; el de las demás es *contrafactual*.

Proposición 4.9 (Estimador por ponderación inversa de la propensión). *Sean registros (x_i, a_i, r_i), $i = 1, \dots, n$, independientes, donde el contexto x_i sigue una distribución fija, la acción a_i se eligió con una política de registro conocida $\pi_0(a \mid x)$ y r_i es la recompensa observada. Sea π una política nueva con $\pi(a \mid x) > 0 \Rightarrow \pi_0(a \mid x) > 0$. Entonces*

$$\hat{V}_{\mathrm{IPS}}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \frac{\pi(a_i \mid x_i)}{\pi_0(a_i \mid x_i)}\, r_i$$

es un estimador insesgado del valor $V(\pi) = \mathbb{E}_x \mathbb{E}_{a \sim \pi(\cdot \mid x)}\big[\mathbb{E}[r \mid x, a]\big]$.

Demostración. Para un registro, condicionando en x: $\mathbb{E}\big[\frac{\pi(a|x)}{\pi_0(a|x)} r \,\big|\, x\big] = \sum_{a:\pi_0(a|x)>0} \pi_0(a \mid x) \frac{\pi(a|x)}{\pi_0(a|x)} \mathbb{E}[r \mid x, a] = \sum_a \pi(a \mid x) \mathbb{E}[r \mid x, a]$, donde la última igualdad usa que las acciones con $\pi_0 = 0$ tienen $\pi = 0$. Tomando esperanza en x y promediando los n registros se obtiene $V(\pi)$. ■

La varianza crece con los cocientes π/π_0: si la política antigua casi nunca tomaba la acción que la nueva prefiere, la estimación es inútil. El estimador doblemente robusto de Dudík *et al.* [11] combina un modelo de la recompensa con la ponderación y es consistente si cualquiera de los dos es correcto. La consecuencia de diseño es sencilla: *registrar siempre la acción, el contexto y la probabilidad con que se eligió*; un sistema determinista que no explora nunca permite evaluar contrafactuales con sus propios datos.

4.5 Fiabilidad y mantenimiento

Definición 4.10 (Fiabilidad y tasa de fallo). Sea $T \geq 0$ la vida útil de un componente, con función de distribución F y densidad f. La *fiabilidad* es $R(t) = 1 - F(t)$ y la *tasa de fallo* es $\lambda(t) = f(t)/R(t)$: $\lambda(t)\,dt$ es la probabilidad de fallar en $(t, t + dt)$ habiendo sobrevivido hasta t. Se cumple $R(t) = \exp\big(-\int_0^t \lambda(u)\,du\big)$ [29]. La vida es de tasa creciente (IFR) si λ es no decreciente y de tasa decreciente (DFR) si es no creciente.

Definición 4.11 (Distribución de Weibull). T sigue una Weibull de forma $\beta > 0$ y escala $\theta > 0$ si $R(t) = \exp\big(-(t/\theta)^{\beta}\big)$. Su tasa de fallo es $\lambda(t) = \frac{\beta}{\theta}\big(\frac{t}{\theta}\big)^{\beta-1}$: decreciente si $\beta < 1$ (rodaje), constante si $\beta = 1$ (exponencial, sin memoria) y creciente si $\beta > 1$ (desgaste). Su media es $\theta\,\Gamma(1 + 1/\beta)$.

Observación 4.12 (Censura). En los históricos de mantenimiento, muchas vidas no terminan en fallo: la máquina se intervino antes. Esas observaciones están *censuradas por la derecha*: solo se sabe que $T > t_c$. La verosimilitud correcta es $\prod_{\text{fallos}} f(t_i) \prod_{\text{censuras}} R(t_c)$. Descartar las censuras, o tratarlas como fallos, sesga la estimación porque elimina precisamente las vidas largas.

Proposición 4.13 (Política de sustitución por edad). *Bajo la política «intervenir preventivamente al alcanzar la edad T, o correctivamente al fallar antes», con costes $c_p < c_f$ y renovación completa tras cada intervención, el coste medio a largo plazo por unidad de tiempo es*

$$C(T) = \frac{c_p R(T) + c_f F(T)}{\int_0^T R(u)\,du}.$$

Sea $G(T) = \lambda(T) \int_0^T R(u)\,du - F(T)$. Entonces $C'(T)$ tiene el signo de $G(T) - c_p/(c_f - c_p)$, y $G'(T) = \lambda'(T) \int_0^T R(u)\,du$. En consecuencia:

(i) si la vida es DFR (en particular, Weibull con $\beta \le 1$), C es decreciente y lo óptimo es no intervenir preventivamente ($T^* = \infty$);

(ii) si λ es creciente y $\lambda(T) \to \infty$ (Weibull con $\beta > 1$), existe un único T^* con $G(T^*) = c_p/(c_f - c_p)$, y en él $C(T^*) = (c_f - c_p)\lambda(T^*)$.

Demostración. Cada ciclo de renovación dura $\min(T_{\text{vida}}, T)$, de esperanza $\int_0^T R(u)\,du$, y cuesta c_f con probabilidad $F(T)$ y c_p con probabilidad $R(T)$; el teorema de renovación con recompensas [2, cap. 4] da $C(T)$ como cociente de esperanzas. Escribiendo $N = c_f - (c_f - c_p)R$ y $D = \int_0^T R$, con $N' = (c_f - c_p)f$ y $D' = R$,

$$C' = \frac{N'D - ND'}{D^2} = \frac{R\,(c_f - c_p)}{D^2}\Big[\lambda D - \tfrac{c_f}{c_f - c_p} + R\Big] = \frac{R\,(c_f - c_p)}{D^2}\Big[G - \tfrac{c_p}{c_f - c_p}\Big],$$

porque $R - \frac{c_f}{c_f - c_p} = -F - \frac{c_p}{c_f - c_p}$. Derivando, $G' = \lambda' D + \lambda R - f = \lambda' D$, ya que $f = \lambda R$. (i) Si $\lambda' \le 0$, G es no creciente y $G(0) = 0$, luego $G < c_p/(c_f - c_p)$ y $C' < 0$ para todo T. (ii) Si $\lambda' > 0$, G es creciente desde $G(0) = 0$; como $D \ge \int_0^1 R > 0$ para $T \ge 1$ y $\lambda \to \infty$, $G \to \infty$, de modo que cruza una sola vez el nivel $c_p/(c_f - c_p)$, donde C pasa de decrecer a crecer. En ese punto $N'D = ND'$, luego $C = N/D = N'/D' = (c_f - c_p)f/R = (c_f - c_p)\lambda(T^*)$. ∎

La política por edad usa solo el tiempo. El mantenimiento *predictivo* usa además el estado medido de cada máquina: un clasificador estima la probabilidad de fallo en el horizonte y la decisión se toma con esa probabilidad, como en la decisión con predicciones del módulo 24 (tema 6, §6.1). Si la probabilidad de fallo en los próximos 7 días es p_7, intervenir hoy compensa frente a no intervenir en la semana cuando $c_p < c_f p_7$, es decir, cuando $p_7 > c_p/c_f$; las restricciones de capacidad y de calendario convierten esa regla en un problema de asignación.

4.6 Caso de estudio: mantenimiento predictivo de una línea de envasado

Ejemplo 4.14 (Optimizador de mantenimiento de una línea de envasado). *Planta.* Una línea de envasado tiene seis máquinas en serie: llenadora, taponadora, etiquetadora, encajonadora, paletizadora y enfardadora. Cada máquina se degrada con el uso; una avería

para la línea unas 6 h y, con la reparación, la pérdida de producción y el producto desechado, cuesta $c_f = 9000$ €. Una intervención preventiva cuesta 1200 € si se hace en una de las *ventanas* de cambio de formato (lunes, miércoles y viernes, cuando la línea ya está parada) y 2000 € en cualquier otro momento. Hay un técnico especializado por ventana. Política actual: intervención preventiva cada 70 días.

Datos. Dos años de registros diarios de tres líneas iguales, simulados con un gemelo de degradación: vibración RMS (mm/s), temperatura del rodamiento (ºC), días desde la última intervención, carga de la línea, pendiente de la vibración en dos días y tipo de máquina; averías e intervenciones. Con la política actual hay de media entre 2,5 (paletizadora) y 8,5 (llenadora) averías al año por máquina y un coste de unos 328 000 € al año por línea.

Modelo de supervivencia. Con las 209 vidas que acabaron en avería y las 53 censuradas por una intervención, la Weibull por máxima verosimilitud con censura tiene $\hat\beta = 2{,}58$ y $\hat\theta = 57{,}6$ días (vida media 51,2 días); ignorando la censura se obtendría $\beta = 3{,}37$ y $\theta = 48{,}0$, un desgaste más rápido del real. Con $c_p = 2000$ €, la proposición 4.13 da $T^* = 30$ días y un coste de 111,6 €/día por máquina, frente a 157,3 con la política actual de 70 días y 175,9 sin preventivo (c_f dividido por la vida media). Se comprueba la identidad del punto óptimo: $(c_f - c_p)\lambda(30) = 7000 \cdot 0{,}01596 = 111{,}7$.

Modelo de riesgo. Una regresión logística con las variables estandarizadas, entrenada con los primeros 18 meses, predice la avería en 7 días. En validación (meses 19 a 24) su pérdida logarítmica es 0,2455, algo peor que el mejor bosque del ejemplo 3.9 (0,2378), pero se prefiere por ser interpretable y barata de servir. En una línea de prueba independiente, el área bajo la curva ROC es 0,804 y la calibración por quintiles es razonable (probabilidad media predicha 0,358 frente a una frecuencia observada de 0,324 en el quintil superior). Los coeficientes mayores son los de la vibración (0,77) y la edad (0,42).

Modelo de decisión. Cada lunes, con las probabilidades $p_{7,i}$ de las seis máquinas, se calcula una tasa diaria $\lambda_i = -\ln(1-p_{7,i})/7$ y la probabilidad de fallar antes del día d, $F_i(d) = 1-e^{-\lambda_i d}$. Con las ventanas w de las dos semanas siguientes, en los días $d_w \in \{0, 2, 4, 7, 9, 11\}$, y variables binarias x_{iw} (intervenir la máquina i en la ventana w) y y_i (no intervenir en el horizonte):

$$\min \sum_{i,w} \big(c_p + c_f F_i(d_w)\big) x_{iw} + \sum_i c_f F_i(14)\, y_i \quad \text{s.a.} \quad \sum_w x_{iw} + y_i = 1 \;\; \forall i, \qquad \sum_i x_{iw} \le 1 \;\; \forall w.$$

Se ejecutan solo las decisiones de la primera semana y el lunes siguiente se vuelve a planificar con datos nuevos: es el horizonte deslizante del módulo 26 (tema 3, §3.2). El problema es de transporte; su matriz de restricciones es totalmente unimodular y la relajación lineal ya da soluciones enteras [4, cap. 7].

Cuadro 4.1: Costes de cada opción en un lunes con $p_7 = (0{,}42;\ 0{,}18;\ 0{,}07;\ 0{,}31;\ 0{,}03;\ 0{,}12)$ y horizonte de una semana (en €). En negrita, la solución óptima con un técnico por ventana.

	llenadora	taponadora	etiquetadora	encajonadora	paletizadora	enfardadora
λ_i (1/día)	0,0778	0,0284	0,0104	0,0530	0,0044	0,0183
lunes ($d = 0$)	**1200**	1200	1200	1200	1200	1200
miércoles ($d = 2$)	2497	1696	1385	**2105**	1278	1523
viernes ($d = 4$)	3607	2165	1566	2920	1355	1834
no intervenir	3780	**1620**	**630**	2790	**270**	**1080**

Con un horizonte de una semana (cuadro 4.1), la llenadora, la taponadora y la encajonadora preferirían el lunes, pero solo hay un técnico. El ahorro respecto a no intervenir es $3780-1200 =$

2580 para la llenadora el lunes y $2790 - 2105 = 685$ para la encajonadora el miércoles; la alternativa (encajonadora el lunes y llenadora el miércoles) ahorra $1590 + 1283 = 2873 < 3265$. El coste óptimo es $10\,170 - 3265 = 6905$ €. Con el horizonte de dos semanas, el modelo añade además la taponadora el viernes, porque no intervenirla en catorce días es más arriesgado que en siete.

Cuadro 4.2: Un año de operación de la línea en el gemelo, con los mismos números aleatorios para todas las políticas (módulo 26, tema 5, §5.1).

Política	Coste (€/año)	Averías	Preventivos	Fuera de ventana
A. Solo correctivo	360 000	40	0	—
H. Actual: cada 70 días	327 400	35	7	5
B. Por edad, $T^* = 30$ días	195 200	10	65	34
C. Umbral $p_7 \geq 0{,}10$, sin calendario	173 200	6	70	44
D. Calendario con riesgo (PLEM)	141 000	7	65	0

El cuadro 4.2 resume la simulación del cuaderno. El umbral de la política C (0,10, cercano a c_p/c_f) se eligió en un año de validación distinto. La política C, además de intervenir fuera de ventana, viola la capacidad del técnico cinco días. La política D respeta todas las restricciones y es la más barata. Que la mejora frente a B sea real, y no suerte de este año, es la pregunta del tema 5.

Cuadro 4.3: Tecnologías del tema 4.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
DVC [7]	versionar datos y modelos junto al código, con canalizaciones reproducibles	abierta (Apache 2.0)	equipos pequeños que ya usan git
MLflow [10]	seguimiento de experimentos, registro de modelos y empaquetado	abierta (Apache 2.0)	trazabilidad de qué modelo está en producción
Great Expectations, Feast	validación de esquemas y almacén de características	abiertas (Apache 2.0)	evitar el sesgo de la definición 4.1
Evidently [8]	informes y monitorización de deriva de datos y predicciones	abierta (Apache 2.0)	vigilancia periódica con pocos recursos
FastAPI [9]	servicio de predicción y optimización como API HTTP con esquema	abierta (MIT)	exponer el modelo y el resolvedor a otros sistemas
Kubernetes [25], Ray Serve [40]	despliegue en contenedores, escalado, versiones en paralelo	abiertas (Apache 2.0)	varios servicios con requisitos de disponibilidad
lifelines, scikit-survival	modelos de supervivencia con censura	abiertas (MIT; GPL-3.0)	estimar fiabilidad y riesgo en función del tiempo

Observación 4.15 (Errores frecuentes en la práctica). (a) *Fugas temporales*: características calculadas con datos posteriores al instante de predicción, o etiquetas («fallo en 7 días») que se solapan entre entrenamiento y validación.

(b) *Ignorar la censura*: tratar las intervenciones preventivas como si la máquina no fuera a fallar, o descartarlas (observación 4.12).

(c) *Reentrenar dentro del propio bucle* sin corregir el efecto de las decisiones (observación 4.3).

(d) *Vigilar solo la exactitud*, que llega tarde, y no las entradas ni las predicciones; o vigilarlo todo con umbrales arbitrarios y producir fatiga de alarmas.

(e) *No registrar la decisión humana*: si el técnico anula la propuesta y no queda constancia, no se puede evaluar ni el modelo ni la supervisión.

(f) *Desplegar sin vuelta atrás*: cada modelo en producción debe poder sustituirse por la versión anterior o por la política de respaldo (aquí, la política por edad) en minutos.

Lecturas recomendadas para este tema

- D. Sculley *et al.* [45] y E. Breck *et al.* [5].
- CS329S [46], clases 6 «Offline evaluation», 8 «Deployment» y 10 sobre fallos, cambios de distribución y monitorización; C. Huyen [18], caps. 7–9.
- R. E. Barlow y F. Proschan [2], cap. 4 «Optimum Maintenance Policies».
- M. Dudík, J. Langford y L. Li [11].

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Clasifique según la definición 4.4: (a) la planta empieza a envasar un producto más viscoso y las vibraciones medias suben, pero la relación entre vibración y avería no cambia; (b) un nuevo lubricante hace que, con la misma vibración, las averías sean menos probables; (c) un lote defectuoso de rodamientos aumenta la proporción de máquinas que van a fallar, pero las máquinas que van a fallar y las que no se siguen manifestando igual en los sensores; (d) el proveedor del sensor cambia la unidad de mm/s a pulgadas por segundo.

Fuente: adaptado de CS329S, Stanford, invierno de 2022, clase 10 «Diagnosis of ML system failures, data distribution shifts & monitoring» [46] (ejemplos propios).

Ejercicio 4.2 ⋆ Asocie cada situación a un factor de la definición 4.2: (a) el equipo de calidad usa las probabilidades de avería para decidir inspecciones sin que el equipo de mantenimiento lo sepa; (b) al añadir la temperatura ambiente como variable cambian todos los coeficientes; (c) hay 40 parámetros en un fichero YAML que nadie revisa; (d) el umbral de 0,10 se fijó con los costes de 2024.

Ejercicio 4.3 ⋆⋆ La vida de una cinta transportadora es uniforme entre 0 y 1500 días. (a) Calcule $R(t)$ y $\lambda(t)$. (b) Compare la tasa de fallo a los 0, 500 y 1000 días. (c) Un proveedor ofrece una cinta usada de 500 días «tan buena como nueva»; ¿es cierto? (d) ¿Para qué distribución de la vida sería cierto?

Fuente: adaptado de MIT 1.010 *Uncertainty in Engineering*, otoño de 2008, *Example Application* 8 (función de riesgo), problema 8.1 [29].

Ejercicio 4.4 ⋆⋆ Las proporciones de una variable en cuatro intervalos son, en entrenamiento y en la última semana,

$$p = (0{,}10;\ 0{,}40;\ 0{,}40;\ 0{,}10), \qquad q = (0{,}05;\ 0{,}30;\ 0{,}45;\ 0{,}20).$$

(a) calcule el PSI; (b) calcule $\mathrm{KL}(p\|q)$ y $\mathrm{KL}(q\|p)$ y compruebe la proposición 4.6; (c) con las proporciones acumuladas P_b y Q_b, calcule el estadístico de Kolmogorov–Smirnov, $\max_b |P_b - Q_b|$.

Ejercicio 4.5 ⋆⋆ Una bomba tiene vida exponencial de media 200 días; $c_p = 500$ € y $c_f = 3000$ €. (a) Calcule $C(T)$ para $T = 50$, 100 y ∞ con la proposición 4.13. (b) Explique el resultado con la propiedad de falta de memoria. (c) Para una Weibull con $\beta = 2$, $\theta = 100$ días y los mismos costes, compruebe que $G(T) = c_p/(c_f - c_p)$ tiene solución sabiendo que $\int_0^T e^{-(u/100)^2} du \approx 37{,}97$ para $T = 40$ y $\approx 46{,}13$ para $T = 50$, y acote T^*.

Fuente: adaptado de R. E. Barlow y F. Proschan [2], cap. 4 «Optimum Maintenance Policies» (política de sustitución por edad).

Ejercicio 4.6 ⋆⋆ La política antigua decidía intervenir una máquina con alarma con probabilidad 0,3 y sin alarma con probabilidad 0,1. Se registran seis casos (x, a, r), con r el coste negativo de la semana en miles de euros: (alarma, interviene, $-1{,}2$), (alarma, no, -9), (alarma, no, 0), (sin alarma, no, 0), (sin alarma, interviene, $-1{,}2$), (sin alarma, no, 0). Estime con la proposición 4.9 el valor de la política nueva «intervenir si y solo si hay alarma» y el de la política antigua, y comente la fiabilidad de la estimación.

Ejercicio 4.7 ⋆⋆ Reentrenar y validar el modelo de riesgo cuesta 3000 €; se estima que cada día sin reentrenar la pérdida esperada crece 4 €/día. (a) Calcule el periodo óptimo y el coste medio diario. (b) ¿Cuánto cuesta reentrenar cada 7 días? (c) ¿Y cada 180?

Ejercicio 4.8 ⋆⋆ En un problema con un 12,5 % de positivos, calcule la exactitud y el F_1 de: (a) predecir al azar con la distribución de las etiquetas; (b) predecir al azar con probabilidad 1/2; (c) predecir siempre la clase mayoritaria. ¿Por qué el informe del clasificador de averías no debe dar solo la exactitud?

Fuente: adaptado de CS329S, Stanford, invierno de 2022, clase 6 «Offline evaluation», apartado de líneas base [46] (proporciones cambiadas).

Ejercicio 4.9 ⋆⋆⋆ Diseñe el plan de monitorización del optimizador del ejemplo 4.14: qué se vigila cada día, cada semana y cada mes (entradas, predicciones, decisiones, anulaciones humanas, averías, coste), con qué estadísticos, qué umbrales y cómo se calibran, qué acción dispara cada alarma y cómo se gestiona el retraso de 7 días de las etiquetas. Justifique cómo evita el bucle de la observación 4.3.

Tema 5

Cuantificar el valor y el riesgo

Un optimizador se justifica si su valor esperado supera a su coste y si sus riesgos son aceptables y están controlados. Este tema da las herramientas para afirmarlo con números: intervalos de confianza para la mejora con diseños pareados (capítulo 2 de Montgomery [33]) y con *bootstrap* (lectura 24 del curso 18.05 del MIT [37]), tamaño de un piloto, valor actual neto (sesiones «Present Value Relations» y «Capital Budgeting» del curso 15.401 del MIT [31]), medidas de riesgo de cola (Rockafellar y Uryasev [43]), análisis de modos de fallo (IEC 60812 [20]) y el marco del Reglamento europeo de inteligencia artificial [42, 6]. Se apoya en el módulo 0.9 (temas 4 y 5), en el contraste del ahorro del módulo 26 (tema 6, ejemplo «Contraste del ahorro») y en los experimentos en línea de Kohavi, Tang y Xu [24].

5.1 Coste y beneficio

Definición 5.1 (Valor actual neto). Un proyecto con inversión inicial I y flujos netos $B_t - O_t$ (beneficios menos costes de operación) al final de cada año $t = 1, \dots, N$, con tipo de descuento anual $\rho > 0$, tiene *valor actual neto*

$$\mathrm{VAN} = -I + \sum_{t=1}^{N} \frac{B_t - O_t}{(1+\rho)^t}.$$

La *tasa interna de rentabilidad* (TIR) es el ρ que anula el VAN, y el *plazo de recuperación*, el primer año en que los flujos acumulados sin descontar cubren I.

Proposición 5.2 (Anualidad). *Si $B_t - O_t = b$ es constante,* $\mathrm{VAN} = -I + b\,a_{N,\rho}$ *con* $a_{N,\rho} = \sum_{t=1}^{N}(1+\rho)^{-t} = \dfrac{1-(1+\rho)^{-N}}{\rho}$.

Demostración. Con $v = (1+\rho)^{-1} < 1$, la suma geométrica es $\sum_{t=1}^{N} v^t = v\frac{1-v^N}{1-v}$. Como $\frac{v}{1-v} = \frac{1}{(1+\rho)-1} = \frac{1}{\rho}$, queda $\frac{1-(1+\rho)^{-N}}{\rho}$. ∎

Los beneficios de un optimizador se miden *respecto a una alternativa*, y la elección de la alternativa decide el resultado. La alternativa honrada no es la práctica actual, sino la mejor solución sencilla que se adoptaría sin IA. En el mantenimiento del tema 4, pasar de intervenir cada 70 días a hacerlo cada 30 (política B) no requiere sensores ni modelos de clasificación: basta un análisis de fiabilidad. El valor atribuible al optimizador con IA es la diferencia entre la política D y la B.

5.2 Intervalos de confianza para la mejora

Proposición 5.3 (Ventaja del diseño pareado). *Sean Y_A e Y_B los costes de dos políticas en el mismo periodo, con varianzas σ_A^2, σ_B^2 y correlación r. La diferencia pareada $D = Y_A - Y_B$*

tiene varianza $\sigma_A^2+\sigma_B^2-2r\sigma_A\sigma_B$*, menor que la de la diferencia de dos medidas independientes,* $\sigma_A^2+\sigma_B^2$*, si y solo si* $r>0$*.*

Demostración. $\operatorname{Var}(Y_A-Y_B)=\operatorname{Var}Y_A+\operatorname{Var}Y_B-2\operatorname{Cov}(Y_A,Y_B)$ y $\operatorname{Cov}=r\sigma_A\sigma_B$; con medidas independientes la covarianza es nula. ■

Comparar dos políticas en los mismos días, o en el gemelo con los mismos números aleatorios (módulo 26, tema 5, §5.1), crea esa correlación positiva: la carga, el desgaste y el azar comunes se cancelan. Con n diferencias pareadas de media $\bar d$ y desviación típica s_d, el intervalo de confianza de nivel $1-\alpha$ para la mejora media es $\bar d \pm t_{n-1,1-\alpha/2}\, s_d/\sqrt{n}$ (módulo 0.9, tema 4, §4.4).

Definición 5.4 (Intervalos *bootstrap*). Sean $d_1,\dots,d_n$ las diferencias observadas y $\bar d$ su media. Se generan B remuestras de tamaño n con reemplazamiento, con medias $\bar d_1^*,\dots,\bar d_B^*$. El intervalo *percentil* de nivel $1-\alpha$ es $[\bar d^*_{(\alpha/2)},\bar d^*_{(1-\alpha/2)}]$, con los cuantiles empíricos de las medias remuestreadas. El intervalo *básico* usa $\delta^*=\bar d^*-\bar d$: $[\bar d-\delta^*_{(1-\alpha/2)},\ \bar d-\delta^*_{(\alpha/2)}]$ [37].

El *bootstrap* no exige normalidad, lo que importa aquí: el coste semanal de una línea es casi siempre pequeño y, de vez en cuando, salta en 9000 € por una avería. Ambos métodos suponen observaciones independientes; si las semanas están autocorrelacionadas hay que remuestrear por bloques.

Proposición 5.5 (Tamaño de un piloto). *Para detectar una mejora media* $\delta>0$ *con un contraste bilateral de nivel* α *y potencia* $1-\beta$ *con diferencias pareadas de desviación típica* σ_D*, hacen falta aproximadamente*

$$n \approx \Big(\frac{(z_{1-\alpha/2}+z_{1-\beta})\,\sigma_D}{\delta}\Big)^2$$

pares (módulo 0.9, tema 5, §5.4).

Demostración. Con la aproximación normal, $\bar D\sim\mathcal{N}(\delta,\sigma_D^2/n)$. Despreciando la cola opuesta, se rechaza la hipótesis nula cuando $\bar D>z_{1-\alpha/2}\,\sigma_D/\sqrt{n}$. La potencia es

$$\mathbb{P}\Big(Z>z_{1-\alpha/2}-\frac{\delta\sqrt{n}}{\sigma_D}\Big),$$

que vale $1-\beta$ si $z_{1-\alpha/2}-\delta\sqrt{n}/\sigma_D=-z_{1-\beta}$; despejando n se obtiene la fórmula. ■

5.3 Caso de estudio: ¿cuánto vale el optimizador de mantenimiento?

Ejemplo 5.6 (Valoración del optimizador del ejemplo 4.14). *Un año de piloto.* Con los costes semanales del año de prueba del cuadro 4.2, las diferencias pareadas entre la política actual y la nueva (H menos D) tienen media 3585 €/semana y desviación típica 6600 €; el intervalo t del 95 % es [1747; 5422] €/semana, es decir, entre 91 000 y 282 000 €/año, y el intervalo *bootstrap* percentil (10 000 remuestras) es casi idéntico, [1858; 5427]. La mejora frente a la práctica actual es clara. Frente a la alternativa sencilla (B menos D), la media es 1042 €/semana con desviación típica 4356 €: $t=1{,}73$ con 51 grados de libertad, intervalo [−170; 2255] €/semana y *bootstrap* [−119; 2254]. Un año de datos *no basta* para afirmar con

un 95 % de confianza que la IA aporta algo sobre un buen plan por edad. La autocorrelación de un retardo de las diferencias es 0,05, de modo que la hipótesis de independencia es aceptable.

Cuántas semanas harían falta. El gemelo, con 20 años simulados con números aleatorios comunes, estima una mejora de B a D de 34 370 €/año (desviación típica entre años 13 637; intervalo del 95 % [27 987; 40 753]), es decir, $\delta = 661$ €/semana. Por la proposición 5.5, con $\alpha = 0{,}05$ y potencia 0,8, $n \approx (2{,}80 \cdot 4356/661)^2 \approx 341$ semanas. Un piloto en una línea durante un año no puede demostrar esa mejora; las opciones razonables son desplegar en varias líneas a la vez, aceptar la evidencia del gemelo validado más un periodo en sombra (módulo 26, tema 6, §6.4), o medir indicadores intermedios con menos ruido, como las horas de parada no planificada.

Valor actual neto. Inversión: 9000 € en sensores (seis a 1500 €), 25 000 en integración con el GMAO y la API, 10 000 en datos y modelos y 4000 en formación; total $I = 48\,000$ €. Operación: 9000 €/año (servidores, mantenimiento del modelo y de los sensores). Horizonte de 5 años y $\rho = 8\,\%$, con $a_{5;0{,}08} = 3{,}9927$. Frente a la política B, VAN $= (34\,370 - 9000) \cdot 3{,}9927 - 48\,000 = 53\,295$ €, TIR del 44 % y recuperación en $48\,000/25\,370 = 1{,}9$ años. Frente a la práctica actual, el VAN sería de casi 599 000 €, una cifra correcta pero engañosa si se atribuye a la IA.

Riesgo del VAN. Con Monte Carlo (módulo 5, tema 5, §5.3), beneficios anuales independientes $\mathcal{N}(34\,370;\ 13\,637^2)$, un sobrecoste de la inversión uniforme entre 0 y 30 % y costes de operación entre −10 % y +30 % de lo previsto, el VAN medio baja a 42 600 € y la probabilidad de VAN negativo es del 4,6 %.

5.4 Robustez y riesgo de cola

La media no lo es todo: a una planta le preocupa el año malo. Para una pérdida L con distribución continua, el *valor en riesgo* al nivel α es el cuantil $\mathrm{VaR}_\alpha(L) = F_L^{-1}(\alpha)$, y el *valor en riesgo condicional* es la media de la cola, $\mathrm{CVaR}_\alpha(L) = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha(L)]$.

Proposición 5.7 (Fórmula de Rockafellar y Uryasev). *Si L tiene densidad y esperanza finita,*

$$\mathrm{CVaR}_\alpha(L) = \min_{\tau \in \mathbb{R}} \Big\{ \tau + \frac{1}{1-\alpha} \mathbb{E}\big[(L-\tau)^+\big] \Big\},$$

y el mínimo se alcanza en $\tau = \mathrm{VaR}_\alpha(L)$.

Demostración. La función $\phi(\tau) = \tau + \mathbb{E}[(L-\tau)^+]/(1-\alpha)$ es convexa (suma de convexas) y derivable, con $\phi'(\tau) = 1 - \mathbb{P}(L > \tau)/(1-\alpha)$, que se anula cuando $\mathbb{P}(L > \tau) = 1 - \alpha$, es decir, en $\tau^* = \mathrm{VaR}_\alpha$. En ese punto, como $\mathbb{P}(L > \tau^*) = 1 - \alpha$, $\phi(\tau^*) = \tau^* + \mathbb{E}[(L-\tau^*)\mathbf{1}[L > \tau^*]]/\mathbb{P}(L > \tau^*) = \tau^* + \mathbb{E}[L - \tau^* \mid L > \tau^*] = \mathbb{E}[L \mid L > \tau^*]$. ∎

La fórmula convierte la minimización del CVaR de una decisión en un programa lineal cuando L se representa con escenarios: basta añadir τ y una variable $u_s \geq L_s - \tau$, $u_s \geq 0$ por escenario [43]. En los 20 años simulados del caso, el coste anual de la política D tiene media 159 300 € y CVaR al 90 % (media de los dos peores años) de 186 300 €; la política B, 193 670 y 216 200 €. D es mejor también en la cola. Las *pruebas de estrés* completan el análisis: repetir la evaluación con un coste de avería un 50 % mayor, con un técnico de baja, con un sensor que se congela en su último valor o con una semana de datos perdidos.

5.5 Seguridad, supervisión y modos de fallo

Los principios del módulo 26 (tema 6) se aplican a cualquier optimizador. (i) Las *restricciones duras* (capacidad de los técnicos, inspecciones obligatorias por ley, bloqueos de seguridad de las máquinas) están en el modelo de decisión y se comprueban otra vez, de forma independiente, antes de emitir una orden. (ii) Hay una *política de respaldo* sencilla (aquí, la política por edad) a la que se vuelve si el modelo o los datos fallan. (iii) El despliegue recorre los modos asesor, supervisado y autónomo con supervisión (módulo 26, tema 6, §6.2), y en cada uno se registra quién decidió qué.

Definición 5.8 (Análisis modal de fallos y efectos). El AMFE (FMEA en inglés; IEC 60812 [20]) enumera los modos de fallo de cada componente, sus efectos y sus causas, y los prioriza. En la variante clásica se puntúan de 1 a 10 la *gravedad* S, la *ocurrencia* O y la *detectabilidad* Dt (10 = indetectable), y se ordena por el *número de prioridad de riesgo* $\mathrm{NPR} = S \cdot O \cdot Dt$; la norma admite otras formas de calcularlo y, en todo caso, los modos de gravedad máxima se tratan aunque su NPR sea bajo.

Cuadro 5.1: AMFE resumido del optimizador de mantenimiento.

Modo de fallo	Efecto	S	O	Dt	NPR	Medida
sensor congelado en su último valor	riesgo infravalorado; avería no prevista	8	3	7	168	regla de varianza nula; respaldo por edad
deriva tras cambio de producto	probabilidades mal calibradas	6	5	4	120	PSI semanal; recalibración
resolvedor sin solución a tiempo	sin plan el lunes	5	2	2	20	plan de la semana anterior; límite de tiempo
confianza excesiva del técnico	se ejecutan planes erróneos	7	4	6	168	mostrar motivos; auditoría de anulaciones
etiquetas mal registradas en el GMAO	modelo entrenado con ruido	6	4	6	144	validación de registros; revisión mensual

5.6 El Reglamento europeo de inteligencia artificial

El Reglamento (UE) 2024/1689 [42] entró en vigor el 1 de agosto de 2024 y sigue un enfoque basado en el riesgo: prohíbe algunas prácticas, impone requisitos a los sistemas de *alto riesgo*, establece obligaciones de transparencia para otros y deja el resto sin obligaciones específicas. Lo que sigue es una descripción general de su texto público; la clasificación de un sistema concreto es una cuestión jurídica.

1. *Alto riesgo* (artículo 6). Lo es un sistema que sea componente de seguridad de un producto, o el propio producto, regulado por la legislación de armonización del anexo I y sujeto a evaluación de la conformidad por terceros; y lo son los sistemas del anexo III, entre ellos los componentes de seguridad en la gestión y operación de infraestructuras críticas, como el suministro de agua, gas, calefacción o electricidad (anexo III, punto 2). Un sistema del anexo III no es de alto riesgo si no influye de forma sustancial en la decisión, por ejemplo si realiza una tarea procedimental limitada o preparatoria (artículo 6,

apartado 3), salvo que elabore perfiles de personas.

2. *Requisitos* (artículos 8 a 15): sistema de gestión de riesgos, gobernanza de los datos, documentación técnica, registro automático de eventos, transparencia hacia quien lo despliega, supervisión humana y precisión, robustez y ciberseguridad. El artículo 14 pide que las personas que supervisan puedan entender las capacidades y los límites del sistema y detectar anomalías, sean conscientes del sesgo de automatización, interpreten correctamente sus salidas, puedan decidir no usarlo o anular sus resultados, e intervenir o detenerlo de forma segura.
3. *Calendario.* El reglamento se aplica con carácter general desde el 2 de agosto de 2026. La modificación conocida como «ómnibus digital», publicada en el Diario Oficial el 24 de julio de 2026 y en vigor desde el 27 de julio, aplaza las obligaciones de alto riesgo: al 2 de diciembre de 2027 para los sistemas del anexo III y al 2 de agosto de 2028 para los del anexo I; según la información publicada, la maquinaria pasa a un régimen sectorial propio [6, 42].

Aplicado al caso: un optimizador del calendario de mantenimiento de una línea de envasado, cuyas órdenes revisa un técnico y que no actúa sobre las funciones de seguridad de las máquinas, no parece encajar en ninguna categoría de alto riesgo. El mismo optimizador aplicado a las bombas de una red de abastecimiento de agua, si actuara como componente de seguridad de su operación, sí podría serlo. En ambos casos, la arquitectura de los temas 4 y 5 (registro de decisiones, supervisión humana con capacidad de anular, política de respaldo, monitorización, documentación) cumple de forma natural buena parte de los requisitos, y es buena práctica adoptarla aunque no sea obligatoria. Si se usan datos de personas (por ejemplo, qué técnico anuló qué orden), se aplica además la normativa de protección de datos.

Cuadro 5.2: Tecnologías del tema 5.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
SciPy (`stats`)	contrastes t, intervalos y *bootstrap* (función `bootstrap`)	abierta (BSD-3)	análisis de pilotos y de simulaciones
statsmodels	modelos con errores autocorrelacionados, potencia y tamaño muestral	abierta (BSD-3)	series semanales, diseños con covariables
numpy-financial	VAN, TIR y anualidades	abierta (BSD-3)	valoración económica reproducible
MLflow [10]	registrar cada evaluación con su modelo, datos y resultado	abierta (Apache 2.0)	trazabilidad exigida por la documentación técnica
Plantillas de AMFE (hoja de cálculo)	análisis de modos de fallo	—	revisión de riesgos con operación, mantenimiento y seguridad

Observación 5.9 (Errores frecuentes en la práctica). (a) *Comparar con la práctica actual* en lugar de con la mejor alternativa sencilla, y atribuir a la IA mejoras que no necesitan IA.

(b) *Comparar antes y después* sin grupo de control ni números aleatorios comunes: cambian la producción, el clima o los productos.

(c) *Piloto infradimensionado*: concluir «no hay efecto» de un intervalo que contiene el cero cuando el piloto no tenía potencia para detectarlo (proposición 5.5).

(d) *Informar solo de la media*: sin intervalo, sin cola y sin pruebas de estrés.

(e) *Olvidar los costes de operación*: el modelo, los sensores y la monitorización cuestan cada año.

(f) *Dejar la regulación para el final*: la supervisión humana, el registro y la documentación son decisiones de arquitectura, baratas al principio y caras después.

Lecturas recomendadas para este tema

- D. C. Montgomery [33], cap. 2 «Simple Comparative Experiments».
- MIT 18.05 [37], lectura 24 «Bootstrap confidence intervals».
- MIT 15.401 [31], sesiones «Present Value Relations» y «Capital Budgeting».
- R. Kohavi, D. Tang y Y. Xu [24], sobre experimentos controlados.
- Reglamento (UE) 2024/1689 [42], artículos 6, 9 y 14, y anexo III.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ Una empresa presenta su optimizador de rutas con IA diciendo que ahorra un 18 % de kilómetros respecto a las rutas que hacían los conductores. Enumere tres alternativas sin IA con las que debería compararse, y explique qué parte del 18 % podría no deberse a la IA.

Ejercicio 5.2 ⋆⋆ El coste semanal de una línea con la política B tiene desviación típica 6500 € y con la política D, 5200 €. (a) Calcule la desviación típica de la diferencia si las semanas se comparan en líneas distintas (independientes) y si se comparan con números aleatorios comunes con correlación 0,78. (b) ¿Cuántas veces más semanas haría falta en el primer caso para la misma precisión?

Fuente: adaptado de D. C. Montgomery [33], cap. 2 «Simple Comparative Experiments», diseño de comparaciones pareadas.

Ejercicio 5.3 ⋆⋆ Los ahorros semanales (en cientos de euros) de diez semanas de piloto son $30, 37, 36, 43, 42, 43, 43, 46, 41, 42$, con media 40,3. En 20 remuestras, las medias ordenadas de menor a mayor tienen el segundo valor en 39,4 y el decimoctavo en 42,4. (a) Dé el intervalo percentil del 80 %. (b) Dé el intervalo básico del 80 %. (c) Calcule el intervalo t del 80 % con $s = 4{,}67$ y $t_{9;\,0{,}90} = 1{,}383$. (d) ¿Por qué en la práctica se usan 10 000 remuestras?

Fuente: adaptado de Orloff y Bloom, MIT 18.05, primavera de 2022, lectura 24 «Bootstrap confidence intervals», ejemplo 6 [37].

Ejercicio 5.4 ⋆⋆ Para el ejemplo 5.6: (a) compruebe $a_{5;0{,}08} = 3{,}9927$ con la proposición 5.2; (b) recalcule el VAN si el tipo de descuento es del 12 %; (c) ¿qué beneficio anual mínimo hace nulo el VAN con $\rho = 8\,\%$?; (d) ¿y si el horizonte es de 3 años?

Fuente: adaptado de MIT 15.401 *Finance Theory I*, otoño de 2008, sesión «Present Value Relations» (valor actual de una anualidad) [31].

Ejercicio 5.5 ⋆⋆ Con $\sigma_D = 4356$ €/semana y $\delta = 661$ €/semana: (a) calcule el número de semanas con $\alpha = 0{,}05$ y potencia 0,9; (b) si el piloto se hace en 6 líneas independientes a la vez, ¿cuántas semanas de calendario hacen falta con potencia 0,8?; (c) si se mide en su lugar la parada no planificada, con $\sigma_D = 3{,}1$ h/semana y $\delta = 0{,}9$ h/semana, ¿cuántas semanas hacen falta con potencia 0,8?

Ejercicio 5.6 ⋆⋆ Los costes anuales (en miles de euros) de una política en 10 escenarios son $152, 148, 171, 160, 139, 183, 157, 166, 149, 175$. (a) Calcule la media, el VaR y el CVaR al 80 % (dos peores escenarios). (b) Compruebe que la función $\phi(\tau)$ de la proposición 5.7, con la distribución empírica, vale lo mismo en $\tau = 175$ y que es mayor en $\tau = 160$ y en $\tau = 183$.

Ejercicio 5.7 ⋆⋆ Añada al cuadro 5.1 dos modos de fallo: (a) un ciberataque modifica las probabilidades enviadas por la API ($S = 9$, $O = 2$, $Dt = 8$); (b) el modelo se reentrena con datos de una semana de vacaciones ($S = 5$, $O = 3$, $Dt = 5$). Calcule los NPR, ordene los siete modos y proponga medidas para los dos primeros.

Ejercicio 5.8 ⋆⋆⋆ Para cada sistema, razone en términos generales si podría ser de alto riesgo según el artículo 6 y el anexo III, qué información adicional haría falta y qué medidas de los artículos 9 a 15 adoptaría en cualquier caso: (a) el optimizador de mantenimiento del caso de estudio; (b) un sistema que abre y cierra automáticamente las válvulas de una red de calefacción urbana para evitar sobrepresiones; (c) un asistente con LLM que redacta borradores de órdenes de trabajo que un técnico revisa.

Tema 6

Guía para construir un optimizador con IA en cualquier ámbito

Este tema reúne la fase VI en un procedimiento. No es un algoritmo, sino una disciplina de ingeniería: el orden en que conviene tomar las decisiones y los documentos que deben quedar escritos. Sigue la estructura del curso 16.842 «Fundamentals of Systems Engineering» del MIT [30], que recorre el modelo en V (partes interesadas y concepto de operación, requisitos con márgenes, arquitectura, exploración del espacio de alternativas, revisión preliminar del diseño), las clases 1 y 2 del curso CS329S de Stanford [46] y el capítulo 2 de Huyen [18] para el planteamiento de sistemas con aprendizaje automático, y la lista de pruebas de Breck *et al.* [5]. Cada paso remite al módulo donde se estudió su contenido.

6.1 Los doce pasos

1. **La decisión y sus responsables.** ¿Qué se decide, quién lo decide hoy, con qué frecuencia y con qué horizonte? ¿Quién usará el sistema, quién responde de sus errores y quién puede pararlo? Entregable: concepto de operación de una página.
2. **Objetivo, indicadores y restricciones.** Un objetivo escalar, los indicadores que se vigilan sin optimizarlos, las restricciones duras (seguridad, ley, física) y las blandas, cada una con su unidad y su fuente (módulo 24, tema 1). Entregable: requisitos numéricos con márgenes.
3. **Línea base y techo de valor.** Coste de la práctica actual, de la mejor alternativa sencilla sin IA y de una política con información perfecta (proposición 6.1). Si el techo no compensa el coste del proyecto, se para aquí.
4. **Datos.** Inventario de fuentes, frecuencias, calidad, retrasos, disponibilidad en el instante de decidir, propiedad y protección; versionado desde el primer día (tema 4).
5. **Formulación y familia de métodos.** El mapa del módulo 24 (tema 1, §1.3): programación matemática, optimización bayesiana para experimentos caros (módulo 25), control y RL para decisiones secuenciales (módulo 26), optimización geométrica (módulo 27), búsqueda de programas cuando lo que se busca es una regla (tema 1).
6. **Modelos predictivos o sustitutos.** Qué se predice, con qué horizonte y con qué métrica de decisión; validación temporal y selección con presupuesto (tema 3).
7. **Resolvedor, salvaguardas y respaldo.** Tiempo máximo de cálculo, comprobación independiente de las restricciones duras, política de respaldo.
8. **Evaluación fuera de línea.** Sobre históricos (sin fugas), en un gemelo con números aleatorios comunes y, si hay registros con propensiones, con estimadores contrafactuales (tema 4).

9. **Piloto.** Modo sombra y después experimento con tamaño calculado (proposición 5.5) y criterio de aceptación escrito antes de empezar.
10. **Despliegue.** API con contrato, interfaz que explica y permite anular, registro de decisiones, versiones y vuelta atrás.
11. **Monitorización y reentrenamiento.** Entradas, predicciones, decisiones, anulaciones e indicadores; disparadores y periodo de reentrenamiento (proposición 4.8).
12. **Valor, riesgo, normativa y ciclo de vida.** Intervalos para la mejora, VAN, AMFE, encaje regulatorio, documentación y criterios de retirada (tema 5).

Proposición 6.1 (Techo de valor). *Sea $f(x,\theta)$ el coste de la decisión $x \in \mathcal{X}$ cuando el estado incierto es θ, y sea $\hat{x}$ cualquier regla de decisión que solo use información disponible antes de decidir. Entonces*

$$\mathbb{E}\big[f(\hat{x},\theta)\big] \;\geq\; \mathbb{E}\Big[\min_{x\in\mathcal{X}} f(x,\theta)\Big].$$

En consecuencia, ninguna mejora de la predicción puede reducir el coste esperado de la mejor regla sin información por debajo del coste con información perfecta, y la diferencia entre ambos (el valor de la información perfecta del módulo 24, tema 6, definición 6.6) acota lo que puede ganarse prediciendo mejor.

Demostración. Para cada realización de θ y de la información, $\hat{x}$ es un punto de $\mathcal{X}$, luego $f(\hat{x},\theta) \geq \min_{x\in\mathcal{X}} f(x,\theta)$. Tomando esperanzas se conserva la desigualdad. ■

En el caso del mantenimiento, una política que conoce el futuro e interviene cada máquina en la última ventana antes de que fuera a fallar cuesta de media 76 170 €/año en el gemelo (sin límite de técnicos), frente a 193 670 de la política por edad y 159 300 del optimizador. Este solo captura un 29 % del margen que deja la alternativa sencilla frente a la información perfecta: hay espacio para mejores sensores o mejores modelos, y el paso 3 permite estimar cuánto valdrían antes de pagarlos.

6.2 Plantilla de documento de diseño

El cuadro 6.1 es la plantilla que se rellena antes de escribir código y se actualiza en cada revisión. Su objetivo no es la burocracia, sino forzar que las decisiones difíciles (qué es éxito, qué no se puede violar, contra qué se compara) se tomen pronto y por escrito, como en la revisión preliminar de diseño de 16.842 [30].

6.3 Mini-caso A: secuenciación de formatos en una fábrica de bebidas

Ejemplo 6.2 (Cambios de formato). *1–2. Decisión y objetivo.* Cada semana, el jefe de producción fija el orden en que una línea embotella cuatro formatos: agua de 0,5 L (A), agua de 1,5 L (B), zumo de 1 L (C) y bebida de soja (D, con alérgeno). El objetivo es minimizar el tiempo total de cambio de formato; restricción dura: tras D, limpieza completa de 90 min antes de cualquier otro producto (seguridad alimentaria); indicadores: nivel de servicio y existencias. *3. Línea base.* El orden actual es A–C–D–B, con un ciclo de $45 + 30 + 90 + 25 = 190$ min (tiempos de cambio en minutos: A→B 20, A→C 45, A→D 40, B→A 25, B→C 35, B→D

Cuadro 6.1: Plantilla del documento de diseño de un optimizador con IA.

Sección	Preguntas que debe responder
1. Contexto y decisión	¿Qué se decide, cada cuánto, con qué horizonte? ¿Qué pasa hoy?
2. Partes interesadas	¿Quién decide, quién ejecuta, quién responde, quién puede parar el sistema?
3. Objetivo y requisitos	Objetivo, indicadores, restricciones duras y blandas con unidades; requisitos de latencia, disponibilidad y precisión con márgenes.
4. Línea base y techo	Coste actual, mejor alternativa sencilla, coste con información perfecta; valor máximo alcanzable.
5. Datos	Fuentes, calidad, retrasos, disponibilidad en el instante de decidir, versionado, protección.
6. Formulación y método	Modelo matemático; familia elegida y alternativas descartadas con su motivo.
7. Modelos y validación	Qué se predice; división temporal; métricas de predicción y de decisión; selección.
8. Salvaguardas y respaldo	Comprobaciones independientes; política de respaldo; tiempo máximo de cálculo.
9. Evaluación fuera de línea	Históricos, gemelo, contrafactual; resultados con intervalos.
10. Piloto	Diseño, tamaño, duración, criterio de aceptación fijado de antemano.
11. Despliegue y supervisión	API, interfaz, modos de operación, registro, vuelta atrás.
12. Monitorización	Señales, umbrales, acciones, reentrenamiento.
13. Valor y riesgo	VAN con incertidumbre, AMFE, pruebas de estrés, encaje normativo.
14. Ciclo de vida	Responsable del mantenimiento, revisiones periódicas, criterios de retirada.

45, C→A 60, C→B 55, C→D 30, D→cualquiera 90). *5. Método.* Es un viajante asimétrico (módulo 24, tema 3); con cuatro formatos basta enumerar los $3! = 6$ ciclos que empiezan en A; con veinte se usaría CP-SAT o un PLEM. Los tamaños de lote salen de la previsión de demanda (módulo 24, tema 6). *Resultado.* El ciclo A–B–C–D cuesta 20+35+30+90 = 175 min, el mínimo de los seis (los demás: 190, 210, 215, 225 y 235). Con tres ciclos por semana se ahorran 45 min de línea parada, unos 1350 €/semana a 1800 €/h. *7–12.* Salvaguarda: el plan se valida contra las reglas de alérgenos del sistema de calidad antes de publicarse; respaldo: el orden actual; seguimiento: minutos de cambio reales frente a planificados.

6.4 Mini-caso B: presupuesto de geometría en un videojuego

Ejemplo 6.3 (Niveles de detalle con presupuesto). *1–2.* En la compilación de cada escena, una herramienta elige el nivel de detalle (LOD) de cada objeto para minimizar el error visual en pantalla con un presupuesto de triángulos que garantiza los 16,7 ms por fotograma en la consola de referencia (restricción dura), como en la selección de niveles de detalle del módulo 27 (tema 5, §5.4). Regla de dirección de arte: el personaje principal nunca baja de LOD0. *3–5.* Línea base: un LOD fijo por distancia. Es una mochila de elección múltiple (PLEM); los parámetros del simplificador de mallas se ajustan aparte con optimización bayesiana (módulo 25). *Datos de una escena*, en miles de triángulos y error ponderado en píxeles: personaje (LOD0: 60 y 0; LOD1: 25 y 1,0; LOD2: 10 y 4,0), edificio (80 y 0; 30 y 0,6; 12 y 2,5) y árboles (50 y 0; 15 y 0,8; 5 y 3,0); presupuesto: 100 000 triángulos. *Resultado.* Sin la regla, lo óptimo es LOD1 para los tres (70 000 triángulos, error 2,4). Con el personaje en LOD0, lo óptimo es edificio en LOD2 y árboles en LOD1 (87 000 triángulos, error 3,3): la regla cuesta 0,9 unidades de error, un dato útil para discutirla con dirección de arte. *8–11.* Evaluación

fuera de línea en integración continua: se renderizan escenas de prueba con cámaras fijas y se mide el percentil 99 del tiempo de fotograma; seguimiento con la telemetría de las versiones de prueba; revisión humana de los recursos principales.

6.5 Mini-caso C: asignación de trabajos en un centro de datos

Ejemplo 6.4 (Heurística de colocación y refrigeración). *1–2.* Un operador de centros de datos decide en línea en qué servidor se coloca cada máquina virtual (CPU y memoria). Objetivo: minimizar los *recursos varados*, es decir, la capacidad libre de un recurso que no se puede usar porque el otro está agotado; restricciones duras: no sobrecomprometer la memoria y respetar las reglas de antiafinidad. *3.* Línea base: *best fit* sobre el recurso dominante. Caso publicado: la heurística de AlphaEvolve para Borg recupera de media un 0,7 % de la capacidad de cómputo de la flota de Google [14]. Como orden de magnitud hipotético (no extrapolable sin evaluación propia), en una flota de 20 000 servidores con un coste total de 2500 € por servidor y año, recuperar un 0,7 % equivaldría a 140 servidores, unos 350 000 €/año. *5–8.* Método: búsqueda de programas (tema 1) sobre la función de prioridad, con un evaluador que reproduce trazas históricas en un simulador del planificador; separación de las trazas por semanas y por clúster para medir la generalización; la regla resultante se lee y se simplifica antes de desplegarla. *9–11.* Despliegue gradual en un clúster de prueba con la regla actual como respaldo y vigilancia de los recursos varados y de los trabajos rechazados. La *refrigeración* del mismo centro es otro optimizador, de control, con su propio caso publicado (módulo 26, tema 5, §5.5) [13].

Cuadro 6.2: Tecnologías del tema 6: pila de referencia de un optimizador con IA.

Capa	Herramientas habituales	Licencia o tipo	Cuándo elegirla
Datos y versionado	DVC [7], Great Expectations	abiertas (Apache 2.0)	desde el primer prototipo
Modelos	scikit-learn, PyTorch; Optuna [1] para el ajuste	abiertas (BSD-3; BSD-3; MIT)	según el tipo de datos (tema 3)
Decisión	SciPy/HiGHS, OR-Tools, CVXPY (módulo 24); BoTorch (módulo 25)	abiertas	según la estructura del problema
Búsqueda con LLM	OpenEvolve [35], DSPy [22]	abiertas (Apache 2.0; MIT)	reglas o cadenas de llamadas con evaluador fiable
Servicio	FastAPI [9], contenedores, Kubernetes [25]	abiertas (MIT; Apache 2.0)	integración con otros sistemas
Seguimiento	MLflow [10], Evidently [8]	abiertas (Apache 2.0)	trazabilidad y deriva
Escala	Ray [40]	abierta (Apache 2.0)	simulaciones y ajustes en paralelo

Observación 6.5 (Errores frecuentes en la práctica). (a) *Empezar por el modelo*: elegir la técnica antes de definir la decisión, el objetivo y la línea base.

(b) *No calcular el techo* (proposición 6.1) y descubrir al final que el problema no daba para

más.

(c) *Requisitos sin margen*: si la latencia máxima es 200 ms y cada componente se diseña para 200 ms, el sistema no la cumple.

(d) *Optimizar el indicador equivocado*: el sistema hace exactamente lo que dice el objetivo, no lo que se quería decir.

(e) *Olvidar a las personas*: un optimizador que los usuarios no entienden se anula o se ignora; uno en el que confían demasiado propaga sus errores.

(f) *Proyecto sin dueño tras el despliegue*: sin responsable de mantenimiento, el modelo se degrada en silencio.

Lecturas recomendadas para este tema

- MIT 16.842 [30], programa del curso y tareas A1–A5.
- CS329S [46], clases 1 «Understanding machine learning production» y 2 «ML and Data Systems Fundamentals»; C. Huyen [18], cap. 2.
- E. Breck *et al.* [5], como lista de comprobación antes del despliegue.

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Un equipo propone: «entrenaremos un transformer con los datos de los sensores, lo desplegaremos en la nube y, cuando funcione, mediremos el ahorro». Identifique qué pasos de la guía faltan o están desordenados y escriba el orden correcto de las cinco primeras tareas.

Ejercicio 6.2 ⋆⋆ El planificador semanal de mantenimiento debe responder en 2 s como máximo desde que el técnico pulsa «recalcular». Componentes: lectura de datos, cálculo de características, predicción, resolución del programa entero y presentación. (a) Reparta el tiempo dejando un margen del 25 % en el nivel de sistema y explique cómo asignaría el resto según la variabilidad de cada componente. (b) Si las mediciones de los componentes en el percentil 99 son 0,30; 0,15; 0,05; 0,70 y 0,20 s, ¿se cumple el requisito con margen? ¿Qué componente revisaría?

Fuente: adaptado de MIT 16.842 *Fundamentals of Systems Engineering*, otoño de 2015, tarea A2 «Requirements definition and analysis, margins allocation» [30].

Ejercicio 6.3 ⋆⋆ Para el mantenimiento se consideran tres conceptos: (I) reglas por edad, (II) predicción y programa entero, (III) aprendizaje por refuerzo sobre el gemelo. Con pesos 0,4 (coste esperado), 0,2 (interpretabilidad), 0,2 (esfuerzo de implantación) y 0,2 (riesgo), y puntuaciones de 1 a 5 (5 = mejor) I: $(2, 5, 5, 4)$, II: $(4, 4, 3, 4)$, III: $(5, 2, 1, 2)$: (a) calcule la puntuación ponderada; (b) determine qué conceptos son de Pareto; (c) discuta la sensibilidad al peso del coste: ¿para qué peso (repartiendo el resto a partes iguales) empataría III con II?

Fuente: adaptado de MIT 16.842 *Fundamentals of Systems Engineering*, otoño de 2015, tarea A4 «Tradespace exploration, concept selection» [30].

Ejercicio 6.4 ⋆⋆ En el caso del mantenimiento: la práctica actual cuesta 330 290 €/año, la política por edad 193 670, el optimizador 159 300 y la política con información perfecta 76 170

(medias del gemelo). (a) ¿Qué fracción del margen total (práctica actual frente a información perfecta) captura la política por edad, y qué fracción el optimizador? (b) Un sensor nuevo cuesta 30 000 € de inversión y 3000 €/año; ¿qué fracción del margen restante debería capturar en 5 años al 8 % para tener VAN positivo?

Ejercicio 6.5 ⋆⋆ En el ejemplo 6.2, el área de calidad exige además que el zumo (C) no vaya inmediatamente después de la bebida de soja (D) aunque haya limpieza. (a) ¿Cambia el óptimo? (b) Si el tiempo de limpieza tras D baja a 60 min, recalcule los seis ciclos. (c) Con 8 formatos, ¿cuántos ciclos distintos habría que enumerar?

Ejercicio 6.6 ⋆⋆ En el ejemplo 6.3, (a) compruebe por enumeración el óptimo con y sin la regla del personaje; (b) ¿qué presupuesto mínimo hace falta para que la regla no cueste nada? (c) formule el problema general como PLEM con variables binarias z_{ok}.

Ejercicio 6.7 ⋆⋆ Se evalúa el optimizador en dos plantas. En la planta 1 evita 81 de 87 averías previsibles y la política anterior 234 de 270; en la planta 2, 192 de 263 y 55 de 80. (a) Calcule las tasas por planta y globales. (b) Explique la paradoja. (c) ¿Cómo debe presentarse la evaluación en el documento de diseño?

Fuente: adaptado de CS329S, Stanford, invierno de 2022, clase 6 «Offline evaluation», ejemplo de evaluación por segmentos y paradoja de Simpson [46].

Ejercicio 6.8 ⋆⋆⋆ Rellene la plantilla del cuadro 6.1 para un optimizador de la climatización de un almacén frigorífico de 20 000 m^3 con tarifa eléctrica horaria, placas solares y una restricción de temperatura del producto entre -22 y -18 $^{\circ}$C. Para cada sección, indique qué módulo del curso aporta el contenido y qué número concreto mediría primero.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa. Los resultados numéricos de los casos de estudio se han obtenido y comprobado con el cuaderno `M28-practica.ipynb`.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. cola de una máquina; trabajos con tiempo de proceso p_j, fecha de entrega d_j y peso w_j.

Planteamiento. identificar los elementos de la definición 1.1 y la definición 1.3.

Desarrollo.

1. Esqueleto: un simulador de la máquina que, cada vez que queda libre, calcula la prioridad de cada trabajo en cola y procesa el de mayor prioridad; garantiza que cada trabajo se procesa una vez y sin solaparse.
2. Firma: `prioridad(p, d, w, t_actual) -> float`, vectorizada sobre los trabajos en cola.
3. Utilidad: $u(\pi, x) = -\sum_j w_j \max(0, C_j - d_j)$, el retraso ponderado cambiado de signo, con C_j el instante de fin; $-\infty$ si el programa falla.
4. Entrenamiento: varias semanas de llegadas históricas (o simuladas con su distribución); prueba: semanas posteriores y, aparte, semanas con más carga.
5. Semillas: la regla del menor tiempo de proceso ponderado, w_j/p_j, y la de la fecha de entrega más temprana, $-d_j$.

Resultado. **esqueleto = simulador de la cola; función que evoluciona = prioridad; u = menos el retraso ponderado; semillas w_j/p_j y $-d_j$.**

Comprobación. cualquier función numérica produce una secuencia factible, como exige la definición 1.3.

Solución del ejercicio 1.2

Datos. $s = (-416{,}7;\ -409{,}3;\ -405{,}3)$; $T = 0{,}1$ sin normalizar; $T = 0{,}3$ con normalización.

Planteamiento. definición 1.4; restar el máximo para evitar desbordamientos.

Desarrollo.

1. (a) $(s - \max s)/T = (-114;\ -40;\ 0)$; $e^{-114} \approx 3 \cdot 10^{-50}$ y $e^{-40} \approx 4 \cdot 10^{-18}$, luego $p \approx (0;\ 0;\ 1)$.
2. (b) Rango 11,4: $z = (-1;\ -0{,}3509;\ 0)$; $z/T = (-3{,}333;\ -1{,}170;\ 0)$; exponenciales $(0{,}0357;\ 0{,}3105;\ 1)$, suma 1,3462; $p = (0{,}0265;\ 0{,}2307;\ 0{,}7428)$.
3. (c) La temperatura solo tiene sentido relativa a la escala de las puntuaciones: con puntuaciones del orden de cientos, $T = 0{,}1$ hace el muestreo voraz; normalizando, la misma T significa lo mismo en cualquier problema.

Resultado. **(a)** $(0; 0; 1)$ **en la práctica; (b)** $(0{,}027;\ 0{,}231;\ 0{,}743)$**; (c) la escala debe fijarse antes de elegir** T**.**

Comprobación. $0{,}0265 + 0{,}2307 + 0{,}7428 = 1$; por la proposición 1.5, al bajar T la probabilidad se concentra en el mejor.

Solución del ejercicio 1.3

Datos. $C = 10$; objetos $3, 7, 4, 7, 8, 3, 5$; $S = 37$.

Planteamiento. aplicar la definición 1.9 y el lema 1.10.

Desarrollo.

1. $L_1 = \lceil 3{,}7 \rceil = 4$.
2. FF: 3 →1; 7 →1 (carga 10); 4 →2; 7 →3; 8 →4; 3 →2 (carga 7); 5: no cabe en 2 (resto 3), 3 (resto 3) ni 4 (resto 2) y abre el 5. Total 5.
3. BF: igual hasta el segundo 3, que cabe en 2 (hueco 3), 3 (hueco 0) y 4 (no cabe, resto 2): va al 3 (carga 10); el 5 cabe en 2 (resto 6, hueco 1) y va allí. Total 4.

Resultado. **FF usa 5 contenedores y BF 4; BF es óptimo porque alcanza** $L_1 = 4$**.**

Comprobación. cargas de BF: $\{3, 7\}$, $\{4, 5\}$, $\{7, 3\}$, $\{8\}$: suman $10 + 9 + 10 + 8 = 37$.

Solución del ejercicio 1.4

Datos. NF con capacidad C; ejemplo con $C = 1$ y objetos $1/2, \varepsilon, 1/2, \varepsilon, \ldots$ ($2k$ objetos).

Planteamiento. cotas por pares de contenedores consecutivos y lema 1.10.

Desarrollo.

1. (a) Si NF cerró el contenedor i para abrir el $i+1$, el primer objeto de $i+1$ no cabía en i: carga(i) + carga$(i+1) > C$.
2. Con m contenedores, los $\lfloor m/2 \rfloor$ pares disjuntos $(1,2), (3,4), \ldots$ suman más de C cada uno: $S > \lfloor m/2 \rfloor C$. Como $S \le \mathrm{OPT} \cdot C$, se tiene $\lfloor m/2 \rfloor < \mathrm{OPT}$ y, por ser enteros, $\lfloor m/2 \rfloor \le \mathrm{OPT} - 1$. Por tanto $m \le 2\lfloor m/2 \rfloor + 1 \le 2\,\mathrm{OPT} - 1$.
3. (b) NF: el contenedor con $1/2$ admite ε (carga $1/2 + \varepsilon$), pero el siguiente $1/2$ ya no cabe; cada par $(1/2, \varepsilon)$ ocupa un contenedor: k contenedores.
4. Óptimo: los k objetos de $1/2$ caben de dos en dos en $\lceil k/2 \rceil$ contenedores, y los k objetos ε (total $k\varepsilon < 1/2$) en uno más (o en el hueco libre si k es impar): $\mathrm{OPT} \le k/2 + 1$.
5. Cociente: $k/(k/2+1) \to 2$ cuando $k \to \infty$.

Resultado. **NF** $\leq 2\,\mathrm{OPT} - 1$**, y el factor 2 es asintóticamente alcanzable:** k **frente a** $k/2 + 1$**.**

Comprobación. con $k = 10$ y $\varepsilon = 0{,}01$: NF usa 10 contenedores; el óptimo, 6 (cinco con dos mitades y uno con los diez ε, carga 0,1); $10 \leq 2 \cdot 6$.

Solución del ejercicio 1.5

Datos. 2 mejoras en 1000 propuestas; 0,004 € por llamada; 20 s de CPU a 0,05 €/h; 10 islas de 300 evaluaciones.

Planteamiento. proposición 1.6 (ii) y (iii).

Desarrollo.

1. (a) $\hat{q} = 2/1000 = 0{,}002$.
2. (b) $(1-q)^B \leq 0{,}05 \iff B \geq \ln 0{,}05 / \ln 0{,}998 = 1496{,}4$: $B = 1497$. Con la cota $e^{-qB} \leq 0{,}05$ se obtiene $B \geq \ln 20/0{,}002 = 1497{,}9$.
3. (c) LLM: $1497 \cdot 0{,}004 = 5{,}99$ €; CPU: $1497 \cdot 20/3600 \cdot 0{,}05 = 0{,}42$ €; total 6,40 €.
4. (d) Cada isla falla con probabilidad $0{,}998^{300} = 0{,}5485$; las diez, $0{,}5485^{10} = 0{,}00246$; alguna mejora con probabilidad 0,9975.

Resultado. $q \approx 0{,}002$**;** $B = 1497$ **evaluaciones; unos** 6,40 **€;** 0,9975 **con 10 islas.**

Comprobación. (d) coincide con $1 - 0{,}998^{3000}$: con independencia, repartir las evaluaciones en islas no cambia la probabilidad de alguna mejora; las islas ayudan por la diversidad, que este modelo no recoge.

Solución del ejercicio 1.6

Datos. $E \to E + E \mid E \times E \mid x \mid 1$.

Planteamiento. contar árboles por la raíz.

Desarrollo.

1. (a) Un árbol de profundidad $\leq d$ es una hoja (2 opciones) o una raíz + o × (2 opciones) con dos subárboles de profundidad $\leq d-1$ (N_{d-1} opciones cada uno): $N_d = 2 + 2N_{d-1}^2$, con $N_0 = 2$.
2. (b) $N_1 = 2 + 2 \cdot 4 = 10$; $N_2 = 2 + 2 \cdot 100 = 202$; $N_3 = 2 + 2 \cdot 202^2 = 81\,610$.
3. (c) Si solo uno de los 81 610 árboles mejora la semilla, $q = 1/81\,610 = 1{,}23 \cdot 10^{-5}$ y $\mathbb{E}[N] \approx 81\,610$ evaluaciones.

Resultado. $N_1 = 10$**,** $N_2 = 202$**,** $N_3 = 81\,610$**;** $q \approx 1{,}2 \cdot 10^{-5}$**.**

Comprobación. el crecimiento es doblemente exponencial ($N_d \approx 2N_{d-1}^2$); por eso una distribución de propuestas informada (un LLM) importa tanto.

Solución del ejercicio 1.7

Datos. $\pi^* = -\min(g, 9) - 5{,}82\,\mathbf{1}[g \leq 20]$.

Planteamiento. estudiar la función por intervalos de g y aplicar la definición 1.9.

Desarrollo.

1. (a) $0 \le g \le 9$: $-g - 5{,}82 \in [-14{,}82; -5{,}82]$; $9 < g \le 20$: $-14{,}82$; $g > 20$: -9. Orden: $g \le 3$ (mayor que -9; se prefiere el menor g), después $g > 20$ (empate en -9), después $3 < g \le 9$ (de $-8{,}82$ a $-14{,}82$, decreciente; para $g \in (3{,}18;\, 9]$ por debajo de -9) y por último $9 < g \le 20$. Precisando: $-g - 5{,}82 > -9 \iff g < 3{,}18$.

2. (b) Si ningún contenedor deja $g \le 20$, todos puntúan -9 y el esqueleto elige el primero (desempate por índice): FF. Si alguno deja $g \le 3$, ese grupo tiene las mayores puntuaciones y, dentro de él, gana el de menor g: BF restringido a esos contenedores.

3. (c) El contenedor vacío tiene $g = C - s$. Si el objeto cabe en un contenedor abierto con $g \le 20$ o $g > 20$, su puntuación es al menos $-14{,}82$, y el vacío puntúa -9 si $C - s > 20$. Luego π^* *puede* abrir un contenedor nuevo con un abierto disponible si todos los abiertos dejan $3{,}18 < g \le 20$ y $s < 80$: *no* es *any fit* en general. Por tanto el teorema 1.11 no se aplica directamente; solo valdría si el esqueleto obligara a usar un abierto cuando el objeto cabe.

Resultado. **preferencia: $g < 3{,}18$, luego $g > 20$ (FF), luego $3{,}18 < g \le 9$, luego $9 < g \le 20$; coincide con FF o BF en los casos descritos; no es *any fit*, de modo que no hereda la garantía.**

Comprobación. con $s = 50$ y un único abierto con $r = 60$ ($g = 10$, puntuación $-14{,}82$), el vacío ($g = 50$) puntúa -9 y gana: se abre un contenedor teniendo hueco. La heurística paga ese contenedor para no dejar un resto de 10 unidades.

Solución del ejercicio 1.8

Datos. colocación de máquinas virtuales con CPU y memoria.

Planteamiento. recorrer los componentes de FunSearch y los errores de la observación 1.14.

Desarrollo.

1. Esqueleto: simulador de colocación en línea; para cada máquina virtual, calcula los servidores factibles (CPU y memoria libres suficientes, reglas de antiafinidad) y llama a `prioridad(cpu, mem, cpu_libre, mem_libre)`.

2. Evaluador: recursos varados medios (capacidad libre de un recurso en servidores con el otro agotado), número de servidores encendidos y peticiones rechazadas; puntuación escalar con pesos acordados; ejecución en contenedor sin red, con tiempo y memoria limitados; el programa no recibe las trazas completas, solo los argumentos de la firma.

3. Datos: trazas por semanas y clústeres; entrenamiento con unas semanas, validación con otras para elegir el programa y prueba con semanas y clústeres no usados, más escenarios de carga alta.

4. Presupuesto: piloto de 1000 propuestas para estimar q; $B \approx 3/q$ para un 95 % de probabilidad de mejora (proposición 1.6); coste estimado antes de lanzar.

5. Despliegue: simplificar y leer el programa; comparar con BF por recurso dominante con intervalos; despliegue en sombra y luego gradual con la regla actual como respaldo.

Resultado. **esqueleto factible por construcción, evaluador aislado y multiobjetivo, división por tiempo y clúster, presupuesto con $1/q$ y despliegue gradual.**

Comprobación. cada riesgo de la observación 1.14 tiene una medida asociada.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. clasificación de partes de avería en seis categorías.

Planteamiento. definición 2.2.

Desarrollo.

1. Cabecera: descripción de la tarea (clasificar un parte en una de seis categorías) y tres ejemplos de partes con su categoría.
2. Trayectoria: las 20 mejores instrucciones con su exactitud de validación, *de menor a mayor*, de modo que las mejores queden al final, junto a la petición.
3. Petición: «escribe una instrucción nueva, distinta de las anteriores, que obtenga mayor exactitud».
4. No debe contener: ejemplos del conjunto de prueba, la respuesta correcta de los ejemplos de validación que se usan para puntuar, ni datos personales de los partes.

Resultado. **tarea y ejemplos, trayectoria ordenada de peor a mejor, petición; sin datos de prueba ni datos personales.**

Comprobación. el orden ascendente sigue la descripción del método [51].

Solución del ejercicio 2.2

Datos. sistema de preguntas y respuestas con recuperación de fragmentos.

Planteamiento. definición 2.3: la métrica guía qué trazas se conservan.

Desarrollo.

1. Métrica propuesta:

   $$m(e, \hat{e}) = \mathbf{1}[\text{respuesta correcta}] \cdot \mathbf{1}[\text{algún fragmento recuperado contiene la respuesta}],$$

   con la corrección medida por coincidencia normalizada o por F_1 de palabras.
2. Con solo la respuesta, el optimizador conservaría como demostraciones trazas que aciertan por casualidad o por conocimiento previo del modelo, sin usar el contexto; el programa aprendería a ignorar la recuperación y fallaría en manuales que el modelo no conoce.

Resultado. **exigir a la vez respuesta correcta y contexto que la respalde.**

Comprobación. es la idea de la función de validación de contexto y respuesta de la tarea de CS224U [39].

Solución del ejercicio 2.3

Datos. $K = 50$; $n = 200$; $J = 0{,}80$.

Planteamiento. proposición 2.4 (ii) con $\sigma = \sqrt{J(1-J)/n}$.

Desarrollo.

1. (a) $\sigma = \sqrt{0{,}16/200} = 0{,}02828$; $\sqrt{2\ln 50} = 2{,}797$; cota $0{,}0791$.

2. (b) $\sqrt{0{,}16/n}\cdot 2{,}797 \le 0{,}02 \iff n \ge 0{,}16\cdot 2\ln 50/0{,}0004 = 3130$.
3. (c) Con $K=5$: $\sqrt{2\ln 5} = 1{,}794$; cota con $n=200$: 0,0507; para 2 puntos, $n \ge 0{,}16\cdot 2\ln 5/0{,}0004 = 1288$.

Resultado. **(a)** $\sigma = 0{,}0283$**, cota** $0{,}079$**; (b) 3130 ejemplos; (c) cota** $0{,}051$ **y 1288 ejemplos.**

Comprobación. la cota crece como $\sqrt{\ln K}$: pasar de 5 a 50 candidatos multiplica el tamaño necesario por $\ln 50/\ln 5 = 2{,}43$, y $3130/1288 = 2{,}43$.

Solución del ejercicio 2.4

Datos. $p = 0{,}7$; $V = 0{,}20$ €; $c = 0{,}004$ €.

Planteamiento. proposición 2.8 y corolario 2.9: seguir mientras $V\Delta_m \ge 2c = 0{,}008$.

Desarrollo.

1. $p(1-p) = 0{,}21$, $2p-1 = 0{,}4$. $\Delta_1 = \binom{1}{0}\cdot 0{,}21\cdot 0{,}4 = 0{,}0840$; $\Delta_3 = \binom{3}{1}\cdot 0{,}21^2\cdot 0{,}4 = 0{,}05292$; $\Delta_5 = \binom{5}{2}\cdot 0{,}21^3\cdot 0{,}4 = 0{,}03704$; $\Delta_7 = \binom{7}{3}\cdot 0{,}21^4\cdot 0{,}4 = 0{,}02723$.
2. $V\Delta_1 = 0{,}0168 \ge 0{,}008$; $V\Delta_3 = 0{,}0106 \ge 0{,}008$; $V\Delta_5 = 0{,}00741 < 0{,}008$: $m^* = 5$.
3. $P_5 = P_1 + \Delta_1 + \Delta_3 = 0{,}7 + 0{,}084 + 0{,}0529 = 0{,}8369$.

Resultado. $m^* = 5$**, con** $P_5 = 0{,}837$ **y beneficio** $0{,}2\cdot 0{,}8369 - 0{,}02 = 0{,}1474$ **€.**

Comprobación. beneficios con $m=3$ y $m=7$: $0{,}2\cdot 0{,}784 - 0{,}012 = 0{,}1448$ y $0{,}2\cdot 0{,}8740 - 0{,}028 = 0{,}1468$, ambos menores.

Solución del ejercicio 2.5

Datos. $f = 0{,}7$ (bucle), 0,2 (lectura), 0,1 (resto).

Planteamiento. proposición 2.10.

Desarrollo.

1. (a) $A = 1/(0{,}3 + 0{,}7/8) = 1/0{,}3875 = 2{,}58$.
2. (b) $1/(0{,}3 + 0{,}7/a) = 2 \iff 0{,}7/a = 0{,}2 \iff a = 3{,}5$.
3. (c) Con $a \to \infty$, $A \to 1/0{,}3 = 3{,}33 < 4$: imposible sin acelerar también la lectura o el resto.

Resultado. **(a)** $2{,}58$**; (b) factor** $3{,}5$**; (c) no: el máximo es** $3{,}33$**.**

Comprobación. con $a = 3{,}5$: $0{,}3 + 0{,}2 = 0{,}5$ y $A = 2$.

Solución del ejercicio 2.6

Datos. $t_A = (2; 4; 1; 8; 3)$, $t_B = (1{,}6; 4{,}4; 0{,}5; 8; 2{,}4)$; 14 victorias de B en 20 pares.

Planteamiento. medias aritmética y geométrica de cocientes; contraste de signos.

Desarrollo.

1. (a) $t_B/t_A = (0{,}8;\ 1{,}1;\ 0{,}5;\ 1;\ 0{,}8)$: media aritmética 0,840, geométrica $(0{,}8\cdot 1{,}1\cdot 0{,}5\cdot 1\cdot 0{,}8)^{1/5} = 0{,}352^{1/5} = 0{,}8115$.
2. (b) $t_A/t_B = (1{,}25;\ 0{,}909;\ 2;\ 1;\ 1{,}25)$: aritmética 1,282, geométrica 1,2322.

3. (c) Las aritméticas dicen «B tarda un 84 % de A» y «A tarda un 128 % de B», lo que implicaría $0{,}84 \cdot 1{,}28 = 1{,}08 \neq 1$: incoherente. Las geométricas cumplen $0{,}8115 \cdot 1{,}2322 = 1$, porque la media geométrica de los inversos es el inverso de la media geométrica.
4. (d) $\mathbb{P}(\mathrm{Bin}(20; 0{,}5) \geq 14) = 0{,}0577 > 0{,}05$: no se alcanza el criterio de 15 victorias.

Resultado. **media geométrica: B tarda un 81,2 % de A; con 14 de 20 victorias no se acepta.**

Comprobación. $0{,}8115^{-1} = 1{,}2323$.

Solución del ejercicio 2.7

Datos. ε_k independientes $\mathcal{N}(0, \sigma^2)$.

Planteamiento. monotonía del máximo; para $K = 2$, la identidad $\max(a, b) = \frac{a+b}{2} + \frac{|a-b|}{2}$.

Desarrollo.

1. $\max_{k \leq K+1} \varepsilon_k \geq \max_{k \leq K} \varepsilon_k$ para cada realización; tomando esperanzas, $\mathbb{E}[\max]$ es no decreciente en K.
2. $K = 2$: $\mathbb{E}[(\varepsilon_1 + \varepsilon_2)/2] = 0$ y $\varepsilon_1 - \varepsilon_2 \sim \mathcal{N}(0, 2\sigma^2)$; para $Z \sim \mathcal{N}(0, \tau^2)$, $\mathbb{E}|Z| = \tau\sqrt{2/\pi}$. Con $\tau = \sigma\sqrt{2}$: $\mathbb{E}|\varepsilon_1 - \varepsilon_2| = 2\sigma/\sqrt{\pi}$.
3. $\mathbb{E}[\max] = \frac{1}{2} \cdot \frac{2\sigma}{\sqrt{\pi}} = \sigma/\sqrt{\pi} = 0{,}5642\,\sigma$.
4. Cota: $\sigma\sqrt{2 \ln 2} = 1{,}1774\,\sigma$.

Resultado. $\mathbb{E}[\max(\varepsilon_1, \varepsilon_2)] = \sigma/\sqrt{\pi} \approx 0{,}564\,\sigma$**, frente a la cota** $1{,}177\,\sigma$**.**

Comprobación. el valor exacto es positivo y menor que la cota, como exige la proposición 2.4.

Solución del ejercicio 2.8

Datos. asistente que redacta órdenes de trabajo; optimización con DSPy.

Planteamiento. aplicar la definición 2.3, la proposición 2.4 y el corolario 2.6.

Desarrollo.

1. Datos: partes históricos con su orden de trabajo aprobada; entrenamiento (para demostraciones), validación (para elegir) y prueba (reservada), divididos por fecha.
2. Métrica: campos obligatorios correctos (equipo, tipo de intervención, repuestos) con comprobación automática, más una valoración de redacción por un LLM juez; el juez se valida contra 100 valoraciones de técnicos (acuerdo y sesgos) antes de usarlo.
3. Presupuesto: número máximo de llamadas por compilación y coste estimado; se detiene si dos rondas no mejoran la validación más allá de su error típico.
4. Aceptación: en el conjunto de prueba, intervalo de confianza de la diferencia pareada entre la versión nueva y la actual sobre los mismos partes; se acepta si el límite inferior es positivo y ningún campo obligatorio empeora.
5. Cambio de modelo del proveedor: fijar la versión del modelo; ante una actualización, repetir la evaluación de prueba y recompilar si empeora.

Resultado. **división temporal en tres partes, métrica verificable con juez validado, presupuesto con parada, aceptación por intervalo pareado y reevaluación ante cambios de modelo.**

Comprobación. el protocolo evita los errores (a)–(d) de la observación 2.12.

A.3 Tema 3

Solución del ejercicio 3.1

Datos. cuatro sistemas.

Planteamiento. las tres dimensiones de Elsken *et al.* [12].

Desarrollo.

1. (a) ENAS: espacio de celdas (o de redes recurrentes) como subgrafos de un supergrafo; estrategia: controlador entrenado con refuerzo; estimación: pesos compartidos.
2. (b) DARTS: espacio de celdas relajado de forma continua; estrategia: descenso de gradiente binivel; estimación: el propio supergrafo entrenado, con pesos compartidos.
3. (c) Real *et al.*: espacio de celdas; estrategia: evolución por torneo con envejecimiento; estimación: entrenamiento de cada candidata con presupuesto reducido.
4. (d) Caso: espacio CASH de tres familias; estrategia: muestreo aleatorio; estimación: multifidelidad por fracción de datos.

Resultado. **ver la lista.**

Comprobación. las tres dimensiones son independientes: cada sistema combina una opción de cada una.

Solución del ejercicio 3.2

Datos. 6 hiperparámetros, 2 importantes; 64 evaluaciones.

Planteamiento. observación 3.2.

Desarrollo.

1. (a) Rejilla $2^6 = 64$: cada hiperparámetro toma 2 valores; los dos importantes forman $2 \cdot 2 = 4$ combinaciones, repetidas 16 veces cada una.
2. (b) Aleatoria: con probabilidad 1, 64 valores distintos de cada uno y 64 combinaciones distintas de los dos importantes.
3. (c) $1 - 0{,}95^n \geq 0{,}95 \iff n \geq \ln 0{,}05 / \ln 0{,}95 = 58{,}4$: $n = 59$.

Resultado. **(a) 2 valores y 4 combinaciones; (b) 64 y 64; (c) 59 puntos.**

Comprobación. $1 - 0{,}95^{59} = 0{,}9515$ y $1 - 0{,}95^{58} = 0{,}9490$.

Solución del ejercicio 3.3

Datos. tres algoritmos con hiperparámetros condicionales.

Planteamiento. definición 3.1 con un hiperparámetro raíz.

Desarrollo.

1. Raíz: algoritmo $\in \{k\text{NN},\ \text{SVM},\ \text{bosque}\}$.
2. Rama kNN: $k \in \{1, \ldots, 50\}$, distancia $\in \{\text{euclídea},\ \text{Manhattan}\}$.

3. Rama SVM: núcleo $\in$ {lineal, gaussiano}, $C > 0$ (escala logarítmica); si núcleo = gaussiano, $\gamma > 0$ (escala logarítmica).
4. Rama bosque: número de árboles y profundidad máxima.
5. Problema: $\min_{j,\lambda\in\Lambda^{(j)}}$ de la pérdida media de validación cruzada.

Resultado. **árbol de profundidad 3 (algoritmo, núcleo, γ). Activos: k y distancia; núcleo y C, más γ si el núcleo es gaussiano; número de árboles y profundidad.**

Comprobación. ningún hiperparámetro está activo en dos ramas a la vez.

Solución del ejercicio 3.4

Datos. $r_{\min} = 2$, $\eta = 2$, $r_{\max} = 10$; escalones $2, 4, 8, 10$; $N = 16$.

Planteamiento. definición 3.6 con el último escalón recortado.

Desarrollo.

1. (a) 16 con 2 épocas, 8 con 4, 4 con 8 y 2 con 10.
2. (b) $16 \cdot 2 + 8 \cdot 4 + 4 \cdot 8 + 2 \cdot 10 = 32 + 32 + 32 + 20 = 116$ épocas.
3. (c) $16 \cdot 10 = 160$ épocas: ahorro del 27,5 %.
4. (d) $r_{\max}/r_{\min} = 5$ no es potencia de η: el último escalón no multiplica por η y no reduce a una sola configuración, y con pocos escalones el cociente $(K+1)/\eta^K$ es poco favorable.

Resultado. **116 épocas frente a 160.**

Comprobación. los tres primeros escalones cuestan $Nr_{\min} = 32$ cada uno, como en la proposición 3.7.

Solución del ejercicio 3.5

Datos. $B = 4$, $|\mathcal{O}| = 5$.

Planteamiento. fórmula $\prod_{b=1}^{B}((b+1)|\mathcal{O}|)^2$.

Desarrollo.

1. (a) $(10 \cdot 15 \cdot 20 \cdot 25)^2 = 75\,000^2 = 5{,}625 \cdot 10^9$.
2. (b) Dos celdas independientes: $(5{,}625 \cdot 10^9)^2 = 3{,}16 \cdot 10^{19}$.
3. (c) $3{,}16 \cdot 10^{19}$ h, unos $3{,}6 \cdot 10^{15}$ años de GPU.

Resultado. $5{,}6 \cdot 10^9$ **celdas;** $3{,}2 \cdot 10^{19}$ **redes; enumeración imposible.**

Comprobación. para el nodo $b = 1$ hay $2 \cdot 5 = 10$ opciones por entrada, y $10^2 = 100$ por nodo.

Solución del ejercicio 3.6

Datos. $\alpha = (1{,}0;\ 0{,}2;\ -0{,}5;\ 0{,}8)$; $x = 1{,}5$; operaciones x, x^2, 0, $2x$.

Planteamiento. definición 3.4; derivada de la softmax $\partial w_o/\partial\alpha_1 = w_o(\mathbf{1}[o = 1] - w_1)$.

Desarrollo.

1. (a) $e^{\alpha} = (2{,}718;\ 1{,}221;\ 0{,}6065;\ 2{,}226)$, suma 6,772; $w = (0{,}4014;\ 0{,}1804;\ 0{,}0896;\ 0{,}3287)$. $\bar{o}(1{,}5) = 0{,}4014 \cdot 1{,}5 + 0{,}1804 \cdot 2{,}25 + 0 + 0{,}3287 \cdot 3 = 0{,}6021 + 0{,}4058 + 0{,}9860 = 1{,}9939$.
2. (b) Mayor α: identidad.

3. (c) $\partial\bar{o}/\partial\alpha_1 = \sum_o o(x)w_o(\mathbf{1}[o=1]-w_1) = w_1\big(o_1(x)-\bar{o}(x)\big) = 0{,}4014\,(1{,}5-1{,}9939) = -0{,}1983$.

Resultado. $w = (0{,}401;\ 0{,}180;\ 0{,}090;\ 0{,}329)$**,** $\bar{o}(1{,}5) = 1{,}994$**; se conserva la identidad; derivada** $-0{,}198$**.**

Comprobación. los pesos suman 1; la derivada es negativa porque la identidad da menos que la mezcla en $x = 1{,}5$.

Solución del ejercicio 3.7

Datos. 12 min proporcionales más 1 min fijo por entrenamiento completo; 0,60 €/h.

Planteamiento. sumar tiempos y aplicar la proposición 3.7 con coste fijo.

Desarrollo.

1. (a) $4^3 = 64$ entrenamientos de 13 min: 832 min = 13,87 h, 8,32 €.
2. (b) 27 de 13 min: 351 min = 5,85 h, 3,51 €.
3. (c) $27(1+12/9)+9(1+12/3)+3(1+12) = 63+45+39 = 147$ min = 2,45 h, 1,47 €.
4. (d) $147/351 = 0{,}419$: el coste fijo se paga $27+9+3 = 39$ veces en el halving y 27 en la búsqueda aleatoria.

Resultado. 8,32**;** 3,51 **y** 1,47 **€; cociente** 0,42 **en lugar de** 1/3**.**

Comprobación. sin coste fijo: $27{\cdot}12/9+9{\cdot}12/3+3{\cdot}12 = 36+36+36 = 108$ min y $108/324 = 1/3$.

Solución del ejercicio 3.8

Datos. $f_1(r) = 0{,}30+0{,}20e^{-r/2}$, $f_2(r) = 0{,}25+0{,}40e^{-r/8}$.

Planteamiento. resolver $f_1 = f_2$; aplicar la proposición 3.8.

Desarrollo.

1. (a) $g(r) = f_1-f_2 = 0{,}05+0{,}2e^{-r/2}-0{,}4e^{-r/8}$; $g(16) = 0{,}0500+0{,}0001-0{,}0541 = -0{,}0041$ y $g(17) = 0{,}0500+0{,}0000-0{,}0477 = +0{,}0023$; por bisección, $r^* \approx 16{,}6$ épocas.
2. (b) Escalón 0 ($r = 16$): $f_1 = 0{,}3001 < f_2 = 0{,}3041$; se conserva λ_1, que con $r = 32$ vale 0,3000, mientras que $f_2(32) = 0{,}25+0{,}4e^{-4} = 0{,}2573$. Se devuelve la peor.
3. (c) Usar $r_{\min} \geq 17$, o ejecutar como Hyperband una ronda sin eliminación ($r_{\min} = r_{\max}$), que evalúa ambas a 32 épocas: coste $2\cdot 32 = 64$ épocas frente a $2\cdot 16+32 = 64$ del halving; aquí no cuesta más. En general, Hyperband multiplica el coste por el número de rondas, a cambio de no depender de una sola $r_{\min}$.

Resultado. $r^* \approx 16{,}6$**; el halving con** $r_{\min} = 16$ **elige** λ_1 **(pérdida final** 0,300 **frente a** 0,257**).**

Comprobación. g es creciente para $r > 4$ (su derivada $-0{,}1e^{-r/2}+0{,}05e^{-r/8}$ es positiva cuando $e^{3r/8} > 2$), luego el cruce es único.

A.4 Tema 4

Solución del ejercicio 4.1

Datos. cuatro situaciones.

Planteamiento. definición 4.4.

Desarrollo.

1. (a) Cambia $P(X)$ (vibraciones), no $P(Y \mid X)$: cambio de covariables.
2. (b) Con la misma X cambia la probabilidad de avería: deriva de concepto.
3. (c) Sube $P(Y)$ con $P(X \mid Y)$ igual: cambio de etiquetas.
4. (d) Cambia la definición de la característica: cambio de la característica (sesgo del sistema, definición 4.1), que debe corregirse en la ingesta, no reentrenando.

Resultado. **(a) covariables; (b) concepto; (c) etiquetas; (d) cambio de unidad de una característica.**

Comprobación. solo en (b) un modelo perfecto con los datos antiguos dejaría de ser correcto aunque se le dieran entradas bien medidas.

Solución del ejercicio 4.2

Datos. cuatro situaciones.

Planteamiento. definición 4.2.

Desarrollo.

1. (a) Consumidores no declarados.
2. (b) Entrelazamiento (CACE).
3. (c) Deuda de configuración.
4. (d) Cambios en el mundo exterior: el umbral óptimo c_p/c_f depende de costes que cambian.

Resultado. **(a) consumidores no declarados; (b) CACE; (c) configuración; (d) mundo exterior.**

Comprobación. cada caso tiene un remedio distinto: registro de consumidores, pruebas de regresión, revisión de configuración, umbrales calculados a partir de costes actualizados.

Solución del ejercicio 4.3

Datos. $T \sim U(0, 1500)$ días.

Planteamiento. definición 4.10.

Desarrollo.

1. (a) $F(t) = t/1500$, $R(t) = 1 - t/1500$, $f = 1/1500$, $\lambda(t) = 1/(1500 - t)$ para $0 \leq t < 1500$.
2. (b) $\lambda(0) = 6{,}67 \cdot 10^{-4}$, $\lambda(500) = 1{,}00 \cdot 10^{-3}$ y $\lambda(1000) = 2{,}00 \cdot 10^{-3}$ por día.
3. (c) No: la tasa de fallo crece (vida IFR); a los 500 días es un 50 % mayor que nueva. Además, la vida restante es uniforme en $(0, 1000)$, de media 500 días, frente a 750 de una nueva.

4. (d) Solo para la exponencial, de tasa constante y sin memoria.

Resultado. $\lambda(t) = 1/(1500 - t)$**, creciente: la cinta usada no es tan buena como nueva.**
Comprobación. $R(t) = \exp(-\int_0^t \frac{du}{1500-u}) = \exp(\ln \frac{1500-t}{1500}) = 1 - t/1500$.

Solución del ejercicio 4.4

Datos. $p = (0{,}10; 0{,}40; 0{,}40; 0{,}10)$, $q = (0{,}05; 0{,}30; 0{,}45; 0{,}20)$.

Planteamiento. definición 4.5 y proposición 4.6.

Desarrollo.

1. (a) Términos $(q_b - p_b)\ln(q_b/p_b)$: 0,0347; 0,0288; 0,0059; 0,0693. PSI = 0,1386.
2. (b) $\mathrm{KL}(p\|q) = 0{,}1\ln 2 + 0{,}4\ln\frac{4}{3} + 0{,}4\ln\frac{8}{9} + 0{,}1\ln\frac{1}{2} = 0{,}0680$; $\mathrm{KL}(q\|p) = 0{,}0707$; suma 0,1386.
3. (c) Acumuladas: $P = (0{,}10; 0{,}50; 0{,}90; 1)$, $Q = (0{,}05; 0{,}35; 0{,}80; 1)$; diferencias 0,05; 0,15; 0,10; 0: estadístico 0,15.

Resultado. PSI = 0,139**;** $\mathrm{KL}(p\|q) = 0{,}068$ **y** $\mathrm{KL}(q\|p) = 0{,}071$**;** **KS** = 0,15**.**
Comprobación. $0{,}0680 + 0{,}0707 = 0{,}1387 \approx 0{,}1386$ (redondeo).

Solución del ejercicio 4.5

Datos. exponencial de media 200 d; $c_p = 500$, $c_f = 3000$; Weibull $\beta = 2$, $\theta = 100$.

Planteamiento. proposición 4.13.

Desarrollo.

1. (a) $\int_0^T e^{-u/200}du = 200(1 - e^{-T/200})$. $T = 50$: $R = 0{,}7788$, $C = (500 \cdot 0{,}7788 + 3000 \cdot 0{,}2212)/44{,}24 = 23{,}80$ €/d. $T = 100$: $R = 0{,}6065$, $C = (303{,}3 + 1180{,}6)/78{,}69 = 18{,}85$. $T = \infty$: $3000/200 = 15{,}00$.
2. (b) Con vida exponencial la máquina usada es como nueva; intervenir antes solo añade costes y acorta los ciclos sin reducir la tasa de fallo: $T^* = \infty$, como predice la proposición para $\beta = 1$.
3. (c) $c_p/(c_f - c_p) = 0{,}2$; $\lambda(T) = 2T/10^4$. $G(40) = 0{,}008 \cdot 37{,}97 - (1 - e^{-0{,}16}) = 0{,}3037 - 0{,}1479 = 0{,}156$; $G(50) = 0{,}01 \cdot 46{,}13 - (1 - e^{-0{,}25}) = 0{,}4613 - 0{,}2212 = 0{,}240$. Como G es creciente y pasa de 0,156 a 0,240, $T^* \in (40, 50)$.

Resultado. **(a)** 23,80**;** 18,85**;** 15,00 **€/día: no conviene el preventivo; (c)** $40 < T^* < 50$ **días (numéricamente** $T^* = 45{,}5$ **y** $C(T^*) = 2500\lambda(T^*) = 22{,}74$ **€/día).**
Comprobación. C decrece en (a) (23,80 > 18,85 > 15,00), como exige el caso DFR (aquí, tasa constante).

Solución del ejercicio 4.6

Datos. $\pi_0(\text{interv.} \mid \text{alarma}) = 0{,}3$; $\pi_0(\text{interv.} \mid \text{sin}) = 0{,}1$; seis registros.

Planteamiento. proposición 4.9 con π determinista.

Desarrollo.

1. Pesos π/π_0: registro 1 (alarma, interviene): $1/0{,}3 = 3{,}333$; 2 y 3 (alarma, no): 0; 4 y 6 (sin alarma, no): $1/0{,}9 = 1{,}111$; 5 (sin alarma, interviene): 0.

2. Suma de pesos por recompensa: $3{,}333 \cdot (-1{,}2) + 1{,}111 \cdot 0 + 1{,}111 \cdot 0 = -4{,}0$.
3. $\hat{V}_{\text{IPS}}(\pi) = -4{,}0/6 = -0{,}667$ miles de euros por semana.
4. Política antigua (media de los registros): $(-1{,}2 - 9 + 0 + 0 - 1{,}2 + 0)/6 = -1{,}9$.

Resultado. $\hat{V}(\pi) \approx -0{,}67$ **frente a** $-1{,}9$ **(miles de euros); la estimación favorece la política nueva, pero descansa en un único registro con alarma e intervención, multiplicado por** $3{,}33$**: es muy poco fiable.**

Comprobación. los pesos de los registros coherentes con π suman $3{,}333 + 1{,}111 + 1{,}111 = 5{,}56$, cerca de $n = 6$, como se espera de media.

Solución del ejercicio 4.7

Datos. $c_r = 3000$ €; $a = 4$ €/día^2.

Planteamiento. proposición 4.8.

Desarrollo.

1. (a) $T^* = \sqrt{6000/4} = 38{,}73$ días; coste $\sqrt{2 \cdot 4 \cdot 3000} = 154{,}9$ €/día.
2. (b) $T = 7$: $3000/7 + 4 \cdot 7/2 = 428{,}6 + 14 = 442{,}6$ €/día.
3. (c) $T = 180$: $16{,}7 + 360 = 376{,}7$ €/día.

Resultado. $T^* \approx 39$ **días con** $154{,}9$ **€/día; semanal** $442{,}6$ **y semestral** $376{,}7$ **€/día.**

Comprobación. en T^* los dos términos coinciden: $3000/38{,}73 = 77{,}46 = 4 \cdot 38{,}73/2$.

Solución del ejercicio 4.8

Datos. tasa de positivos $\pi = 0{,}125$.

Planteamiento. calcular precisión, sensibilidad, F_1 y exactitud de predictores que ignoran X.

Desarrollo.

1. (a) Predice positivo con probabilidad 0,125: precisión $= \pi = 0{,}125$, sensibilidad $= 0{,}125$, $F_1 = 0{,}125$; exactitud $\pi^2 + (1-\pi)^2 = 0{,}0156 + 0{,}7656 = 0{,}781$.
2. (b) Con probabilidad 1/2: precisión 0,125, sensibilidad 0,5, $F_1 = 2 \cdot 0{,}125 \cdot 0{,}5/0{,}625 = 0{,}2$; exactitud 0,5.
3. (c) Siempre negativo: exactitud 0,875; sensibilidad 0 y $F_1 = 0$.

Resultado. **(a) exactitud** $0{,}781$**,** $F_1 = 0{,}125$**; (b)** $0{,}5$ **y** $0{,}2$**; (c)** $0{,}875$ **y** 0**.**

Comprobación. la mejor exactitud la da un predictor inútil; por eso se informa de F_1, del área ROC, de la pérdida logarítmica y de la calibración.

Solución del ejercicio 4.9

Datos. optimizador del ejemplo 4.14; etiquetas con 7 días de retraso.

Planteamiento. combinar la definición 4.4, la proposición 4.6, la proposición 4.8 y la observación 4.3.

Desarrollo.

1. Diario: esquema y ausencias de los sensores; detección de sensores congelados (varianza nula en 24 h); distribución de las probabilidades predichas; latencia y errores del servicio. Acción: alarma al responsable de datos y paso a la política por edad para la máquina afectada.
2. Semanal: PSI por variable y por máquina frente a una referencia de 8 semanas estables, con umbral calibrado como el percentil 99 del PSI entre semanas estables; tasa de anulaciones humanas y sus motivos; plan frente a ejecución.
3. Mensual (con etiquetas): pérdida logarítmica, calibración por quintiles y área ROC de las predicciones de hace al menos 7 días; averías y coste frente al gemelo; decisión de reentrenar por periodo (T^*) o por disparador.
4. Bucle: cada decisión se registra con p_7, la acción y quién la aprobó; las intervenciones se tratan como censura; el reentrenamiento usa solo datos de máquinas sin intervención en la ventana de la etiqueta, o modelos de supervivencia con censura.

Resultado. **plan en tres frecuencias con estadísticos, umbrales calibrados, acciones definidas y tratamiento de la censura.**

Comprobación. cubre entradas, predicciones, decisiones, resultados y salud del servicio, como pide la clase 10 de CS329S [46].

A.5 Tema 5

Solución del ejercicio 5.1

Datos. ahorro declarado del 18 % frente a las rutas de los conductores.

Planteamiento. la alternativa honrada es la mejor solución sin IA (observación 5.9).

Desarrollo.

1. Alternativas: un resolvedor clásico de rutas con ventanas horarias sin predicción (módulo 24, tema 3); la misma optimización con tiempos medios históricos por franja; reglas operativas mejoradas (agrupar por zonas, secuencias fijas).
2. Parte no atribuible a la IA: la que se obtiene solo por optimizar las rutas con un resolvedor (pasar de rutas manuales a rutas optimizadas) y la debida a cambios simultáneos (menos pedidos, otra flota).

Resultado. **comparar con un resolvedor sin predicción y con reglas mejoradas; la IA solo aporta la diferencia con la mejor de ellas.**

Comprobación. en el caso de mantenimiento, la IA aporta 34 370 €/año de los 170 990 de mejora total.

Solución del ejercicio 5.2

Datos. $\sigma_B = 6500$, $\sigma_D = 5200$; $r = 0{,}78$.

Planteamiento. proposición 5.3.

Desarrollo.

1. (a) Independientes: $\sqrt{6500^2 + 5200^2} = \sqrt{6{,}929 \cdot 10^7} = 8324$ €. Pareadas:

$$\sqrt{6{,}929 \cdot 10^7 - 2 \cdot 0{,}78 \cdot 6500 \cdot 5200} = \sqrt{1{,}656 \cdot 10^7} = 4070 \text{ €}.$$

2. (b) Para la misma precisión de la media, n es proporcional a la varianza: $6{,}929/1{,}656 = 4{,}18$ veces más semanas.

Resultado. **8324 € frente a 4070 €; harían falta unas 4,2 veces más semanas sin emparejar.**

Comprobación. $8324^2/4070^2 = 4{,}18$.

Solución del ejercicio 5.3

Datos. $n = 10$, $\bar{d} = 40{,}3$; segunda y decimoctava medias *bootstrap*: 39,4 y 42,4; $s = 4{,}67$; $t_{9;0{,}90} = 1{,}383$.

Planteamiento. definición 5.4 con $\alpha = 0{,}2$ y 20 remuestras (cuantiles 0,1 y 0,9 = valores 2.º y 18.º).

Desarrollo.

1. (a) Percentil: $[39{,}4;\ 42{,}4]$.
2. (b) $\delta^*_{(0,1)} = 39{,}4 - 40{,}3 = -0{,}9$ y $\delta^*_{(0,9)} = 42{,}4 - 40{,}3 = 2{,}1$; básico: $[40{,}3 - 2{,}1;\ 40{,}3 + 0{,}9] = [38{,}2;\ 41{,}2]$.
3. (c) $40{,}3 \pm 1{,}383 \cdot 4{,}67/\sqrt{10} = 40{,}3 \pm 2{,}04$: $[38{,}26;\ 42{,}34]$.
4. (d) Con 20 remuestras los cuantiles dependen de dos valores y varían mucho entre ejecuciones; con 10 000, el error de Monte Carlo del cuantil es despreciable.

Resultado. **percentil** $[39{,}4; 42{,}4]$**; básico** $[38{,}2; 41{,}2]$**;** t**:** $[38{,}3; 42{,}3]$ **(en cientos de euros por semana).**

Comprobación. el intervalo básico es el percentil reflejado respecto a $\bar{d}$: $40{,}3 \pm (42{,}4 - 40{,}3)$ y $40{,}3 \pm (40{,}3 - 39{,}4)$ intercambiados.

Solución del ejercicio 5.4

Datos. $I = 48\,000$ €; flujo neto 25 370 €/año; $N = 5$; $\rho = 8\,\%$.

Planteamiento. proposición 5.2.

Desarrollo.

1. (a) $(1 - 1{,}08^{-5})/0{,}08 = (1 - 0{,}68058)/0{,}08 = 3{,}9927$.
2. (b) $a_{5;0{,}12} = (1 - 1{,}12^{-5})/0{,}12 = 3{,}6048$; VAN $= 25\,370 \cdot 3{,}6048 - 48\,000 = 43\,453$ €.
3. (c) Flujo neto mínimo $48\,000/3{,}9927 = 12\,022$ €, es decir, beneficio bruto de $12\,022 + 9000 = 21\,022$ €/año.
4. (d) $a_{3;0{,}08} = 2{,}5771$; VAN $= 25\,370 \cdot 2{,}5771 - 48\,000 = 17\,381$ €.

Resultado. **(a)** 3,9927**; (b)** 43 453 **€; (c)** 21 022 **€/año de beneficio; (d)** 17 381 **€.**

Comprobación. con $\rho = 8\,\%$ y $N = 5$: $25\,370 \cdot 3{,}9927 - 48\,000 = 53\,295$, el valor del ejemplo 5.6.

Solución del ejercicio 5.5

Datos. $\sigma_D = 4356$, $\delta = 661$; parada no planificada: $\sigma_D = 3{,}1$ h, $\delta = 0{,}9$ h.

Planteamiento. proposición 5.5.

Desarrollo.

1. (a) $z_{0,975} + z_{0,9} = 1{,}960 + 1{,}282 = 3{,}242$; $n = (3{,}242 \cdot 4356/661)^2 = 21{,}36^2 = 456{,}3$: 457 semanas.
2. (b) Con potencia 0,8 hacen falta 341 pares; con 6 líneas en paralelo, $341/6 = 56{,}8$: 57 semanas.
3. (c) $(2{,}802 \cdot 3{,}1/0{,}9)^2 = 9{,}65^2 = 93{,}1$: 94 semanas.

Resultado. **(a) 457; (b) 57; (c) 94 semanas.**

Comprobación. la relación señal-ruido δ/σ_D pasa de 0,15 (coste) a 0,29 (horas de parada), y el tamaño se divide por $(0{,}29/0{,}15)^2 \approx 3{,}7$.

Solución del ejercicio 5.6

Datos. costes ordenados $139, 148, 149, 152, 157, 160, 166, 171, 175, 183$ (miles de euros).

Planteamiento. VaR y CVaR empíricos; proposición 5.7 con la distribución empírica.

Desarrollo.

1. (a) Media $1600/10 = 160$. $\mathrm{VaR}_{0,8}$: el menor valor con frecuencia acumulada $\geq 0{,}8$, el octavo, 171. $\mathrm{CVaR}_{0,8}$: media de los dos peores, $(175 + 183)/2 = 179$.
2. (b) $\phi(\tau) = \tau + \frac{1}{0,2} \cdot \frac{1}{10} \sum_s (c_s - \tau)^+$. $\phi(175) = 175 + 0{,}5 \cdot 8 = 179$. $\phi(160) = 160 + 0{,}5 \cdot (6 + 11 + 15 + 23) = 187{,}5$. $\phi(183) = 183 + 0 = 183$.

Resultado. **media 160; VaR 171; CVaR 179;** $\phi(175) = 179 < \phi(183) = 183 < \phi(160) = 187{,}5$**.**

Comprobación. con distribución discreta, ϕ es constante e igual a 179 en todo $[171, 175]$; el mínimo de la proposición 5.7 no es único, pero su valor sí.

Solución del ejercicio 5.7

Datos. cinco modos del cuadro 5.1 y dos nuevos.

Planteamiento. definición 5.8.

Desarrollo.

1. NPR nuevos: ciberataque $9 \cdot 2 \cdot 8 = 144$; datos de vacaciones $5 \cdot 3 \cdot 5 = 75$.
2. Orden: sensor congelado (168) y confianza excesiva (168); etiquetas mal registradas (144) y ciberataque (144, con gravedad 9); deriva (120); vacaciones (75); resolvedor (20).
3. Medidas para los primeros: sensor congelado, regla de varianza nula y redundancia del sensor de vibración en la llenadora; confianza excesiva, mostrar motivos y probabilidades, auditoría mensual de órdenes aceptadas sin cambios y formación sobre el sesgo de automatización.

Resultado. **NPR 144 y 75; prioridad: sensor congelado y confianza excesiva; el ciberataque se trata en todo caso por su gravedad.**

Comprobación. la gravedad 9 del ciberataque obliga a tratarlo aunque su NPR empate con otros (definición 5.8).

Solución del ejercicio 5.8

Datos. tres sistemas.

Planteamiento. artículo 6 y anexo III del Reglamento [42], en términos generales; no es una calificación jurídica.

Desarrollo.

1. (a) Optimizador de mantenimiento de una línea de envasado: no gestiona una infraestructura crítica ni actúa sobre funciones de seguridad de las máquinas; un técnico revisa sus órdenes. No parece de alto riesgo; faltaría confirmar que no se integra en un producto regulado como componente de seguridad. Medidas recomendables: registro, supervisión humana, documentación y monitorización.
2. (b) Válvulas de una red de calefacción urbana para evitar sobrepresiones: puede ser un componente de seguridad en la operación del suministro de calefacción (anexo III, punto 2), por tanto candidato a alto riesgo. Faltaría saber si es la función de seguridad o una capa de optimización sobre un sistema de seguridad independiente (módulo 26, tema 6). Medidas: todas las de los artículos 9 a 15, con especial atención a la robustez y la supervisión del artículo 14.
3. (c) Asistente que redacta borradores revisados: tarea preparatoria sin influencia sustancial en decisiones de alto riesgo; en general no es de alto riesgo. Medidas: evaluación de calidad, registro, revisión humana obligatoria antes de emitir la orden.

Resultado. **(a) y (c) previsiblemente no; (b) posible alto riesgo según su función; en todos, adoptar registro, supervisión y documentación.**

Comprobación. el criterio decisivo es la función de seguridad y el ámbito del anexo III, no la técnica de IA empleada.

A.6 Tema 6

Solución del ejercicio 6.1

Datos. propuesta «modelo primero».

Planteamiento. comparar con los doce pasos.

Desarrollo.

1. Faltan: la decisión y sus responsables (1), el objetivo y las restricciones (2), la línea base y el techo (3), el inventario de datos (4) y la formulación (5); la medida del ahorro se deja al final sin diseño (9).
2. Orden correcto de las cinco primeras tareas: definir la decisión y quién la toma; fijar objetivo, indicadores y restricciones; medir la línea base, la mejor alternativa sencilla y el techo; inventariar los datos y su disponibilidad; formular el problema y elegir la familia de métodos. Solo entonces se elige el modelo predictivo (que puede no ser un transformer).

Resultado. **la propuesta salta los pasos 1–5 y deja la evaluación sin diseñar.**

Comprobación. la arquitectura (nube) y el modelo (transformer) son decisiones de los pasos 6 y 10.

Solución del ejercicio 6.2

Datos. 2 s; margen del 25 %; percentiles 99: 0,30; 0,15; 0,05; 0,70; 0,20 s.

Planteamiento. asignación de presupuesto con margen de sistema.

Desarrollo.

1. (a) Presupuesto asignable: $0{,}75 \cdot 2 = 1{,}5$ s. Se reparte en proporción a la variabilidad esperada, dando más al resolvedor (tiempo variable) y a la lectura de datos (depende de la red): por ejemplo 0,35; 0,15; 0,10; 0,65; 0,25 s.
2. (b) Suma de percentiles 99: 1,40 s $\leq$ 1,5 s: se cumple con margen (0,60 s, 30 %). La suma de percentiles es conservadora (el percentil 99 de la suma suele ser menor). Revisar el resolvedor: es el mayor y el más variable; fijar un límite de tiempo y arrancar desde el plan anterior.

Resultado. **1,5 s asignables; las mediciones suman 1,40 s y cumplen; vigilar el resolvedor.**

Comprobación. $0{,}35 + 0{,}15 + 0{,}10 + 0{,}65 + 0{,}25 = 1{,}50$.

Solución del ejercicio 6.3

Datos. pesos $(0{,}4; 0{,}2; 0{,}2; 0{,}2)$; I $(2, 5, 5, 4)$, II $(4, 4, 3, 4)$, III $(5, 2, 1, 2)$.

Planteamiento. suma ponderada y dominancia de Pareto (módulo 24, tema 5).

Desarrollo.

1. (a) I: $0{,}8+1+1+0{,}8 = 3{,}6$; II: $1{,}6+0{,}8+0{,}6+0{,}8 = 3{,}8$; III: $2{,}0+0{,}4+0{,}2+0{,}4 = 3{,}0$.
2. (b) Ninguno domina a otro: III es el mejor en coste; I, en interpretabilidad y esfuerzo; II es mejor que I en coste y que III en todo lo demás. Los tres son de Pareto.
3. (c) Con peso w para el coste y $(1-w)/3$ para cada uno de los otros: $\text{II} = 4w + \frac{11}{3}(1-w)$, $\text{III} = 5w + \frac{5}{3}(1-w)$. Igualando: $4w + \frac{11}{3} - \frac{11}{3}w = 5w + \frac{5}{3} - \frac{5}{3}w$, es decir, $-3w = -2$ y $w = 2/3$.

Resultado. **II gana con** $3{,}8$**; los tres son de Pareto; III empataría con II si el coste pesara** $2/3$**.**

Comprobación. con $w = 2/3$: $\text{II} = 8/3 + 11/9 = 35/9$ y $\text{III} = 10/3 + 5/9 = 35/9$.

Solución del ejercicio 6.4

Datos. 330 290; 193 670; 159 300; 76 170 €/año; sensor: 30 000 € y 3000 €/año; 5 años al 8 %.

Planteamiento. fracciones del margen; proposición 5.2.

Desarrollo.

1. (a) Margen total $330\,290 - 76\,170 = 254\,120$. Política por edad: $136\,620/254\,120 = 53{,}8\,\%$. Optimizador: $170\,990/254\,120 = 67{,}3\,\%$.
2. (b) Margen restante $159\,300 - 76\,170 = 83\,130$. VAN $\geq 0 \iff (b - 3000) \cdot 3{,}9927 \geq 30\,000 \iff b \geq 10\,514$ €/año, un 12,6 % del margen restante.

Resultado. **53,8 % y 67,3 %; el sensor debe capturar al menos un 12,6 % del margen restante.**

Comprobación. $(10\,514 - 3000) \cdot 3{,}9927 = 30\,001$.

Solución del ejercicio 6.5

Datos. matriz de cambios del ejemplo 6.2.

Planteamiento. enumerar los seis ciclos.

Desarrollo.

1. (a) Prohibir D→C elimina los ciclos A–B–D–C (215) y A–D–C–B (210). El óptimo A–B–C–D (D→A) sigue permitido: no cambia.
2. (b) Cada ciclo contiene exactamente un cambio desde D; con 60 min se restan 30 a todos: 145, 160, 180, 185, 195 y 205 para A–B–C–D, A–C–D–B, A–D–C–B, A–B–D–C, A–D–B–C y A–C–B–D. El orden no cambia.
3. (c) Con 8 formatos y el primero fijo, $7! = 5040$ ciclos.

Resultado. **(a) no cambia (175 min); (b) 145 min con el mismo orden; (c) 5040 ciclos.**

Comprobación. $175 - 30 = 145$.

Solución del ejercicio 6.6

Datos. tres objetos con tres LOD; presupuesto 100 (miles de triángulos).

Planteamiento. enumeración de las 27 combinaciones; mochila de elección múltiple.

Desarrollo.

1. (a) Factibles con menor error sin regla: (LOD1, LOD1, LOD1) con 70 y error 2,4; las de error menor, como (0,1,1) con error 1,4 y (1,1,0) con 1,6, usan 105 y no caben. Con el personaje en LOD0: (0,2,1) con 87 y error 3,3; (0,1,2) con 95 y 3,6; (0,2,2) con 77 y 5,5.
2. (b) La regla no cuesta nada cuando el óptimo sin regla ya tiene el personaje en LOD0: con presupuesto 105 el óptimo es (0,1,1), error 1,4. Por debajo de 105, el óptimo sin regla es (1,1,1) o (1,1,0), con el personaje en LOD1. Presupuesto mínimo: 105 000 triángulos.
3. (c) $\min \sum_{o,k} e_{ok} z_{ok}$ s.a. $\sum_k z_{ok} = 1$ para cada objeto, $\sum_{o,k} t_{ok} z_{ok} \le B$, $z_{\text{personaje},k} = 0$ para $k \ge 1$, $z_{ok} \in \{0,1\}$.

Resultado. **sin regla: error** 2,4**; con regla:** 3,3**; la regla es gratuita desde 105 000 triángulos.**

Comprobación. (0,2,1): $60 + 12 + 15 = 87 \le 100$ y $0 + 2{,}5 + 0{,}8 = 3{,}3$.

Solución del ejercicio 6.7

Datos. planta 1: 81/87 y 234/270; planta 2: 192/263 y 55/80.

Planteamiento. tasas por segmento y globales.

Desarrollo.

1. (a) Planta 1: optimizador 93,1 %, anterior 86,7 %. Planta 2: 73,0 % y 68,8 %. Global: optimizador $273/350 = 78{,}0\,\%$, anterior $289/350 = 82{,}6\,\%$.
2. (b) El optimizador se evaluó sobre todo en la planta 2 (263 de 350 casos), donde las averías son más difíciles de evitar; la política anterior, sobre todo en la planta 1. La mezcla, no la calidad, invierte el resultado global: paradoja de Simpson.
3. (c) Presentar los resultados por segmento, con el tamaño de cada uno, y comparar con la misma mezcla (o con diseño pareado por planta).

Resultado. **el optimizador gana en cada planta y pierde en el total por la mezcla de casos.**

Comprobación. $81 + 192 = 273$ y $234 + 55 = 289$, ambos sobre 350.

Solución del ejercicio 6.8

Datos. almacén frigorífico de 20 000 m^3; tarifa horaria; placas solares; $-22 \leq T \leq -18$ ºC.

Planteamiento. recorrer el cuadro 6.1.

Desarrollo.

1. Decisión (1–2): consignas horarias de los compresores para las próximas 24 h; responsable: jefe de mantenimiento; puede pararlo el operador de turno.
2. Objetivo y requisitos (3): minimizar el coste eléctrico; restricción dura $-22 \leq T \leq -18$ ºC en todas las sondas de producto; indicadores: kWh/día, horas cerca del límite, arranques de compresor. Módulo 24, tema 1.
3. Línea base y techo (4): coste actual con termostato fijo; techo con información perfecta de precios y radiación. Primer número: coste diario actual.
4. Datos (5): sondas, contadores, precios, predicción de radiación; versionado. Módulo 28, tema 4.
5. Formulación (6): control predictivo con modelo térmico identificado (módulo 26, temas 2–3); programa lineal o cuadrático por hora.
6. Modelos (7): predicción de la generación solar y de las aperturas de puertas; validación temporal (módulo 8). Primer número: error de predicción de la generación a 24 h.
7. Salvaguardas (8): termostato existente como respaldo; filtro de seguridad de consignas (módulo 26, tema 6, §6.5).
8. Evaluación (9–10): gemelo térmico con números aleatorios comunes; piloto en sombra y después días alternos con tamaño calculado (proposición 5.5).
9. Despliegue, monitorización, valor y ciclo de vida (11–14): API e interfaz del operador; vigilancia de residuos del modelo térmico; VAN e intervalos (módulo 28, tema 5); revisión anual.

Resultado. **documento completo con un responsable, una restricción dura de temperatura, un MPC lineal, respaldo por termostato y un primer número por sección.**

Comprobación. cada sección de la plantilla queda respondida y enlazada con su módulo.

Bibliografía

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. Optuna: A next-generation hyperparameter optimization framework. En *Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2019. Documentación oficial; licencia MIT. https://optuna.org/

[2] R. E. Barlow y F. Proschan. *Mathematical Theory of Reliability*. SIAM, colección Classics in Applied Mathematics, 1996 (original: Wiley, 1965). Cap. 4, «Optimum Maintenance Policies».

[3] J. Bergstra e Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

[4] D. Bertsimas y J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997. Cap. 7, «Network flow problems».

[5] E. Breck, S. Cai, E. Nielsen, M. Salib y D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. En *IEEE International Conference on Big Data*, 2017.

[6] CMS. The EU's Digital Omnibus: streamlined AI Act compliance rules are now in force. Nota informativa, 2026 (publicación en el Diario Oficial el 24 de julio de 2026; entrada en vigor el 27 de julio de 2026). https://cms.law/en/gbr/publication/the-eu-s-digital-omnibus-streamlined-ai-act-compliance-rules-are-now-in-force

[7] Desarrolladores de DVC. DVC: Data Version Control. Documentación oficial; licencia Apache 2.0. https://dvc.org/doc

[8] Desarrolladores de Evidently. Evidently: evaluación y monitorización de modelos de aprendizaje automático. Documentación oficial; licencia Apache 2.0. https://docs.evidentlyai.com/

[9] Desarrolladores de FastAPI. FastAPI. Documentación oficial; licencia MIT. https://fastapi.tiangolo.com/

[10] Desarrolladores de MLflow. MLflow: plataforma para el ciclo de vida del aprendizaje automático. Documentación oficial; licencia Apache 2.0. https://mlflow.org/docs/latest/

[11] M. Dudík, J. Langford y L. Li. Doubly robust policy evaluation and learning. En *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011. https://arxiv.org/abs/1103.4601

[12] T. Elsken, J. H. Metzen y F. Hutter. Neural architecture search. Cap. 3 de [19], 2019.

[13] R. Evans y J. Gao. DeepMind AI reduces Google data centre cooling bill by 40 %. Blog de Google DeepMind, 20 de julio de 2016. https://deepmind.google/discover/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/

[14] Google DeepMind (equipo de AlphaEvolve). AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. Blog de Google DeepMind, 14 de mayo de 2025. https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/

[15] A. Fawzi y B. Romera-Paredes. FunSearch: Making new discoveries in mathematical sciences using large language models. Blog de Google DeepMind, 14 de diciembre de 2023 (con una actualización de 2024). https://deepmind.google/discover/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models/

[16] Google DeepMind. FunSearch: repositorio oficial (implementación del algoritmo evolutivo, cuaderno `bin_packing` con los conjuntos OR3 y Weibull 5k y las heurísticas descubiertas). Licencia Apache 2.0. https://github.com/google-deepmind/funsearch

[17] J. Herrmann y G. Pallez. An in-depth study of LLM contributions to the bin packing problem. Prepublicación, 2025. https://arxiv.org/abs/2510.27353

[18] C. Huyen. *Designing Machine Learning Systems*. O'Reilly, 2022. Texto asociado al curso CS329S de Stanford.

[19] F. Hutter, L. Kotthoff y J. Vanschoren (eds.). *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019. Edición de acceso abierto. Cap. 1 «Hyperparameter Optimization» (M. Feurer y F. Hutter) y cap. 3 «Neural Architecture Search». https://www.automl.org/book/

[20] IEC. IEC 60812:2018. *Failure modes and effects analysis (FMEA and FMECA)*, 3.ª ed. International Electrotechnical Commission, 2018. Se usa solo el resumen público. https://webstore.iec.ch/en/publication/26359

[21] K. Jamieson y A. Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. En *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016. https://arxiv.org/abs/1502.07943

[22] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia y C. Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. Prepublicación, 2023. Biblioteca con licencia MIT. https://arxiv.org/abs/2310.03714

[23] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Cap. 9 «Population Methods» y cap. 20 «Expression Optimization». https://algorithmsbook.com/optimization/

[24] R. Kohavi, D. Tang y Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020.

[25] The Kubernetes Authors. Kubernetes. Documentación oficial; licencia Apache 2.0. https://kubernetes.io/docs/

[26] L. Li y A. Talwalkar. Random search and reproducibility for neural architecture search. En *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2019. https://arxiv.org/abs/1902.07638

[27] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh y A. Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

[28] H. Liu, K. Simonyan e Y. Yang. DARTS: Differentiable architecture search. En *International Conference on Learning Representations (ICLR)*, 2019. https://arxiv.org/abs/1806.09055

[29] MIT OpenCourseWare. 1.010: Uncertainty in Engineering. Massachusetts Institute of Technology, otoño de 2008. *Example Application* 8, «Old better than new, new better than old…» (función de riesgo), problema 8.1. https://ocw.mit.edu/courses/1-010-uncertainty-in-engineering-fall-2008/resources/app_08/

[30] O. de Weck. 16.842: Fundamentals of Systems Engineering. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2015. Programa (modelo en V) y tareas A1 (partes interesadas, concepto de operación), A2 (requisitos y asignación de márgenes) y A4 (exploración de alternativas y selección del concepto). https://ocw.mit.edu/courses/16-842-fundamentals-of-systems-engineering-fall-2015/

[31] A. Lo. 15.401: Finance Theory I. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2008. Sesiones «Present Value Relations» y «Capital Budgeting». https://ocw.mit.edu/courses/15-401-finance-theory-i-fall-2008/

[32] C. Leiserson y J. Shun. 6.172: Performance Engineering of Software Systems. Massachusetts Institute of Technology, MIT OpenCourseWare, otoño de 2018. Clase 10, «Measurement and Timing» (transcripción). https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/

[33] D. C. Montgomery. *Design and Analysis of Experiments*, 9.ª ed. Wiley, 2017. Cap. 2, «Simple Comparative Experiments».

[34] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner *et al.* AlphaEvolve: A coding agent for scientific and algorithmic discovery. Prepublicación, Google DeepMind, 2025. https://arxiv.org/abs/2506.13131

[35] Desarrolladores de OpenEvolve. OpenEvolve: implementación abierta inspirada en AlphaEvolve. Repositorio oficial; licencia Apache 2.0. https://github.com/algorithmicsuperintelligence/openevolve

[36] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia y O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. En *Proceedings of EMNLP*, 2024. https://arxiv.org/abs/2406.11695

[37] J. Orloff y J. Bloom. 18.05: Introduction to Probability and Statistics. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2022. Lectura 24, «Bootstrap confidence intervals». https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

[38] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le y J. Dean. Efficient neural architecture search via parameter sharing. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. https://arxiv.org/abs/1802.03268

[39] C. Potts. CS224U: Natural Language Understanding. Stanford University, primavera de 2023. Tarea «Few-shot OpenQA with DSPy». https://web.stanford.edu/class/cs224u/

[40] P. Moritz, R. Nishihara, S. Wang *et al.* Ray: A distributed framework for emerging AI applications. En *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. Documentación de Ray Tune y Ray Serve; licencia Apache 2.0. https://docs.ray.io/

[41] E. Real, A. Aggarwal, Y. Huang y Q. V. Le. Regularized evolution for image classifier architecture search. En *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019. https://arxiv.org/abs/1802.01548

[42] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). *Diario Oficial de la Unión Europea*, 12 de julio de 2024. Texto consultado, con sus modificaciones, en https://artificialintelligenceact.eu/ (artículos 6, 14 y 113; anexos I y III).

[43] R. T. Rockafellar y S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

[44] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli y A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[45] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo y D. Dennison. Hidden technical debt in machine learning systems. En *Advances in Neural Information Processing Systems 28 (NIPS)*, 2015.

[46] C. Huyen y equipo docente de CS329S. CS329S: Machine Learning Systems Design. Stanford University, invierno de 2022. Clases 1 «Understanding machine learning production», 2 «ML and Data Systems Fundamentals», 6 «Offline evaluation», 8 «Deployment» y 10 «Diagnosis of ML system failures, data distribution shifts & monitoring», con notas de clase. https://stanford-cs329s.github.io/syllabus.html

[47] J. E. Smith y R. L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[48] C. Thornton, F. Hutter, H. H. Hoos y K. Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. En *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.

[49] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery y D. Zhou. Self-consistency improves chain of thought reasoning in language models. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2203.11171

[50] D. P. Williamson y D. B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011. Edición electrónica libre. §3.3, «The bin-packing problem». https://www.designofapproxalgs.com/

[51] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou y X. Chen. Large language models as optimizers. Prepublicación, 2023 (versión 3, 2024). https://arxiv.org/abs/2309.03409

[52] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §19.1 «What Is Hyperparameter Optimization?» y §19.4 «Multi-Fidelity Hyperparameter Optimization». https://d2l.ai/

[53] B. Zoph y Q. V. Le. Neural architecture search with reinforcement learning. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1611.01578